

Visualization 101 2026

Data Visualization In R

Boris Shor

2026-05-18

Table of contents

1	Visualization 101 2026	5
1.1	Schedule	5
1.2	Setup	6
1.3	Files	6
2	Start Here: Workflow, Quarto, And RStudio	7
2.1	Learning Goals	7
2.2	The Course Workflow	7
2.3	Plain R Scripts	7
2.4	The Console	8
2.5	R, RStudio, Posit Cloud, And Quarto	8
2.6	Project Files And Portable Paths	9
2.7	The RStudio Layout	9
2.8	Source Mode And Visual Mode	9
2.9	Markdown Basics	10
2.10	Code Chunks	10
2.11	Chunk Names And Navigation	11
2.12	Chunk Options	11
2.13	Rendering	12
2.14	A First Reproducible Example	12
2.15	Reading Output Carefully	13
2.16	Short Exercise	13
2.17	Extras: RStudio Settings	14
2.18	Extras: Output Formats	14
	2.18.1 Presentation Demo	15
	2.18.2 Paper Or Report Demo	15
2.19	What Comes Next	16
3	Data Import And The Tidyverse	17
3.1	Learning Goals	17
3.2	Load Packages	17
3.3	Importing Local Data	17
3.4	Tibbles And First Inspection	18
3.5	Base R Import And Inspection	20

3.6	Other Local File Types	20
3.6.1	Excel	20
3.6.2	Stata	20
3.6.3	SPSS	21
3.6.4	SAS	21
3.7	Wide And Long Data	21
3.8	Core Tidyverse Verbs	22
3.9	<code>filter()</code> : Keep Rows	23
3.10	<code>select()</code> : Keep Columns	24
3.11	<code>mutate()</code> : Create Variables	24
3.12	<code>group_by()</code> And <code>summarize()</code> : Aggregate Data	25
3.13	Readable Pipelines	26
3.14	Joins	27
3.15	Short Exercise	27
3.16	Extras: Google Sheets	28
3.17	Census Data With <code>tidycensus</code>	28
3.18	Extras: Qualtrics And Canvas	31
3.19	What Comes Next	31
4	Core ggplot2	32
4.1	Learning Goals	32
4.2	Load Packages	32
4.3	Data For This Chapter	32
4.4	Comparing Base R Graphics And <code>ggplot2</code>	33
4.5	The Grammar Of Graphics	34
4.6	The Base Layer: <code>ggplot()</code> And <code>aes()</code>	35
4.7	Add A Geom Layer	35
4.8	Saving A Plot Object	36
4.9	Mapping Versus Setting	37
4.10	Smoothers	40
4.11	Log Scales	42
4.12	Labels With <code>labs()</code>	43
4.13	Themes	44
4.14	Color Palettes	47
4.15	Putting It All Together	51
4.16	Short Exercise	52
4.17	Extras: Continuous Color	53
4.18	Extras: Shape And Size	54
4.19	Extras: External Themes And Rich Text	55
4.20	What Comes Next	58
5	Separating and Comparing Data	59
5.1	Why separate data?	59

5.2	Start with one plot	59
5.3	Color as separation	60
5.4	Facets as small multiples	62
5.5	Reordering facets	63
5.6	Free scales	64
5.7	Direct labels	65
5.8	<code>facet_grid()</code>	67
5.9	Patchwork	68
5.10	Extra: State Policy Snapshot	69
5.11	Extra: PISA international scores	74
5.12	Exercise	78
6	Distributions	79
6.1	Why distributions matter	79
6.2	Histograms	79
6.3	Comparing histograms	82
6.4	Density plots	83
6.5	Boxplots	86
6.6	Ridge plots	88
6.7	Distributions in Gapminder	89
6.8	Choosing among histograms, densities, and boxplots	92
6.9	Exercise 1: histogram	92
6.10	Exercise 2: density plot	93
6.11	Exercise 3: boxplot	94
6.12	Extra: distribution intervals with <code>ggdist</code>	94
6.13	Extra: worked exercise variants	95
6.14	Extra: PISA score distributions	99
6.15	Extra: heatmaps for dense two-variable distributions	101
7	Lines, Bars, and Annotation	103
7.1	Line charts	103
7.2	Working with dates	106
7.3	Highlighting trends	109
7.4	Bar charts	113
7.5	<code>geom_col()</code> versus <code>geom_bar()</code>	116
7.6	Dodged bars	117
7.7	Bar chart example: OECD life expectancy gap	119
7.8	Exercise	120
7.9	Extra: slopegraphs	121

1 Visualization 101 2026

This book is the working text for a five-day course on data visualization in R. It is designed to be read, run, edited, rendered, and reused. The goal is not only to make attractive graphs, but to make visual choices that clarify an argument, reveal a pattern, and leave a reproducible record of how the result was made.

Visualization is a sequence of choices. A plot emphasizes some comparisons and pushes others into the background. It chooses what counts as the main unit, what deserves color or space, what should be labeled directly, what can remain implicit, and what should be excluded because it distracts from the point. The course therefore treats visualization as part of analysis rather than as decoration added at the end.

R is the statistical language used throughout the course. RStudio Desktop and Posit Cloud are the interfaces used to write and run code. The tidyverse is the central programming framework: it supplies a readable grammar for importing, reshaping, summarizing, and plotting data. `ggplot2`, the tidyverse visualization package, is the main plotting system.

Quarto is the document system used for the course files. A Quarto document can contain prose, code, output, figures, tables, citations, and links in one place. The same source document can produce HTML, PDF, slides, dashboards, and reports. When the data or code changes, the outputs can be regenerated from the source instead of manually updated across Word documents, spreadsheets, and slide decks.

Literate programming also supports replication. Code and interpretation live together, which makes it easier to return to a project after a long break, share work with collaborators, and identify exactly how a result was produced. Writing the explanation forces decisions to be made explicit while they are still fresh.

1.1 Schedule

Day	Main Focus	Chapters
1	Workflow, Quarto, data import, and an introduction to <code>ggplot2</code>	2–3, start 4
2	Core <code>ggplot2</code> , separating and comparing data, distributions	finish 4, 5–6

Day	Main Focus	Chapters
3	Lines, bars, annotation, and from question to final figure	7–8
4	Interactivity, dashboards, and tables	9–10
5	Model visualization, mapping, and agentic AI	11–13

Each day has five instructional sessions. The written chapters contain more material than the minimum live path so that faster pacing, review, and post-course study are supported.

1.2 Setup

The Posit Cloud project should already include the packages needed for the main course. The file `install_packages.R` automates installation if the materials are moved to a local computer. The file `install_extra_packages.R` installs optional packages used only in extra sections.

Run `check_setup.R` to verify that the main packages and data files are available.

1.3 Files

The main book chapters are the numbered `.qmd` files in this project. Data files are under `Data/`. Standalone examples are under `Demos/`. Short exercises and solutions are under `Exercises/`.

The course examples use files included with the project, such as `Data/gapminder/gapminder.csv`.

2 Start Here: Workflow, Quarto, And RStudio

2.1 Learning Goals

By the end of this chapter, the main goals are:

- open and navigate the course project
- understand the relationship among R, RStudio, Posit Cloud, and Quarto
- run code chunks inside a `.qmd` file
- render a Quarto document to HTML
- use source mode, visual mode, and the document outline intentionally

2.2 The Course Workflow

This course uses R for computation, RStudio for the working interface, and Quarto for the documents that combine prose, code, and output. The goal is not only to make individual plots. The goal is to build a reproducible workflow where the text explains the question, the code performs the work, and the output records what happened.

During class, most work will happen inside `.qmd` files like this one. A Quarto file is a plain-text document that can contain headings, paragraphs, lists, links, code chunks, tables, figures, and notes about interpretation. When Quarto renders the file, it runs the code chunks and combines the results with the surrounding text.

This is different from working only in the console. The console is useful for quick tests, but it does not automatically preserve the reasoning around the command. A Quarto document keeps the analysis in order: what question was being asked, what data were used, what code was run, what output appeared, and what conclusion followed.

2.3 Plain R Scripts

The simplest kind of R source file is a plain R script. A plain R script uses the `.R` extension and contains only R code. It is useful for setup tasks, repeated checks, helper functions, and other code that does not need surrounding prose or rendered output.

This project includes a few plain R scripts:

```
install_packages.R  
install_extra_packages.R  
check_setup.R
```

The file `check_setup.R` is a good example. It checks whether the main packages and course data files are available. Opening that file and clicking **Source** runs the whole script. The result is a quick setup check, not a rendered document.

Use `.R` scripts when the goal is to run code directly. Use `.qmd` files when the goal is to combine explanation, code, and output.

2.4 The Console

The console is the fastest place to test a command, inspect an object, or rerun something without rendering a document.

Two console habits are especially helpful:

- the up arrow cycles through recent commands
- `Ctrl + R` searches command history in many RStudio setups

The console is not a replacement for a source file. If a command becomes part of the analysis, move it into a `.qmd` or `.R` file so it is preserved.

2.5 R, RStudio, Posit Cloud, And Quarto

These tools have different jobs:

- R is the programming language that reads data, transforms it, estimates models, and makes graphics.
- RStudio is the interface where you edit files, run code, inspect objects, view plots, and render documents.
- Posit Cloud is a browser-based version of the RStudio workflow that avoids many local installation problems.
- Quarto is the publishing system that turns `.qmd` source files into HTML, PDF, Word, slides, dashboards, and books.

R Markdown was the predecessor used in earlier versions of this course. Quarto keeps the same basic idea, but it is now the preferred format for this version of the workshop. If you have used `.Rmd` files before, the transition should feel familiar: a `.qmd` file still mixes text and code. The main difference is that Quarto uses a more general and consistent document system.

2.6 Project Files And Portable Paths

The examples in this course assume that the project has been opened in RStudio or Posit Cloud. Files are organized inside the project in folders such as `Data/`, `Demos/`, and `Exercises/`.

The data examples use file names like these:

```
Data/gapminder/gapminder.csv
Data/penguins/penguins.csv
Demos/presentation-demo.qmd
Exercises/04-scatterplot-solution.R
```

Avoid file names that depend on a personal Desktop, Downloads folder, or another location on one person's computer.

Keeping files inside the project matters because the course should work in Posit Cloud, on another computer, or after being archived for later use.

2.7 The RStudio Layout

RStudio usually has four main panes:

- the source pane, where `.qmd` and `.R` files are edited
- the console, where commands can be typed and run directly
- the environment and history pane, where current objects and previous commands are listed
- the files, plots, packages, help, and viewer pane

The exact pane arrangement can be changed under Global Options. The important habit is to know which pane is doing which job. The source pane is for durable work. The console is for quick tests. The environment pane shows what objects currently exist. The viewer pane is often where rendered HTML output appears.

2.8 Source Mode And Visual Mode

RStudio can edit Quarto documents in source mode or visual mode.

Source mode shows the underlying Markdown and code exactly as written. It is the most transparent mode for learning because it makes headings, chunks, chunk options, links, and lists visible.

Visual mode is a more word-processor-like interface. It can be helpful for editing prose, tables, footnotes, and links. It is still editing the same `.qmd` file, but it hides some of the raw syntax while you write.

Both modes are useful. During this course, source mode is usually the clearest mode for understanding how the document works. Visual mode can be helpful later when revising narrative text.

2.9 Markdown Basics

Quarto uses Markdown for ordinary text formatting. A few patterns cover most needs:

```
# First-level heading

## Second-level heading

Regular paragraph text.

- a bullet point
- another bullet point

**bold text**

*italic text*

[Quarto website](https://quarto.org)
```

The number of `#` symbols controls the heading level. Headings also create the document outline, which makes long lessons easier to navigate.

2.10 Code Chunks

An R code chunk starts with three backticks and `{r}`. A useful chunk also has a short name after `r`, then ends with three backticks.

In the source file, the opening line would be ````{r ch02-code-chunks-1}` and the closing line would be `````. The R code between those lines could be:

```
1 + 1
```

The code inside the chunk can be run interactively, and it can also be run during rendering. In RStudio, you can run a chunk with the green play button or with keyboard shortcuts. You can insert a new chunk from the menu or by typing the chunk delimiters yourself.

If that chunk runs, the result would be:

```
[1] 2
```

Chunks can contain more than one line. For example:

```
x <- 10
y <- 25
x + y
```

The assignment operator `<-` stores a value in an object. In the example above, `x` stores 10 and `y` stores 25. After those objects exist, R can use them in later commands.

2.11 Chunk Names And Navigation

Most real code chunks in these course files have a name. The name appears in the opening line of the chunk. For example, this opening line names a chunk `ch02-first-calculation`:

```
# opening line in the source file:
# ```{r ch02-first-calculation}
```

Chunk names make long documents easier to navigate. In RStudio, the document outline can show both section headings and code chunks, so a descriptive chunk name makes it easier to jump to a specific part of the file. Chunk names also make error messages easier to interpret because Quarto can report which chunk failed.

Chunk names should be short, unique, and descriptive. A name such as `ch04-first-scatterplot` is more useful than a name such as `chunk1`.

2.12 Chunk Options

Chunk options control how code behaves during rendering. In Quarto, options are often written as special comment lines at the top of the chunk.

This chunk runs but hides the code:

```
[1] "2026-05-15"
```

This chunk displays code without running it:

```
Sys.Date()
```

`eval: false` is important for examples that require credentials, internet access, private files, or intentional manual action. The code remains visible as a template, but it does not run automatically.

2.13 Rendering

Rendering turns a source `.qmd` file into an output document. For this course, HTML is the primary output because it is easy to view in a browser and works well for code, figures, tables, and links.

When you render a document, Quarto starts from the top, runs the chunks in order, and inserts the output where it belongs. If a chunk fails, rendering stops. That is useful because it exposes broken code early.

The render button in RStudio is the easiest way to start. Open the file and click **Render** to create the output document. PDF output is also useful for papers, handouts, and print-ready reports, but it depends on a working TeX installation. HTML will be the most reliable output during the workshop.

2.14 A First Reproducible Example

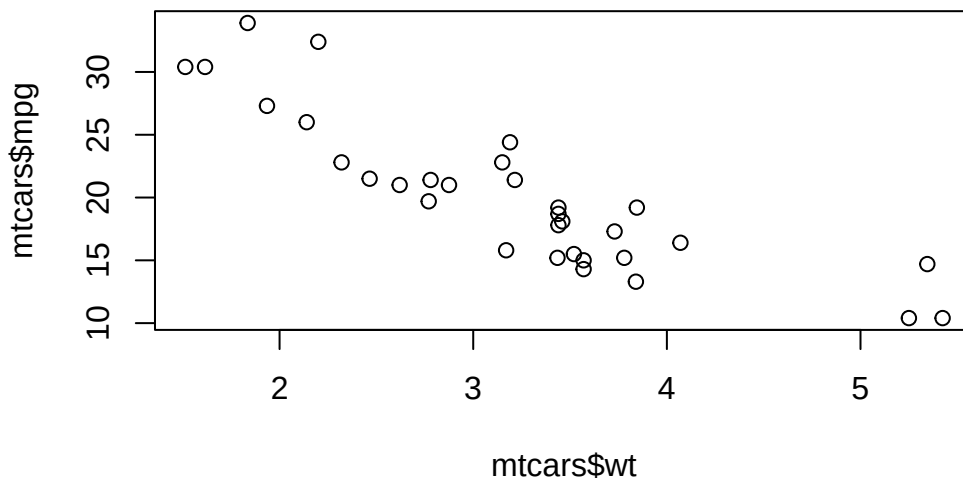
The built-in `mtcars` dataset is available in base R, so it is useful for a first test that does not depend on course data files.

```
head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

This command creates a simple base R plot:

```
plot(mtcars$wt, mtcars$mpg)
```



The plot is not polished, but it confirms that R is running, code chunks are executing, and graphical output appears in the document.

2.15 Reading Output Carefully

A rendered document is not automatically correct just because it renders. Rendering answers one question: did the code run from top to bottom? You still have to ask whether the data are the right data, whether the variables mean what you think they mean, whether missing values were handled correctly, and whether the plot supports the claim being made.

This course will return to that distinction throughout the week. Reproducibility is necessary, but it is not enough. A reproducible mistake is still a mistake.

2.16 Short Exercise

Create a new code chunk below this paragraph. In that chunk:

1. store the number 100 in an object named `start_value`
2. store the number 25 in an object named `change_value`
3. subtract `change_value` from `start_value`

```
# Write your code here.
```

2.17 Extras: RStudio Settings

A few RStudio settings can make workshop work more comfortable:

- turn off restoring `.RData` into the workspace at startup
- set saving the workspace on exit to “Never”
- turn on rainbow parentheses if available
- use the document outline for long `.qmd` files
- choose a source editor font and size that are comfortable for several hours of reading

These settings do not change the analysis, but they reduce friction. The main principle is that the project files should define the work, not a hidden workspace image from a previous session.

2.18 Extras: Output Formats

Quarto can produce many output formats from similar source files:

- HTML documents and books
- PDF reports
- Word documents
- RevealJS slides
- dashboards
- websites

The same analysis can therefore be adapted for different audiences. A rough exploration may become a class exercise. A refined figure may become part of a report. A report may become a short presentation. Quarto is useful because those outputs can remain connected to the same source code and data workflow.

2.18.1 Presentation Demo

Quarto can render slides. A presentation file uses a different YAML header:

```
---
title: "Presentation Demo"
format:
  revealjs:
    theme: simple
---
```

The course includes a standalone revealjs demo at `Demos/presentation-demo.qmd`. The main difference from a book chapter is that second-level headings become slides. Code chunks work the same way, but slides usually hide more code and show more finished output.

```
## Slide Title

Slide text goes here.
```

2.18.2 Paper Or Report Demo

A paper-style Quarto document emphasizes prose, citations, figures, tables, and cross-references. The course includes `Demos/paper-example.qmd` and `Demos/paper-example.bib`. The `.bib` file stores citation records; the `.qmd` file cites them with keys such as `[@wickham_ggplot2_2016]`.

A minimal paper-style header looks like this:

```
---
title: "Paper Demo"
author: "Name"
bibliography: paper-example.bib
format:
  html: default
  pdf: default
---
```

Paper-style files usually keep more prose visible than slides or dashboards. They also make more use of citations, captions, footnotes, and cross-references.

2.19 What Comes Next

The next chapter begins the data workflow: importing files, inspecting tibbles, reshaping data, and using the core tidyverse verbs that prepare data for visualization.

3 Data Import And The Tidyverse

3.1 Learning Goals

By the end of this chapter, the main goals are:

- read local data included with the project
- recognize the difference between a data frame and a tibble
- inspect a dataset before plotting it
- distinguish wide data from long data
- reshape data with `pivot_longer()` and `pivot_wider()`
- use the core tidyverse verbs for common data preparation tasks
- recognize which import examples should not run automatically because they require credentials or external services

3.2 Load Packages

The **tidyverse** is a collection of packages for importing, transforming, and visualizing data. Loading it makes packages such as **readr**, **dplyr**, **tidyr**, **ggplot2**, **stringr**, and **tibble** available.

```
library(tidyverse)
library(skimr)
```

3.3 Importing Local Data

Many projects begin with a local CSV file. CSV files are plain-text tables where columns are separated by commas. They are common because nearly every spreadsheet, statistical package, and database can export them.

The preferred tidyverse function is `read_csv()` from the **readr** package. Compared with base R's `read.csv()`, it usually prints more clearly, guesses column types more consistently, and returns a tibble.

The course data live under `Data/`. The example below reads the Gapminder CSV file included with the project.

```
gap <- read_csv("Data/gapminder/gapminder.csv", show_col_types = FALSE)
```

```
gap
```

```
# A tibble: 1,704 x 6
  country      continent year lifeExp      pop gdpPercap
  <chr>        <chr>    <dbl>  <dbl>    <dbl>    <dbl>
1 Afghanistan Asia      1952   28.8  8425333    779.
2 Afghanistan Asia      1957   30.3  9240934    821.
3 Afghanistan Asia      1962   32.0 10267083    853.
4 Afghanistan Asia      1967   34.0 11537966    836.
5 Afghanistan Asia      1972   36.1 13079460    740.
6 Afghanistan Asia      1977   38.4 14880372    786.
7 Afghanistan Asia      1982   39.9 12881816    978.
8 Afghanistan Asia      1987   40.8 13867957    852.
9 Afghanistan Asia      1992   41.7 16317921    649.
10 Afghanistan Asia      1997   41.8 22227415    635.
# i 1,694 more rows
```

The object name `gap` now points to a table in memory. The assignment operator `<-` reads as “store the result on the right in the object named on the left.” Object names should be short enough to type but descriptive enough to understand later.

3.4 Tibbles And First Inspection

A tibble is the tidyverse version of a data frame. It is still a rectangular table with rows and columns, but it prints in a more controlled way. A tibble tells you the number of rows and columns, shows variable types, and avoids flooding the console with every row of a large dataset.

Start every new dataset with a quick inspection:

```
glimpse(gap)
```

```
Rows: 1,704
Columns: 6
$ country    <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", ~
```

```
$ continent <chr> "Asia", "Asia", "Asia", "Asia", "Asia", "Asia", "Asia", "Asi~
$ year      <dbl> 1952, 1957, 1962, 1967, 1972, 1977, 1982, 1987, 1992, 1997, ~
$ lifeExp   <dbl> 28.801, 30.332, 31.997, 34.020, 36.088, 38.438, 39.854, 40.8~
$ pop       <dbl> 8425333, 9240934, 10267083, 11537966, 13079460, 14880372, 12~
$ gdpPercap <dbl> 779.4453, 820.8530, 853.1007, 836.1971, 739.9811, 786.1134, ~
```

```
skim(gap)
```

Table 3.1: Data summary

Name	gap
Number of rows	1704
Number of columns	6
Column type frequency:	
character	2
numeric	4
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
country	0	1	4	24	0	142	0
continent	0	1	4	8	0	5	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
year	0	1	1979.50	17.27	1952.00	1965.75	1979.50	1993.25	2007.0	
lifeExp	0	1	59.47	12.92	23.60	48.20	60.71	70.85	82.6	
pop	0	1	29601212336	157896670	11100793664	7023595105	852211751	8683096.0		
gdpPer-cap	0	1	7215.33	9857.45	241.17	1202.06	3531.85	9325.46	113523.1	

`glimpse()` comes from `dplyr`, which is loaded as part of the tidyverse. It gives a compact structural view of the data. `skim()` comes from the `skimr` package and gives a broader summary, including missingness and descriptive statistics. These checks are not just housekeeping. They help identify the unit of observation, variable types, unexpected missing values, and suspicious ranges before those problems enter a plot.

3.5 Base R Import And Inspection

Base R can also import data. The syntax is common in older examples and in some online answers:

```
gap_base <- read.csv("Data/gapminder/gapminder.csv")
str(gap_base)
head(gap_base)
```

3.6 Other Local File Types

CSV is only one possible format. Social science projects often use Excel, Stata, SPSS, or SAS files. The syntax below should be treated as a template. These chunks are non-executing unless the named files are present and the relevant packages are installed.

3.6.1 Excel

The function `read_excel()` comes from the `readxl` package.

```
library(readxl)

gap_excel <- read_excel(
  path = "Data/gapminder/gapminder.xlsx",
  sheet = "gapminder"
)

gap_excel
```

3.6.2 Stata

The functions for Stata, SPSS, and SAS files below come from the `haven` package.

```
library(haven)

state_legislatures <- read_dta("Data/state_policy/shor_mccarty_state_data.dta")

state_legislatures
```

3.6.3 SPSS

The example uses a small SPSS file that is installed with the **haven** package.

```
library(haven)

path <- system.file("examples", "iris.sav", package = "haven")
survey_data <- read_sav(path)

survey_data
```

3.6.4 SAS

The example uses a small SAS file that is installed with the **haven** package.

```
library(haven)

path <- system.file("examples", "iris.sas7bdat", package = "haven")
admin_data <- read_sas(path)

admin_data
```

3.7 Wide And Long Data

Data can be organized in more than one shape. A *wide* dataset has one row per unit and a separate column for each measurement (`gdpPercap`, `lifeExp`). A *long* dataset stacks the measurement names into one column and the values into another. Wide data is often easier for people to read; long data is often easier for `ggplot2` and group-wise summaries because the measurement name itself can be mapped to color, facets, or groups.

`pivot_longer()` and `pivot_wider()` from `tidyr` move data between the two shapes.

```
wide_data <- tibble(
  country = c("Afghanistan", "Brazil"),
  year = c(2007, 2007),
  gdpPercap = c(974.6, 9065.8),
  lifeExp = c(43.8, 72.4)
)

long_data <- wide_data |>
```

```

pivot_longer(
  cols = c(gdpPercap, lifeExp),
  names_to = "variable",
  values_to = "value"
)

long_data

```

```

# A tibble: 4 x 4
  country      year variable  value
  <chr>      <dbl> <chr>    <dbl>
1 Afghanistan 2007 gdpPercap 975.
2 Afghanistan 2007 lifeExp  43.8
3 Brazil       2007 gdpPercap 9066.
4 Brazil       2007 lifeExp  72.4

```

```

long_data |>
  pivot_wider(names_from = variable, values_from = value)

```

```

# A tibble: 2 x 4
  country      year gdpPercap lifeExp
  <chr>      <dbl>    <dbl>    <dbl>
1 Afghanistan 2007      975.     43.8
2 Brazil       2007     9066.     72.4

```

The pipe operator `|>` means “take the result so far and send it into the next function.” It lets a sequence of data steps read in the same order as the logic of the analysis.

3.8 Core Tidyverse Verbs

Most data preparation before plotting uses a small set of verbs:

- `filter()` keeps rows
- `select()` keeps columns
- `mutate()` creates or modifies variables
- `group_by()` defines groups
- `summarize()` collapses rows into summaries
- joins combine information from two tables

These verbs come from `dplyr`, which is loaded as part of the tidyverse. They are useful because they correspond to ordinary analytical questions. Which cases belong in this plot? Which variables matter? Do we need a new rate, logged value, or category? What is the summary by group? Does another table contain information that should be merged in?

3.9 `filter()`: Keep Rows

`filter()` keeps observations that satisfy logical conditions. The operator `==` tests exact equality. The operator `%in%` tests whether a value belongs to a set.

```
gap |>
  filter(year == 2007)
```

```
# A tibble: 142 x 6
  country    continent  year lifeExp      pop gdpPercap
  <chr>      <chr>      <dbl> <dbl>    <dbl> <dbl>
1 Afghanistan Asia      2007   43.8  31889923   975.
2 Albania    Europe    2007   76.4   3600523  5937.
3 Algeria    Africa    2007   72.3  33333216  6223.
4 Angola     Africa    2007   42.7  12420476  4797.
5 Argentina  Americas  2007   75.3  40301927 12779.
6 Australia  Oceania   2007   81.2  20434176 34435.
7 Austria    Europe    2007   79.8   8199783  36126.
8 Bahrain    Asia      2007   75.6   708573   29796.
9 Bangladesh Asia      2007   64.1 150448339   1391.
10 Belgium   Europe    2007   79.4  10392226 33693.
# i 132 more rows
```

```
gap |>
  filter(continent %in% c("Americas", "Europe"))
```

```
# A tibble: 660 x 6
  country    continent  year lifeExp      pop gdpPercap
  <chr>      <chr>      <dbl> <dbl>    <dbl> <dbl>
1 Albania Europe    1952   55.2  1282697   1601.
2 Albania Europe    1957   59.3  1476505   1942.
3 Albania Europe    1962   64.8  1728137   2313.
4 Albania Europe    1967   66.2  1984060   2760.
5 Albania Europe    1972   67.7  2263554   3313.
6 Albania Europe    1977   68.9  2509048   3533.
```

```

7 Albania Europe      1982    70.4 2780097    3631.
8 Albania Europe      1987    72   3075321    3739.
9 Albania Europe      1992    71.6 3326498    2497.
10 Albania Europe     1997    73.0 3428038    3193.
# i 650 more rows

```

Use `==` for one value. Use `%in%` when several values are acceptable.

3.10 `select()`: Keep Columns

`select()` keeps variables. This is useful when a dataset has many columns but the next step only needs a few.

```

gap |>
  select(country, continent, year, lifeExp, gdpPercap)

```

```

# A tibble: 1,704 x 5
  country      continent year lifeExp gdpPercap
  <chr>        <chr>    <dbl>  <dbl>    <dbl>
1 Afghanistan Asia      1952    28.8     779.
2 Afghanistan Asia      1957    30.3     821.
3 Afghanistan Asia      1962    32.0     853.
4 Afghanistan Asia      1967    34.0     836.
5 Afghanistan Asia      1972    36.1     740.
6 Afghanistan Asia      1977    38.4     786.
7 Afghanistan Asia      1982    39.9     978.
8 Afghanistan Asia      1987    40.8     852.
9 Afghanistan Asia      1992    41.7     649.
10 Afghanistan Asia      1997    41.8     635.
# i 1,694 more rows

```

Rows are observations. Columns are variables. `filter()` works on rows; `select()` works on columns.

3.11 `mutate()`: Create Variables

`mutate()` creates new variables or rewrites existing ones. In the next example, `round()` makes GDP per capita easier to read, and `log10()` creates a transformed variable that will be useful for plotting.


```
gap |>
  mutate(
    gdp_pc_round = round(gdpPercap),
    log_gdp_pc = log10(gdpPercap)
  ) |>
  select(country, gdpPercap, gdp_pc_round, log_gdp_pc)
```

```
# A tibble: 1,704 x 4
  country      gdpPercap gdp_pc_round log_gdp_pc
  <chr>      <dbl>      <dbl>      <dbl>
1 Afghanistan  779.         779         2.89
2 Afghanistan  821.         821         2.91
3 Afghanistan  853.         853         2.93
4 Afghanistan  836.         836         2.92
5 Afghanistan  740.         740         2.87
6 Afghanistan  786.         786         2.90
7 Afghanistan  978.         978         2.99
8 Afghanistan  852.         852         2.93
9 Afghanistan  649.         649         2.81
10 Afghanistan 635.         635         2.80
# i 1,694 more rows
```

`round(gdpPercap)` rounds each value in the `gdpPercap` column. `log10(gdpPercap)` takes the base-10 logarithm. The original variable remains unless you explicitly overwrite it.

3.12 `group_by()` And `summarize()`: Aggregate Data

Many plots need data at a different level than the imported file. For example, the Gapminder data are country-year observations, but we may want a continent-level summary.

```
continent_summary <- gap |>
  group_by(continent) |>
  summarize(
    median_life_exp = median(lifeExp, na.rm = TRUE),
    median_gdp_pc = median(gdpPercap, na.rm = TRUE),
    n_countries = n_distinct(country)
  )

continent_summary
```

```
# A tibble: 5 x 4
  continent median_life_exp median_gdp_pc n_countries
  <chr>          <dbl>          <dbl>          <int>
1 Africa          47.8          1192.           52
2 Americas        67.0          5466.           25
3 Asia            61.8          2647.           33
4 Europe          72.2          12082.          30
5 Oceania         73.7          17983.           2
```

`na.rm = TRUE` tells functions such as `median()` to ignore missing values. Without it, one missing value can make a summary missing.

3.13 Readable Pipelines

Long pipelines are easier to read when each verb gets its own line and arguments are indented consistently.

```
gap |>
  filter(year == 2007) |>
  mutate(gdp_pc_round = round(gdpPercap)) |>
  group_by(continent) |>
  summarize(
    median_life_exp = median(lifeExp, na.rm = TRUE),
    median_gdp_pc = median(gdp_pc_round, na.rm = TRUE)
  ) |>
  arrange(desc(median_life_exp))
```

```
# A tibble: 5 x 3
  continent median_life_exp median_gdp_pc
  <chr>          <dbl>          <dbl>
1 Oceania          80.7          29810
2 Europe           78.6          28054
3 Americas         72.9           8948
4 Asia             72.4           4471
5 Africa           52.9           1452
```

The line breaks are not just style. They make the data workflow visible. Each line is one transformation.

3.14 Joins

Joins combine two tables using one or more key variables. A common pattern is to keep all rows from the main table and add matching columns from a smaller lookup table. That is a `left_join()`, another `dplyr` function.

```
region_notes <- tibble(
  continent = c("Africa", "Americas", "Asia", "Europe", "Oceania"),
  example_note = c(
    "Many countries, large range of outcomes",
    "Includes North, Central, and South America",
    "Large population range",
    "High median life expectancy in many examples",
    "Small number of countries in many teaching datasets"
  )
)

continent_summary |>
  left_join(region_notes, by = "continent")
```

```
# A tibble: 5 x 5
  continent median_life_exp median_gdp_pc n_countries example_note
  <chr>          <dbl>          <dbl>      <int> <chr>
1 Africa          47.8          1192.         52 Many countries, large ran~
2 Americas        67.0          5466.         25 Includes North, Central, ~
3 Asia           61.8          2647.         33 Large population range
4 Europe          72.2         12082.         30 High median life expectan~
5 Oceania         73.7         17983.          2 Small number of countries~
```

Rows without a match in the lookup table would remain, but the new variables would be NA. That behavior is usually safer than silently dropping observations.

3.15 Short Exercise

Use the built-in `mtcars` dataset. Write a pipeline that:

1. converts `mtcars` to a tibble with row names stored as a column named `car`
2. groups by `cyl`
3. calculates average `mpg` and maximum `wt`
4. returns one row per value of `cyl`

```
# Write your code here.
```

3.16 Extras: Google Sheets

Google Sheets can be useful when data are collaboratively maintained. These examples require authentication and should not run automatically in a rendered course book.

```
library(googlesheets4)

sheet_data <- read_sheet(
  ss = "PASTE-GOOGLE-SHEET-ID-OR-URL-HERE",
  sheet = "Sheet1"
)

sheet_data
```

```
library(googlesheets4)

ss <- gs4_create("Visualization 101 2026 Example Sheet")

mtcars |>
  as_tibble(rownames = "car") |>
  sheet_write(ss = ss, sheet = "mtcars")
```

3.17 Census Data With tidycensus

Some data sources are best accessed through specialized packages. These examples are templates, not automatically running chunks.

The `tidycensus` package wraps the U.S. Census Bureau's APIs. The two most common entry points are the American Community Survey (`get_acs()`) and the decennial census (`get_decennial()`). A free Census API key is required. Request one at https://api.census.gov/data/key_signup.html, then register it once on your machine with `census_api_key("YOUR_KEY", install = TRUE)`. After that, the key is available to every R session and the line can be removed from teaching files.

```
library(tidycensus)

#census_api_key("2c5ba48be91062db570c7fc5fa49dbca03306c33", install = TRUE)
```

`load_variables()` lists the variables available in a given dataset and year. Caching the result avoids re-downloading the table.

```
acs_vars <- load_variables(2020, "acs5", cache = TRUE) |>
  distinct(concept, .keep_all = TRUE)

acs_vars
```

If `load_variables()` errors with a JSON parse message like `invalid char in json text. <html>`, the Census Bureau returned an HTML error page instead of the variable table. That usually means the requested year is not yet published for that dataset, the Census server is temporarily down, or the request was rate-limited. Try a slightly older year (for ACS 5-year, two years behind the current year is usually safe), re-run the call after a minute, or check <https://www.census.gov/data/developers/data-sets.html> for the dataset's release schedule.

The `variables` argument of `get_acs()` accepts named variable codes. Naming the entries gives readable column names instead of raw codes like `B19013_001`. `output = "wide"` puts each variable in its own column, which is usually easier to plot from than the default long format.

```
acs_raw <- get_acs(
  geography = "congressional district",
  variables = c(
    median_income = "B19013_001",
    total_pop = "B01003_001"
  ),
  year = 2022,
  survey = "acs5",
  output = "wide"
)

acs_raw
```

The `GEOID` column is a concatenation of geographic identifiers. For congressional districts it combines the two-digit state FIPS code with the two-digit district number. Splitting it with `str_sub()` gives joinable columns.

```
acs <- acs_raw |>
  mutate(
    fips = str_sub(GEOID, 1, 2),
    cd = str_sub(GEOID, 3, 4)
  )
```

```
acs
```

State-level summaries are simpler because the `GE0ID` is just the state FIPS code. The wide format makes it easy to compare one variable against another.

```
acs_state <- get_acs(  
  geography = "state",  
  variables = c(  
    median_income = "B19013_001",  
    median_rent = "B25064_001"  
  ),  
  year = 2022,  
  survey = "acs5",  
  output = "wide"  
)  
  
acs_state |>  
  summarize(  
    n_states = n(),  
    cor_income_rent = cor(median_incomeE, median_rentE, use = "complete.obs")  
  )
```

Note that the wide variable names get an E suffix for estimates and an M suffix for the margin of error. Both are usually returned. For most teaching purposes the E columns are enough; in published work the M columns should be acknowledged.

The decennial census uses `get_decennial()` instead. It returns counts rather than estimates and does not need a `survey` argument.

```
pop_2020 <- get_decennial(  
  geography = "state",  
  variables = c(total_pop = "P1_001N"),  
  year = 2020,  
  sumfile = "p1"  
)  
  
pop_2020
```

3.18 Extras: Qualtrics And Canvas

Survey platforms and learning management systems are also possible sources of data. The main point is that API-based imports should be treated as retrieval steps, not as code that runs every time a teaching document renders.

```
library(qualtRics)

raw_survey <- fetch_survey(
  surveyID = "YOUR-SURVEY-ID",
  label = FALSE,
  convert = FALSE
)
```

```
library(rcanvas)

courses <- get_course_list()
```

3.19 What Comes Next

The next chapter uses these data ideas to build plots. `ggplot2` works best when the data are already in a clear rectangular form and the variables needed for the plot have been inspected, selected, transformed, or summarized.

4 Core ggplot2

4.1 Learning Goals

By the end of this chapter, the main goals are:

- compare base R graphics with `ggplot2`
- understand the grammar of graphics at a practical level
- build plots from `ggplot()`, `aes()`, and geoms
- use the `+` operator to build a visualization pipeline
- distinguish mapped aesthetics from fixed aesthetics
- add scatterplot layers, smoothers, log scales, and labels
- apply built-in themes and color palettes

4.2 Load Packages

`ggplot2` is part of the tidyverse. The `scales` package is useful for readable axis labels. `RColorBrewer` provides named color palettes.

```
library(tidyverse)
library(scales)
library(RColorBrewer)
```

4.3 Data For This Chapter

The main examples use the local Gapminder country-year dataset in `Data/gapminder/gapminder.csv`.

```
gap <- read_csv("Data/gapminder/gapminder.csv", show_col_types = FALSE)

gap_2007 <- gap |>
  filter(year == max(year, na.rm = TRUE))

gap_2007
```

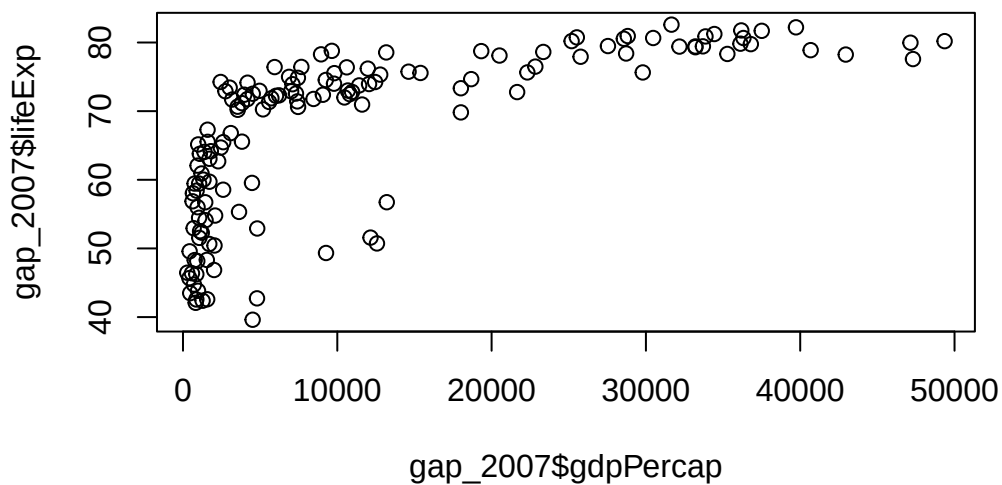


```
# A tibble: 142 x 6
  country    continent year lifeExp      pop gdpPercap
  <chr>      <chr>    <dbl>  <dbl>    <dbl>    <dbl>
1 Afghanistan Asia      2007   43.8  31889923    975.
2 Albania    Europe    2007   76.4   3600523   5937.
3 Algeria    Africa    2007   72.3  33333216   6223.
4 Angola     Africa    2007   42.7  12420476   4797.
5 Argentina  Americas  2007   75.3  40301927  12779.
6 Australia  Oceania   2007   81.2  20434176  34435.
7 Austria    Europe    2007   79.8   8199783  36126.
8 Bahrain    Asia      2007   75.6    708573  29796.
9 Bangladesh Asia      2007   64.1 150448339   1391.
10 Belgium   Europe    2007   79.4  10392226  33693.
# i 132 more rows
```

4.4 Comparing Base R Graphics And ggplot2

Base R has plotting functions built in. They are useful for quick checks and appear in many older examples.

```
plot(gap_2007$gdpPercap, gap_2007$lifeExp)
```



That plot works, but the labels are not very informative, the x-axis is hard to read, and customization quickly becomes a matter of remembering many arguments.

A base R histogram shows a similar pattern:

```
hist(gap_2007$lifeExp, breaks = 8)
```



The problem is not that base R cannot make plots. The problem is that complex plots become difficult to revise. `ggplot2` gives us a more structured way to build visualizations.

4.5 The Grammar Of Graphics

A `ggplot2` figure is built in layers. Every plot needs three minimum elements before any layer can sit on top of them:

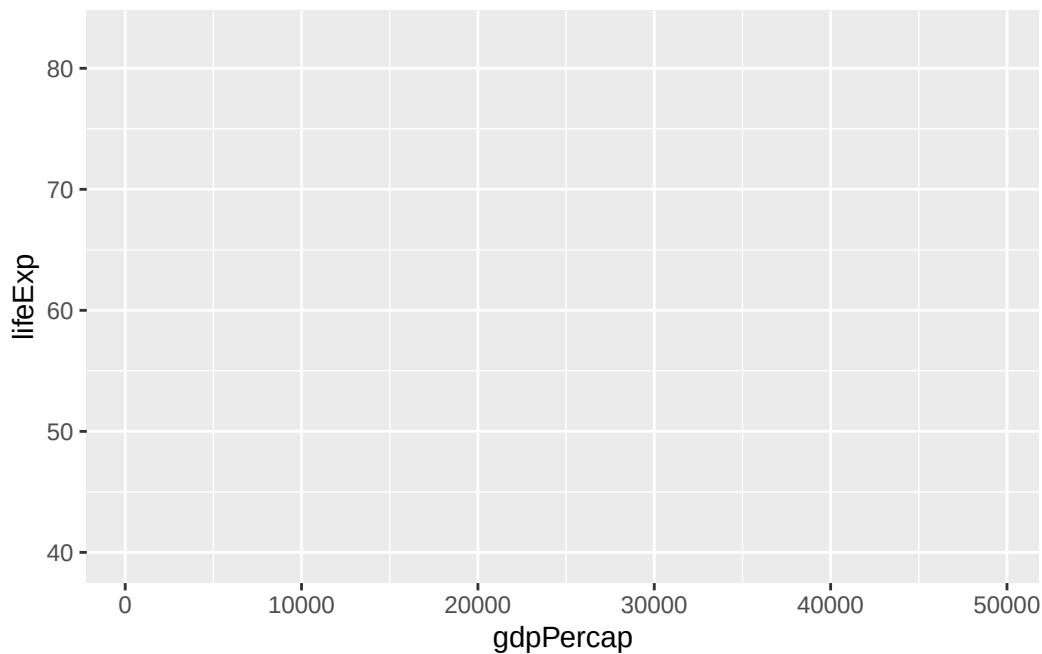
- data: the table being plotted
- mappings: the links between variables and visual features
- geoms: the visual marks, such as points, lines, bars, or histograms

This separation is what makes `ggplot2` powerful. You can change the data, the mapping, or the geom without rewriting the whole plot from scratch.

4.6 The Base Layer: `ggplot()` And `aes()`

The `ggplot()` function starts a plot. The `aes()` function defines aesthetic mappings. Aesthetic mappings connect variables in the data to visual properties such as x-position, y-position, color, size, shape, or transparency.

```
ggplot(  
  data = gap_2007,  
  mapping = aes(x = gdpPercap, y = lifeExp)  
)
```

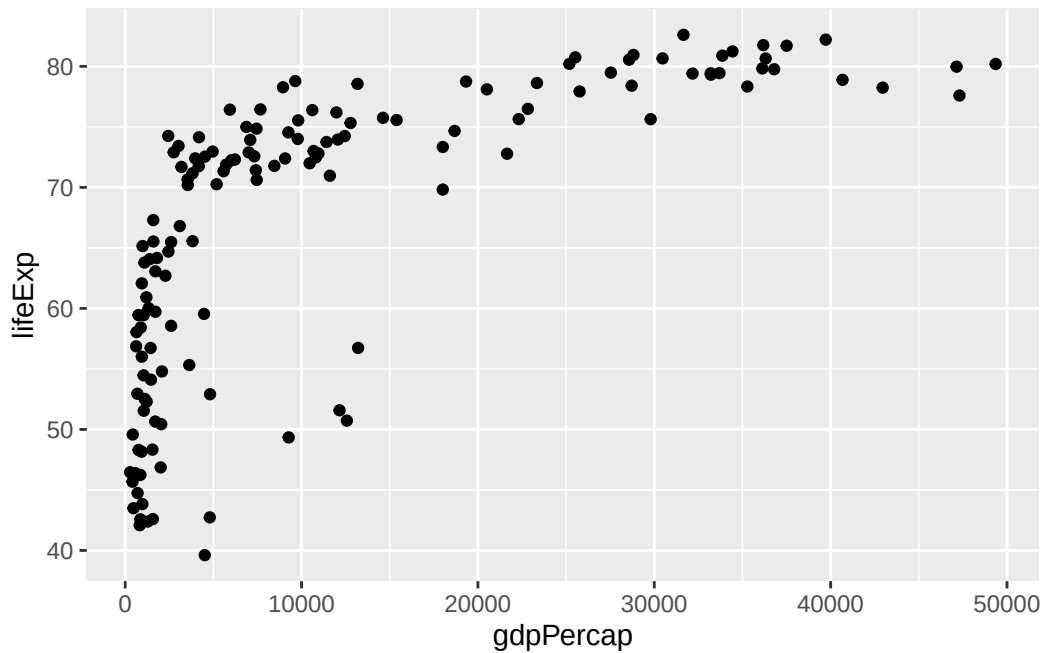


This is the **base layer**. The data and mappings are set, the axes are drawn, but nothing is plotted yet because no geom has been added. Every later layer — points, lines, smoothers, labels, scales — sits on top of this base.

4.7 Add A Geom Layer

`geom_point()` draws points. Because each row in `gap_2007` is a country, each point is a country.

```
ggplot(  
  data = gap_2007,  
  mapping = aes(x = gdpPercap, y = lifeExp)  
) +  
  geom_point()
```

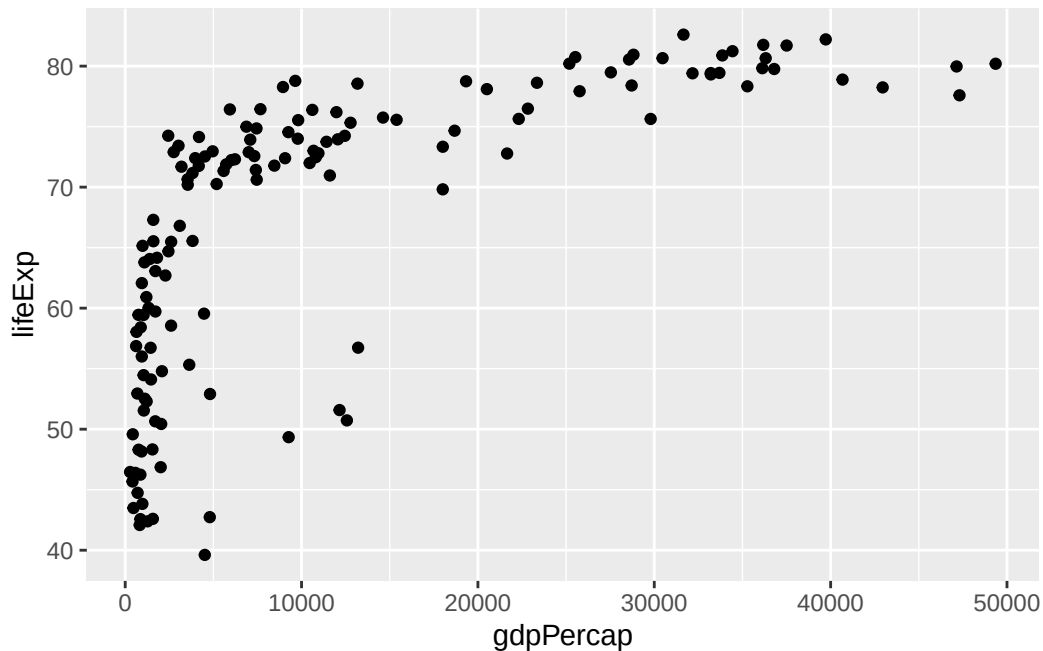


The `+` operator means “add the next plot component.” In `ggplot2`, line breaks usually put `+` at the end of the line so R knows the plot is not finished.

4.8 Saving A Plot Object

It is often useful to store the base plot and add layers later.

```
p_gap <- ggplot(  
  data = gap_2007,  
  mapping = aes(x = gdpPercap, y = lifeExp)  
)  
  
p_gap +  
  geom_point()
```



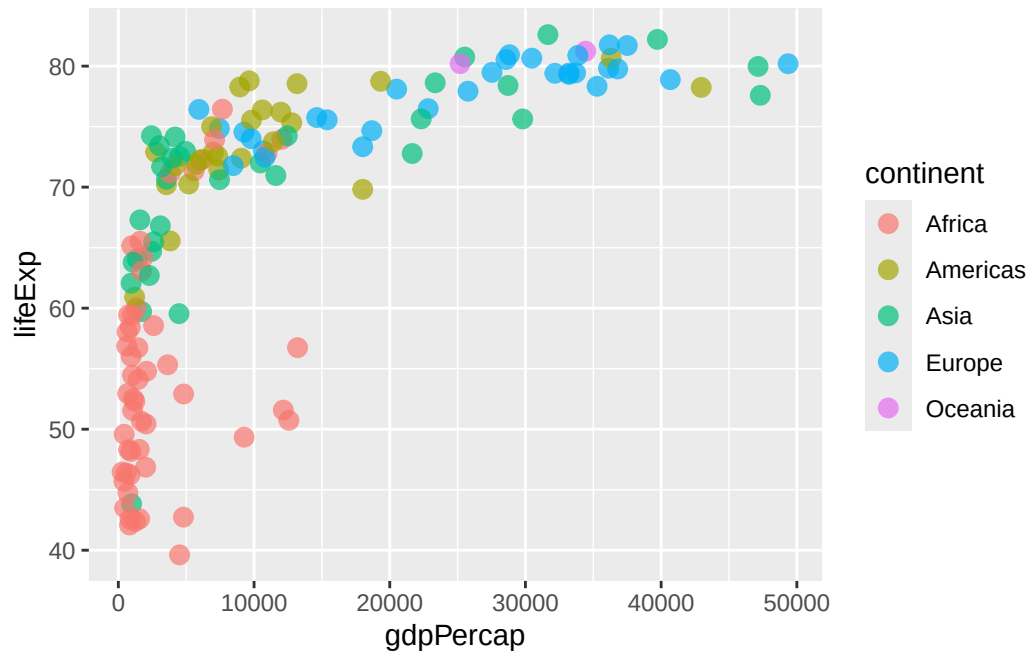
The object `p_gap` stores the data and default mappings. It does not lock the plot into one final form. We can keep adding layers.

4.9 Mapping Versus Setting

One of the most important `ggplot2` distinctions is the difference between mapping and setting.

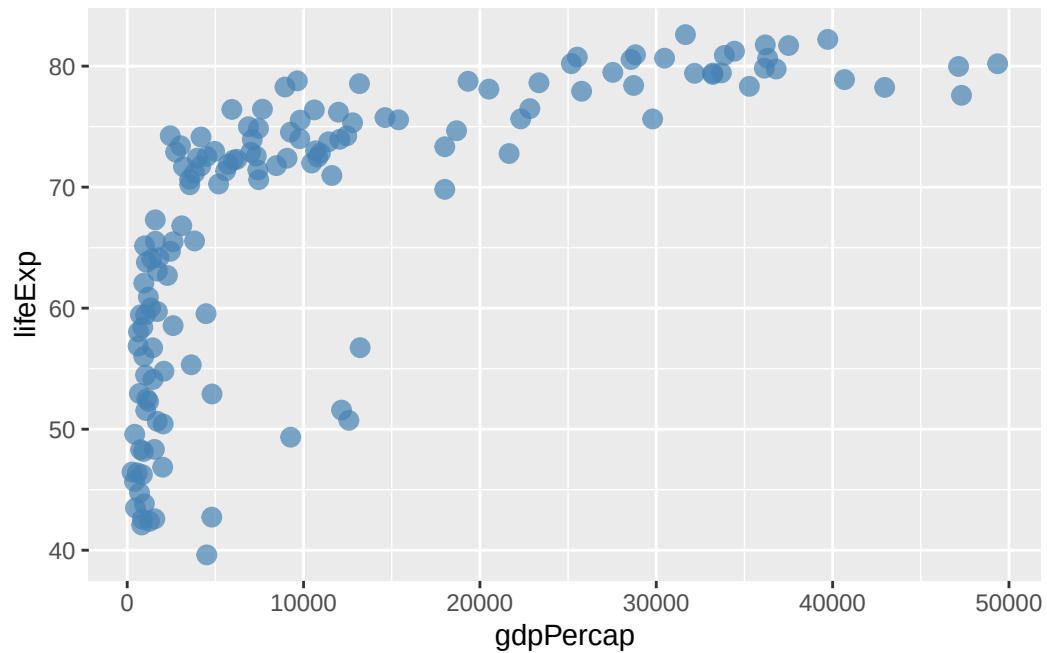
When an aesthetic is mapped, it goes inside `aes()` and represents data. In the next plot, color means continent.

```
p_gap +  
  geom_point(aes(color = continent), size = 3, alpha = 0.7)
```



When an aesthetic is set, it stays fixed and goes outside `aes()`. In the next plot, all points are the same color. The color does not encode information.

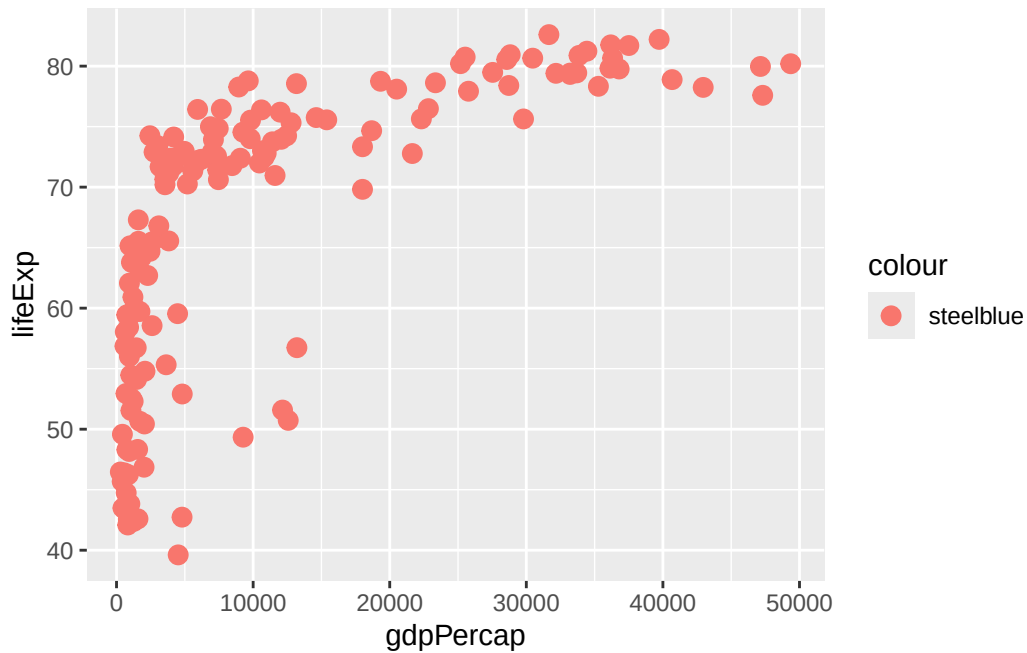
```
p_gap +  
  geom_point(color = "steelblue", size = 3, alpha = 0.7)
```



This distinction matters because viewers interpret visual differences. If color varies because it is mapped to a variable, it carries meaning. If color is fixed, it is a design choice.

A common mistake is to put a fixed color inside `aes()`:

```
p_gap +  
  geom_point(aes(color = "steelblue"), size = 3)
```



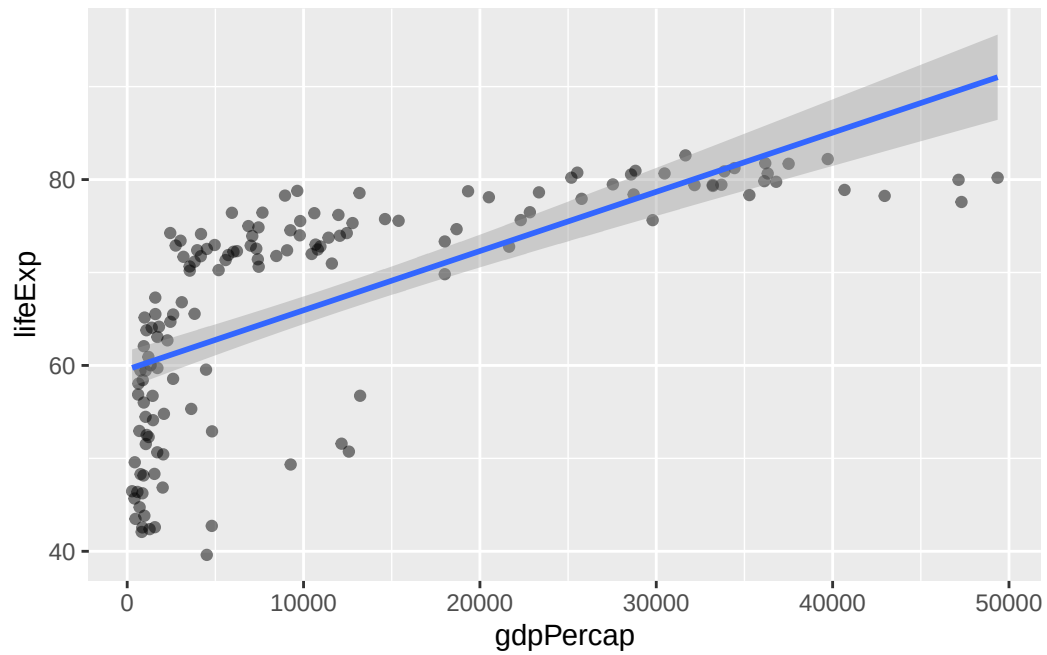
In that plot, "steelblue" is treated like a one-value variable, not as a fixed color instruction. The result is a legend that does not help the reader. The fix is to move fixed settings outside `aes()`.

4.10 Smoothers

Scatterplots show individual observations. Smoothers add a summary of the relationship.

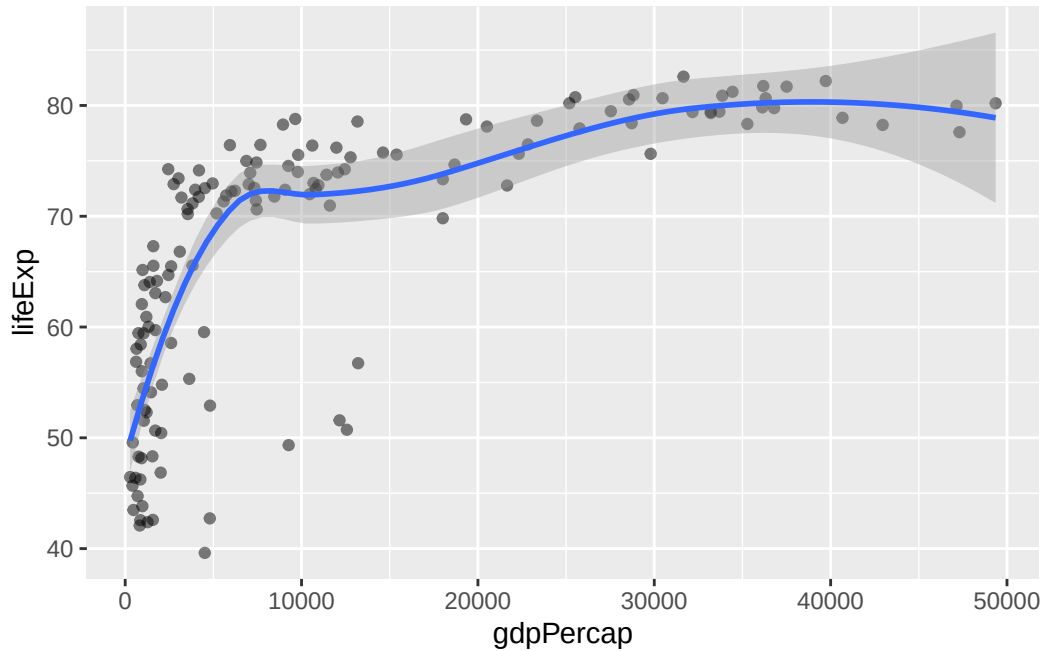
A linear smoother fits a straight line:

```
p_gap +  
  geom_point(alpha = 0.5) +  
  geom_smooth(method = "lm", formula = y ~ x)
```

A loess smoother is more flexible:

```
p_gap +  
  geom_point(alpha = 0.5) +  
  geom_smooth(method = "loess", formula = y ~ x)
```



Neither smoother is automatically correct. A straight line is easy to interpret but may miss curvature. A loess curve can reveal bends in the data but is more descriptive than explanatory. The right choice depends on the purpose of the plot.

4.11 Log Scales

GDP per capita is usually highly skewed. A few very high values can compress most observations into the left side of the plot. A log scale spreads the lower and middle values out while keeping the order intact.

```
p_gap +  
  geom_point(alpha = 0.6) +  
  scale_x_log10(labels = label_dollar())
```

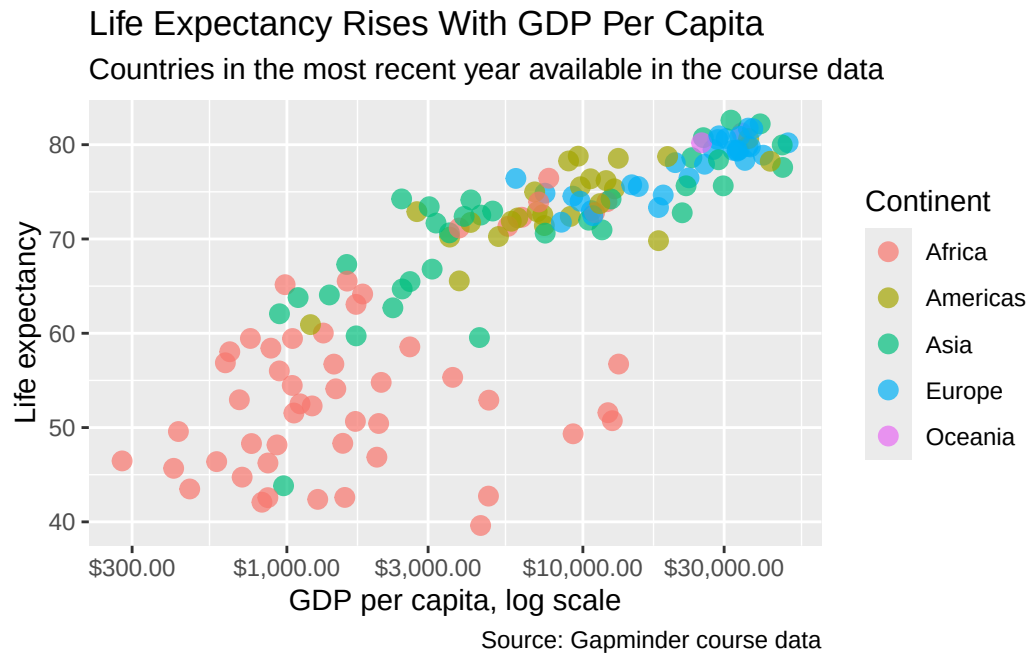


The command `scale_x_log10()` changes the x-axis, not the underlying data object. The argument `labels = label_dollar()` formats the axis as dollars instead of scientific notation.

4.12 Labels With `labs()`

Plots should be understandable outside the code chunk. `labs()` adds titles, subtitles, axis labels, legend labels, and captions.

```
p_gap +
  geom_point(aes(color = continent), alpha = 0.7, size = 3) +
  scale_x_log10(labels = label_dollar()) +
  labs(
    title = "Life Expectancy Rises With GDP Per Capita",
    subtitle = "Countries in the most recent year available in the course data",
    x = "GDP per capita, log scale",
    y = "Life expectancy",
    color = "Continent",
    caption = "Source: Gapminder course data"
  )
```

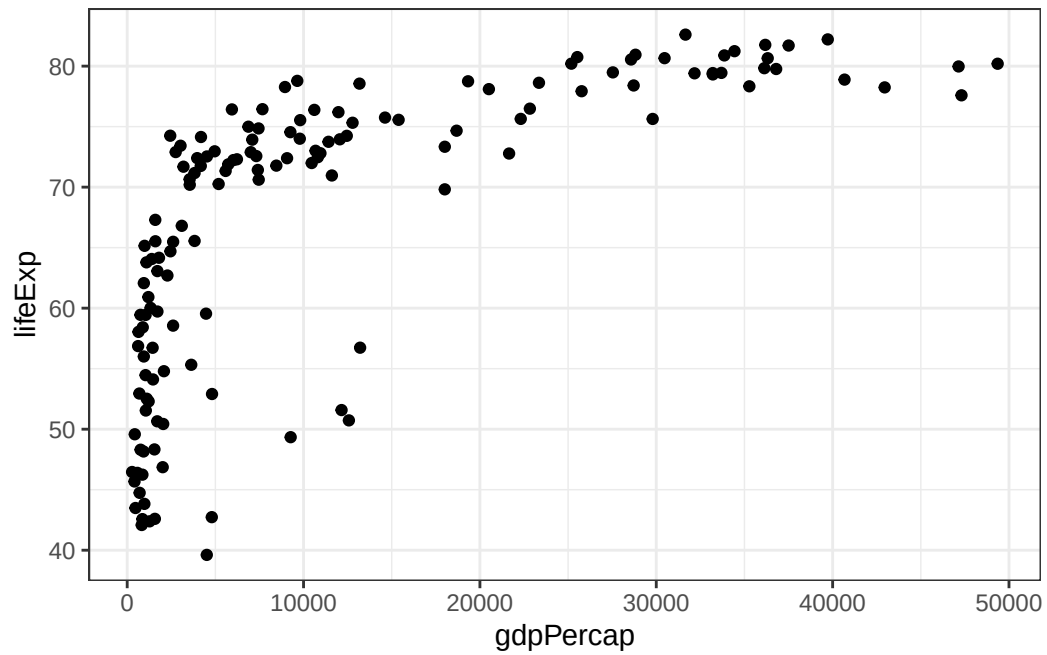


The title tells the reader the main subject. The subtitle adds context. Axis labels state the meaning of the plotted variables. The legend title clarifies the mapped aesthetic. The caption is a natural place for data source information.

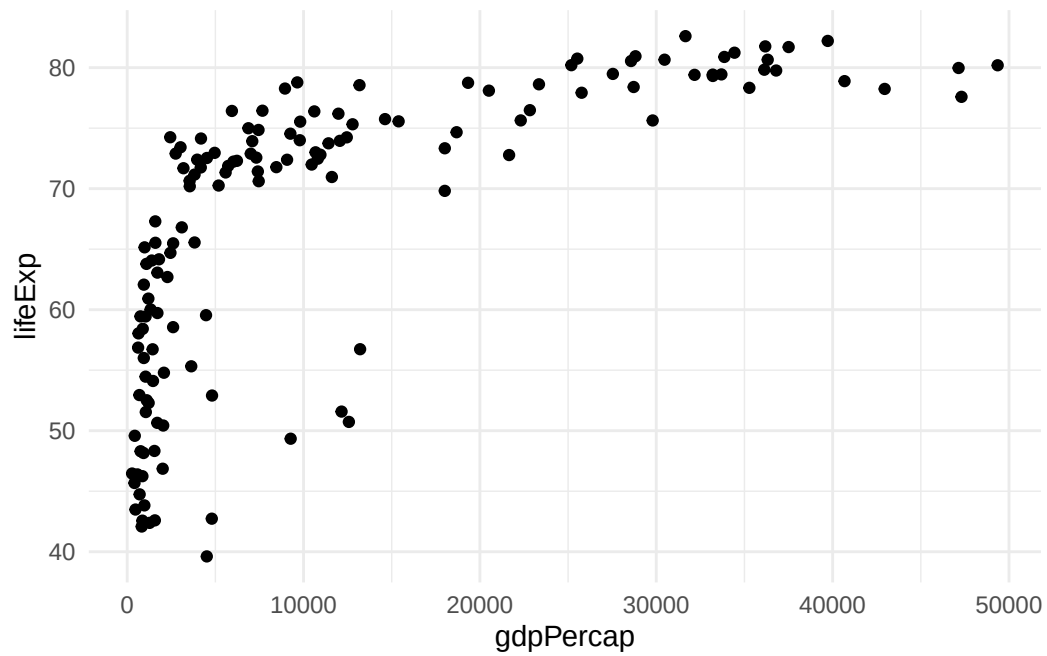
4.13 Themes

Themes control the non-data parts of the plot: background, grid lines, axis text, title placement, legend placement, and similar elements. A theme does not change the data being shown, but it can change how easy the plot is to read.

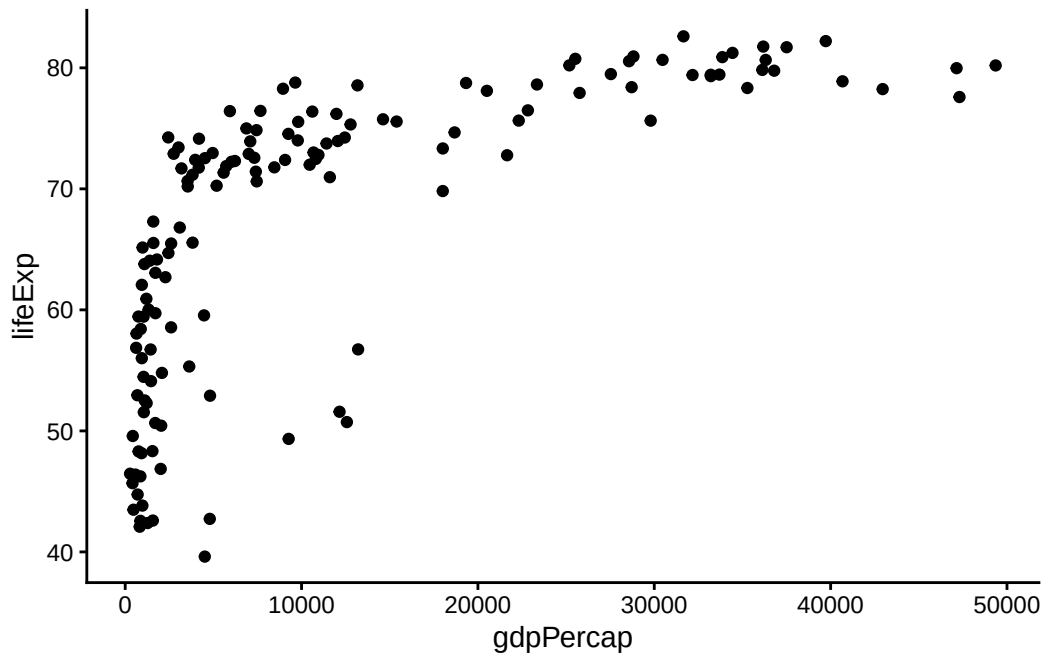
```
p_gap + geom_point() + theme_bw()
```



```
p_gap + geom_point() + theme_minimal()
```

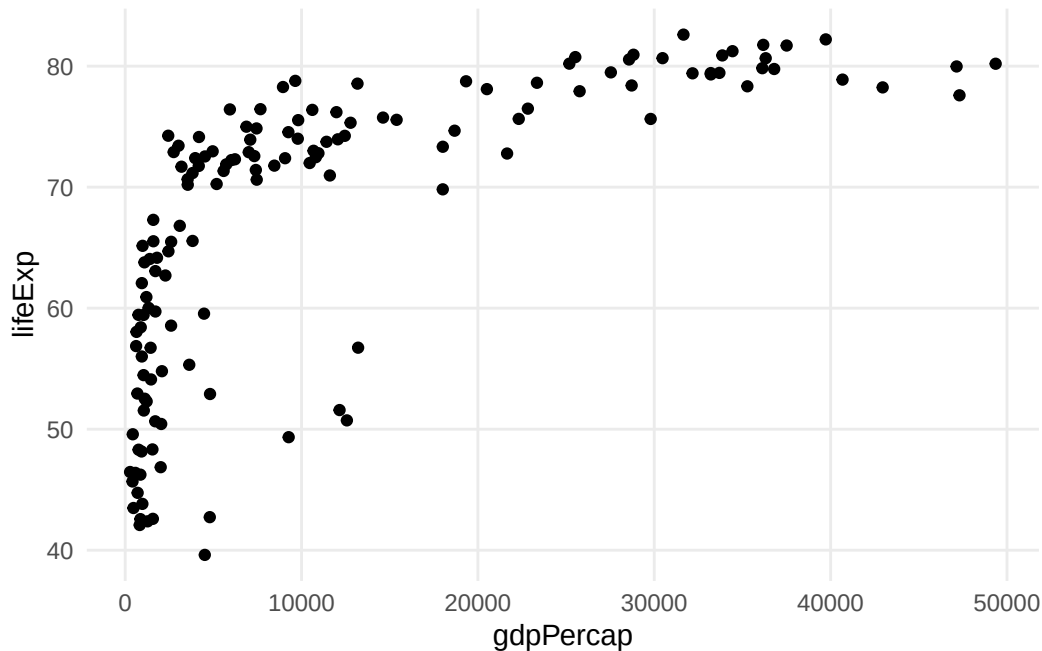


```
p_gap + geom_point() + theme_classic()
```



Built-in themes are good starting points. Individual elements can then be adjusted with `theme()`.

```
p_gap +
  geom_point() +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 16, face = "bold"),
    plot.subtitle = element_text(size = 11),
    axis.title = element_text(size = 11),
    panel.grid.minor = element_blank()
  )
```



The most important rule is that the theme should support the comparison. A visually dramatic theme that makes the data harder to read is not helping.

4.14 Color Palettes

Default colors are fine for early drafts. For more intentional figures, choose palettes that match the type of data.

- Qualitative palettes distinguish categories with no natural order.
- Sequential palettes show ordered values from low to high.
- Diverging palettes show distance from a meaningful midpoint.

Start from a plot that maps a categorical variable to color:

```
p_gap_color <- p_gap +
  geom_point(aes(color = continent), alpha = 0.75, size = 2) +
  scale_x_log10(labels = label_dollar()) +
  labs(
    x = "GDP per capita, log scale",
    y = "Life expectancy",
    color = "Continent"
  ) +
```

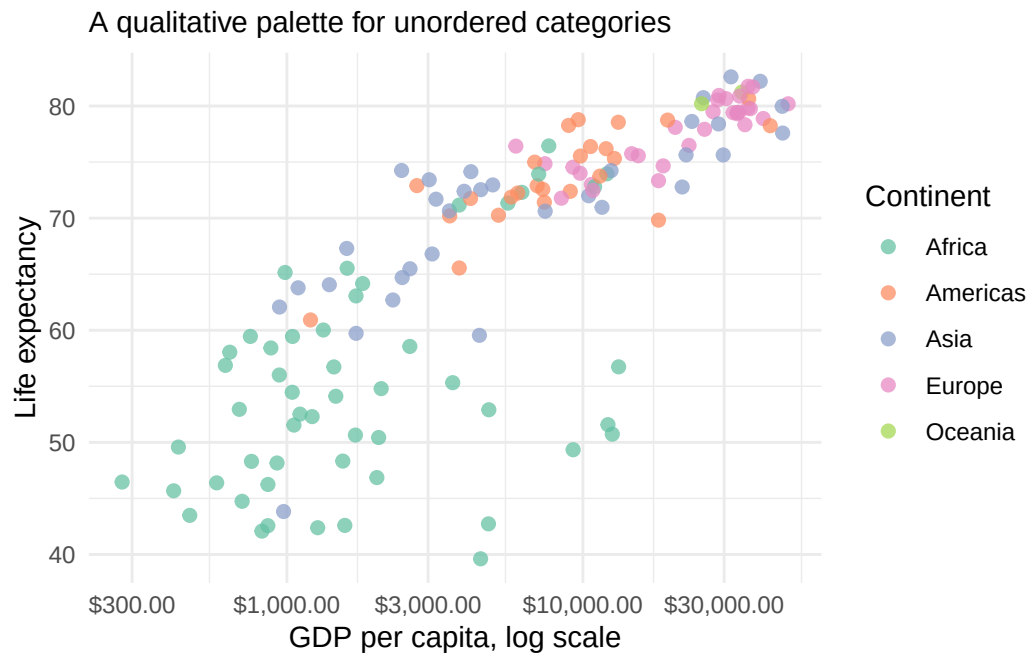
```
theme_minimal()
```

```
p_gap_color
```



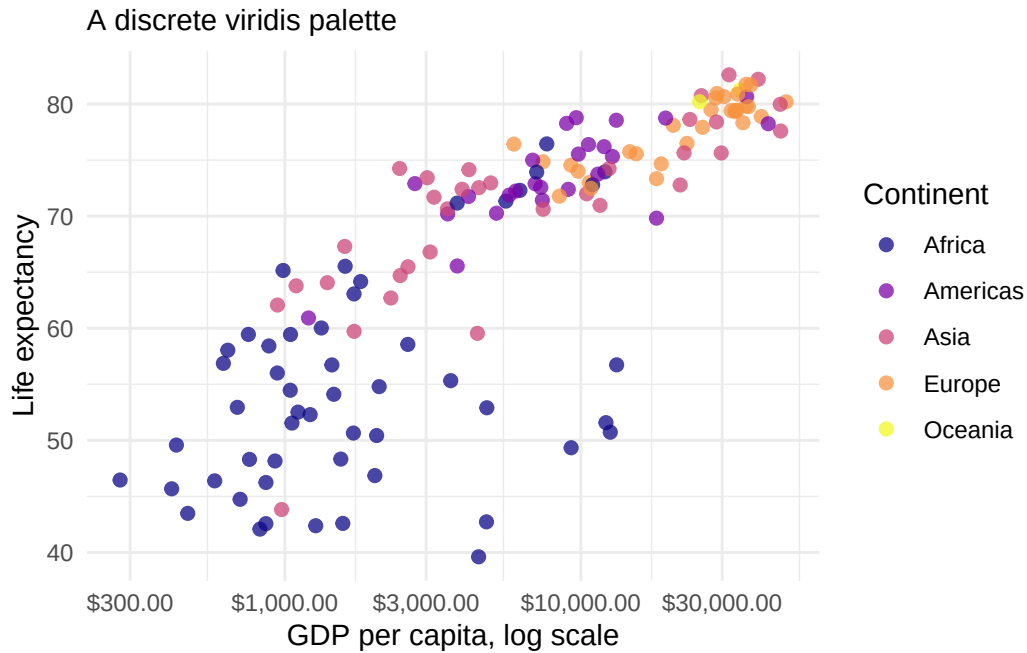
RColorBrewer provides many named palettes.

```
p_gap_color +  
  scale_color_brewer(palette = "Set2") +  
  labs(subtitle = "A qualitative palette for unordered categories")
```

The viridis scales are also useful because they are designed to be readable in color and when converted to grayscale.

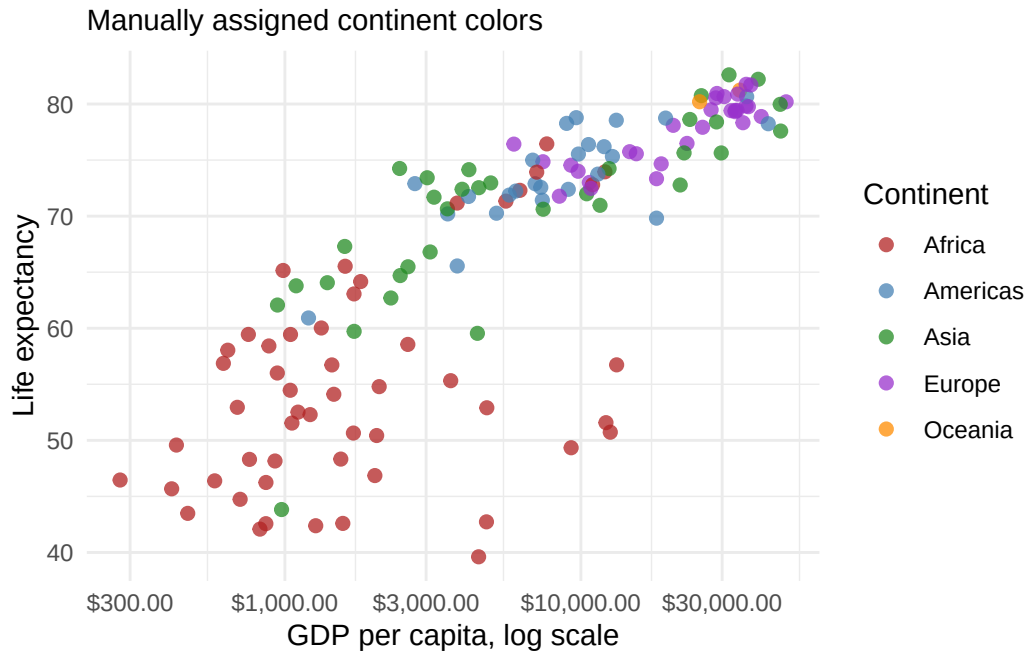
```
p_gap_color +  
  scale_color_viridis_d(option = "plasma") +  
  labs(subtitle = "A discrete viridis palette")
```



Manual color scales are useful when specific categories should keep specific colors across several plots.

```
continent_colors <- c(
  "Africa" = "firebrick",
  "Americas" = "steelblue",
  "Asia" = "forestgreen",
  "Europe" = "darkorchid",
  "Oceania" = "darkorange"
)

p_gap_color +
  scale_color_manual(values = continent_colors) +
  labs(subtitle = "Manually assigned continent colors")
```



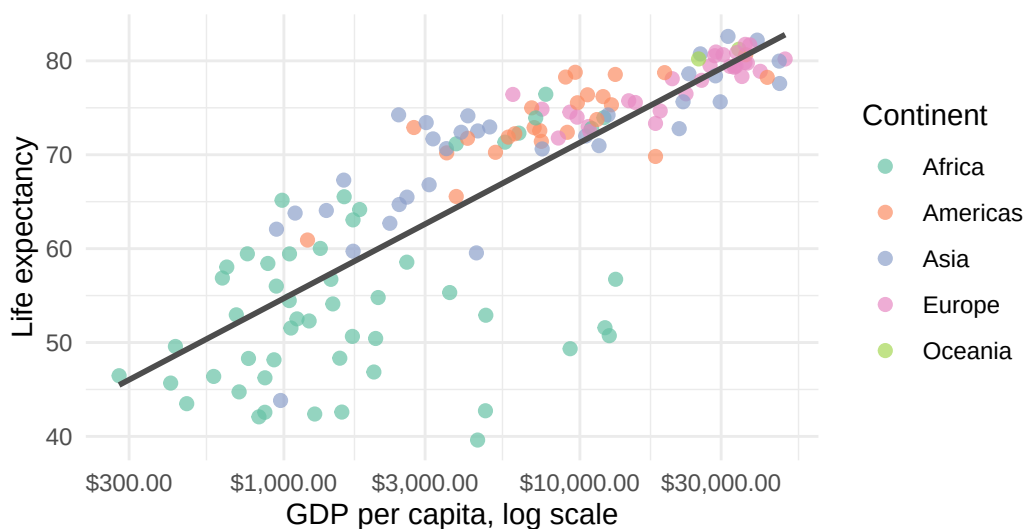
4.15 Putting It All Together

Each piece of this chapter has been introduced as one layer at a time. The full plot stacks them in order: base layer, point geom, smoother, scale, labels, palette, theme.

```
ggplot(
  data = gap_2007,
  mapping = aes(x = gdpPercap, y = lifeExp)
) +
  geom_point(aes(color = continent), alpha = 0.7, size = 2) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, color = "gray30") +
  scale_x_log10(labels = label_dollar()) +
  scale_color_brewer(palette = "Set2") +
  labs(
    title = "Life Expectancy Rises With GDP Per Capita",
    subtitle = "Countries in the most recent year available in the course data",
    x = "GDP per capita, log scale",
    y = "Life expectancy",
    color = "Continent",
    caption = "Source: Gapminder course data"
  ) +
  theme_minimal()
```

Life Expectancy Rises With GDP Per Capita

Countries in the most recent year available in the course data



Source: Gapminder course data

Read the code from the top:

- `ggplot(...)` sets the base layer with data and mappings.
- `geom_point(...)` draws each country as a point and maps continent to color.
- `geom_smooth(...)` adds a single linear summary across all continents.
- `scale_x_log10(...)` transforms the x-axis so the lower values are easier to read.
- `scale_color_brewer(...)` swaps the default palette for a qualitative one.
- `labs(...)` adds the title, subtitle, axis labels, legend title, and caption.
- `theme_minimal()` controls the non-data styling.

If the plot needs to change, the line responsible for that change is usually easy to identify.

4.16 Short Exercise

The built-in `mtcars` dataset is always available. Convert it to a tibble first so the car names become a column:

```
cars_tbl <- mtcars |>
  as_tibble(rownames = "car")
```

Using `cars_tbl`, build a scatterplot with:

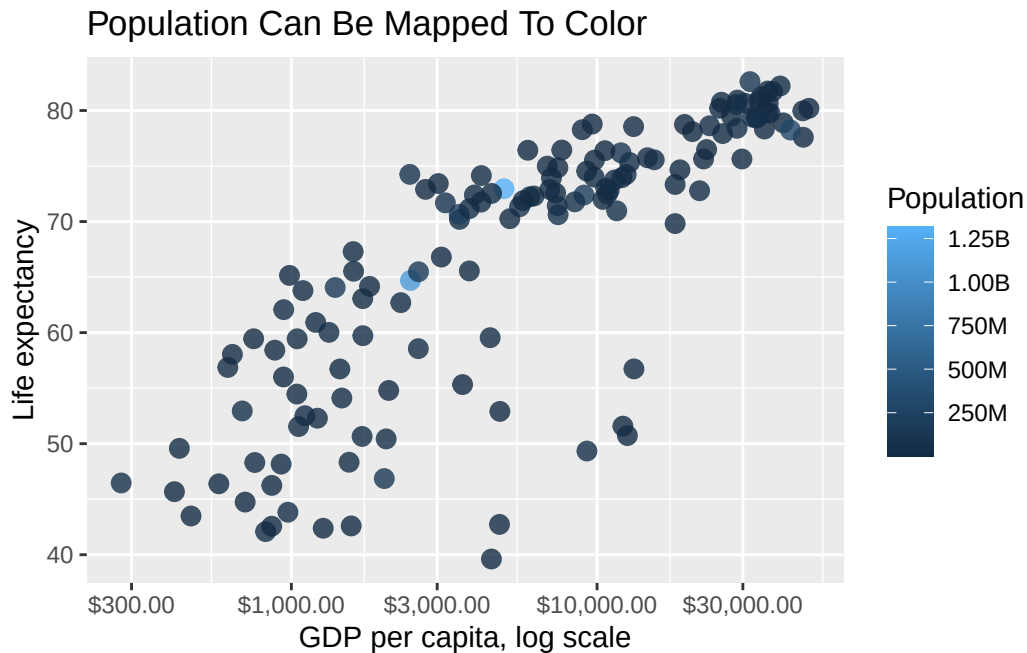
1. `hp` on the x-axis
2. `mpg` on the y-axis
3. `cyl` mapped to color
4. a linear smoother with `se = FALSE`
5. readable labels with `labs()`

```
# Write your code here.
```

4.17 Extras: Continuous Color

Color can represent a continuous variable as well as a category. In the next plot, color represents population.

```
p_gap +  
  geom_point(aes(color = pop), size = 3, alpha = 0.8) +  
  scale_x_log10(labels = label_dollar()) +  
  scale_color_continuous(labels = label_number(scale_cut = cut_short_scale())) +  
  labs(  
    title = "Population Can Be Mapped To Color",  
    x = "GDP per capita, log scale",  
    y = "Life expectancy",  
    color = "Population"  
  )
```

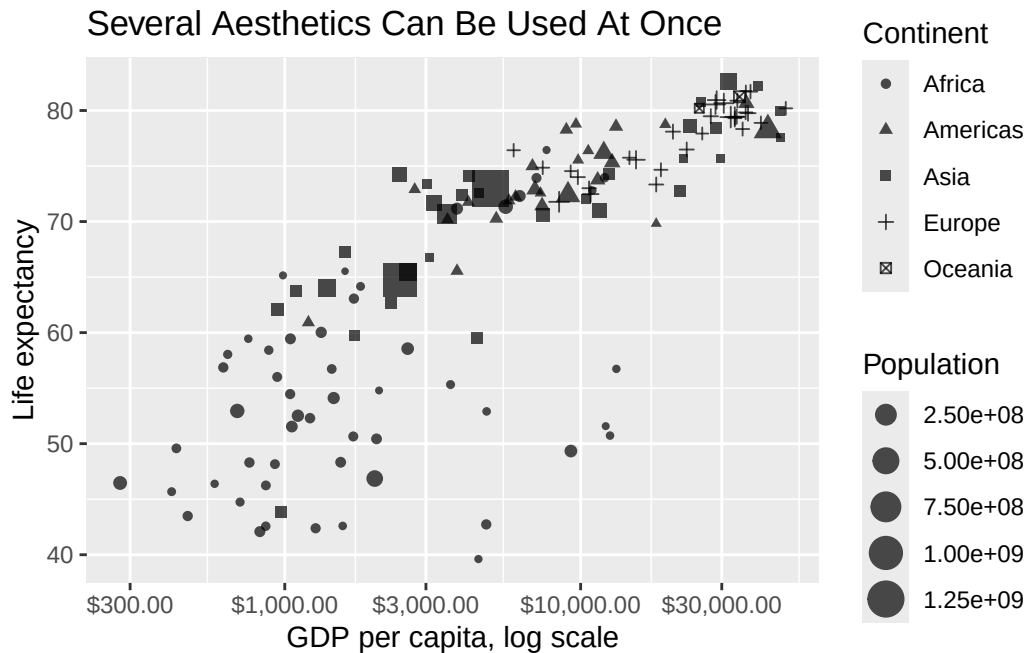


This is possible, but it is not always the best choice. Continuous color scales can be hard to read precisely. Use them when the added variable helps interpretation rather than simply making the plot look more complex.

4.18 Extras: Shape And Size

Other aesthetics can also be mapped or set. Shape works best for a small number of categories. Size can work for quantities, but area comparisons are difficult for readers.

```
ggplot(gap_2007, aes(x = gdpPercap, y = lifeExp)) +
  geom_point(aes(shape = continent, size = pop), alpha = 0.7) +
  scale_x_log10(labels = label_dollar()) +
  labs(
    title = "Several Aesthetics Can Be Used At Once",
    x = "GDP per capita, log scale",
    y = "Life expectancy",
    shape = "Continent",
    size = "Population"
  )
```



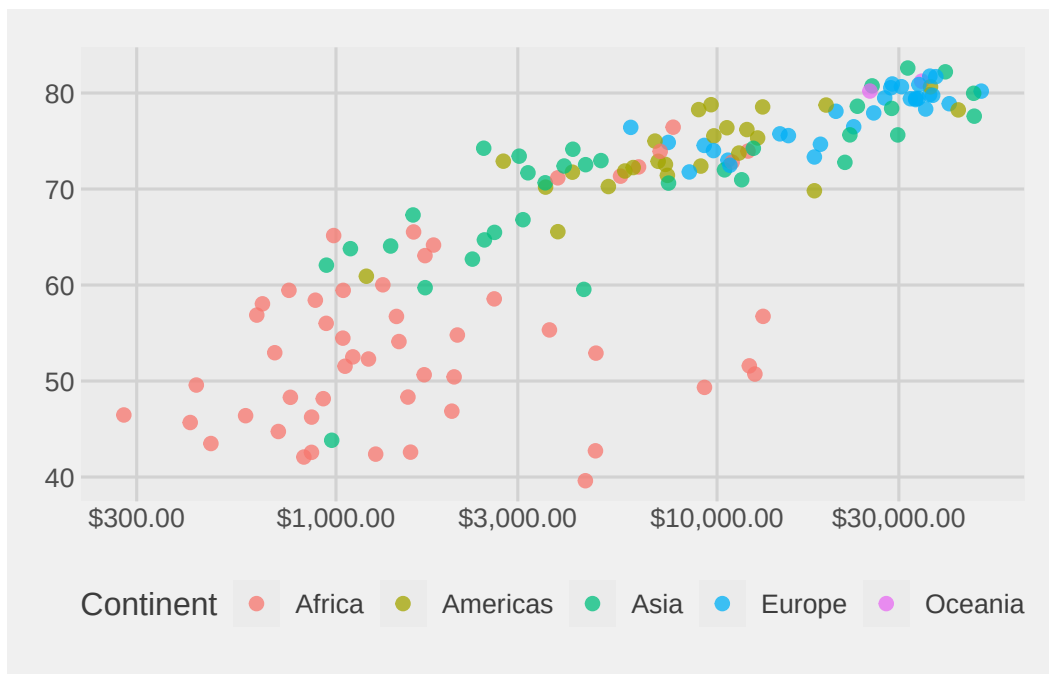
Adding more aesthetics is not automatically an improvement. Each additional encoding asks the reader to do more work. The best plot is usually the one that makes the intended comparison clear with the fewest necessary encodings.

4.19 Extras: External Themes And Rich Text

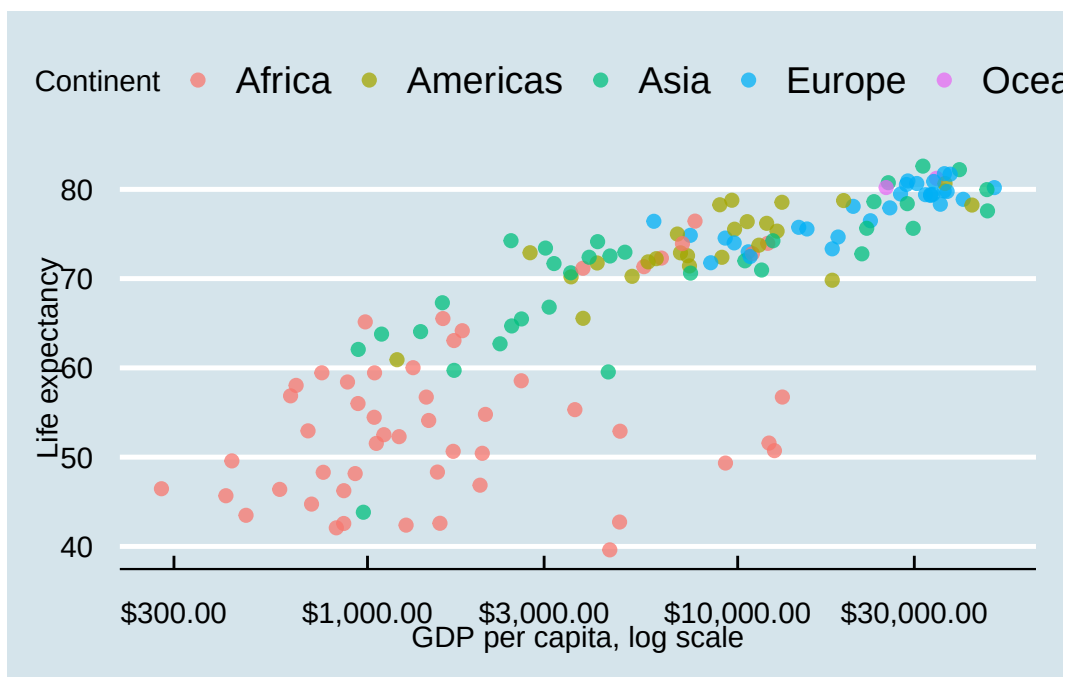
The packages `ggthemes`, `hrbrthemes`, and `ggtext` provide additional styling options. These are useful when a plot needs to match a publication, organization, or report style.

`ggthemes` includes themes that imitate familiar publication and media styles.

```
p_gap_color +
  ggthemes::theme_fivethirtyeight()
```

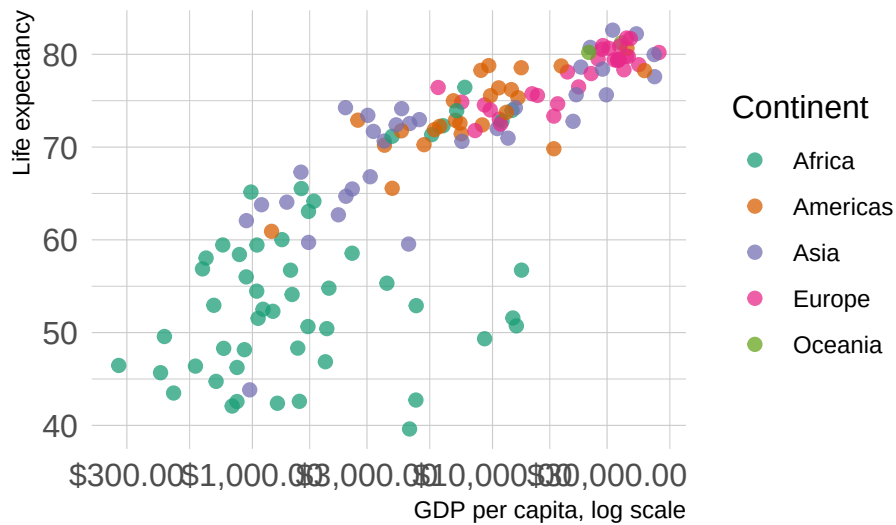


```
p_gap_color +
  ggthemes::theme_economist()
```



The `hrbrthemes` package is especially useful for clean report-style plots.


```
p_gap_color +
  scale_color_brewer(palette = "Dark2") +
  hrbrthemes::theme_ipsum(base_family = "sans")
```

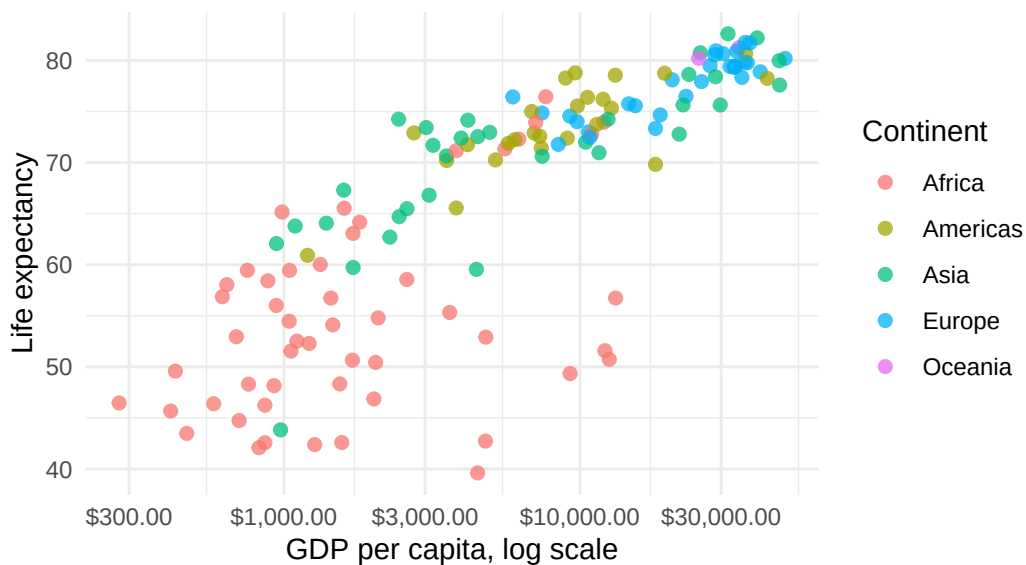


`ggtext` allows limited Markdown and HTML-like formatting inside plot text. This is useful for emphasizing one phrase in a title or coloring part of a subtitle.

```
p_gap_color +
  labs(
    title = "Economic Growth and **Life Expectancy**",
    subtitle = "Countries in <span style='color:darkorange;'>2007</span>"
  ) +
  theme_minimal() +
  theme(
    plot.title = ggtext::element_markdown(),
    plot.subtitle = ggtext::element_markdown()
  )
```

Economic Growth and Life Expectancy

Countries in 2007



Style should refine a figure, not rescue an unclear comparison.

4.20 What Comes Next

The next chapter moves from basic plot construction to visual comparison: separating groups with color, facets, small multiples, free scales, direct labels, and combining plots with **patchwork**. The focus shifts from making a plot work to making a comparison easier to see.

5 Separating and Comparing Data

5.1 Why separate data?

Many plots become more useful when we separate the data into meaningful groups. The relationship between two variables might look simple overall, but the pattern can differ across continents, years, regions, parties, income groups, or other subgroups.

There are two common `ggplot2` strategies:

- Map a grouping variable to an aesthetic such as `color`, `shape`, `fill`, or `linetype`.
- Use facets to create small multiples: repeated panels that show the same plot for different subsets of the data.

Both approaches are useful. The question is not which one is always better. The question is which one makes the comparison easier.

5.2 Start with one plot

We will use Gapminder data to compare GDP per capita and life expectancy. First, filter to one year so that each point is one country.

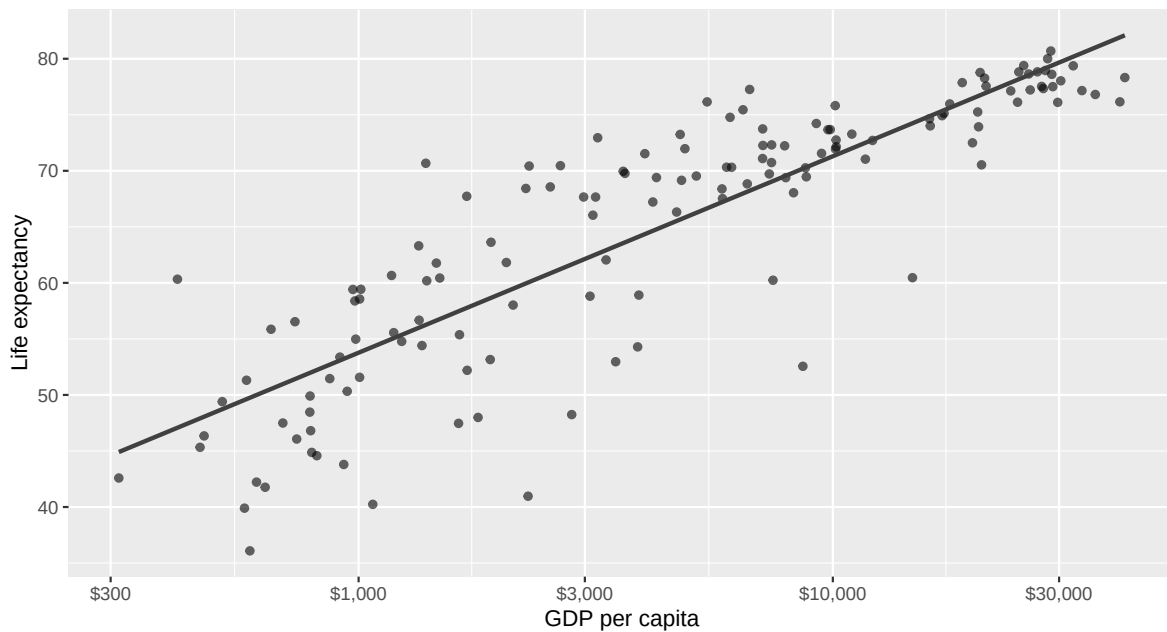
```
gap_1997 <- gapminder |>
  filter(year == 1997)

base_gap <- ggplot(
  data = gap_1997,
  mapping = aes(x = gdpPercap, y = lifeExp)
) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, color = "gray25") +
  scale_x_log10(labels = dollar_format(accuracy = 1)) +
  scale_y_continuous(breaks = seq(20, 90, by = 10)) +
  labs(
    title = "Economic Growth and Life Expectancy",
    subtitle = "Countries in 1997",
```

```
x = "GDP per capita",
y = "Life expectancy",
caption = "Source: Gapminder."
)
```

base_gap

Economic Growth and Life Expectancy
Countries in 1997



Source: Gapminder.

The log scale on the x-axis helps because GDP per capita is very skewed. Without it, the lower-income countries would be compressed into a small part of the plot.

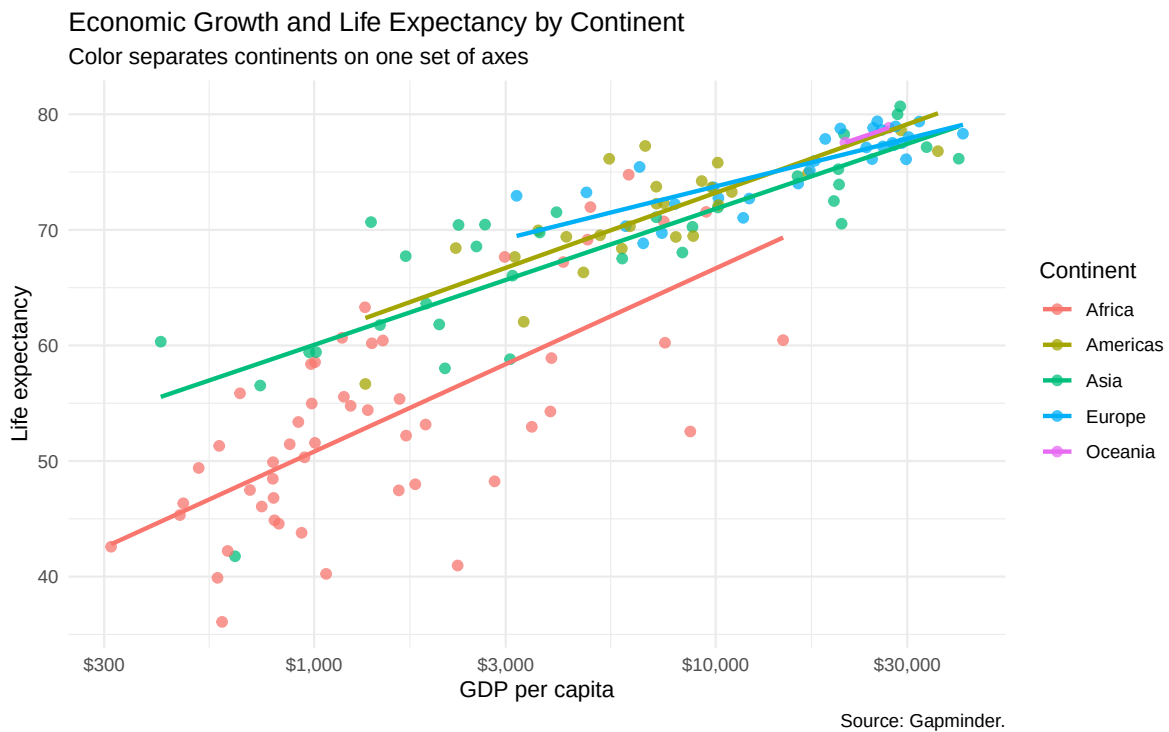
5.3 Color as separation

One way to separate groups is to map a variable to color inside `aes()`.

```
gap_color <- ggplot(
  data = gap_1997,
  mapping = aes(x = gdpPercap, y = lifeExp, color = continent)
) +
  geom_point(alpha = 0.75, size = 2) +
```

```
geom_smooth(method = "lm", formula = y ~ x, se = FALSE) +
scale_x_log10(labels = dollar_format(accuracy = 1)) +
labs(
  title = "Economic Growth and Life Expectancy by Continent",
  subtitle = "Color separates continents on one set of axes",
  x = "GDP per capita",
  y = "Life expectancy",
  color = "Continent",
  caption = "Source: Gapminder."
) +
theme_minimal()

gap_color
```

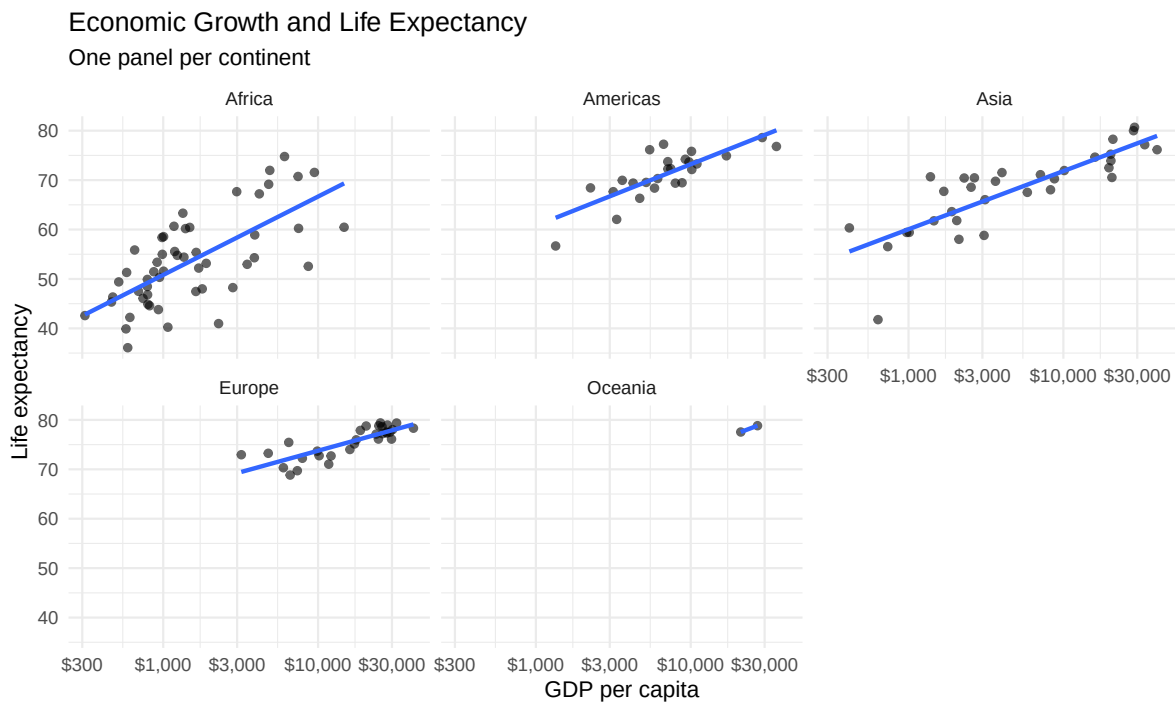


Because `continent` is mapped inside `aes()`, `ggplot2` treats color as part of the data mapping and creates a legend. If we wrote `geom_point(color = "steelblue")`, every point would be steel blue and no legend would be needed.

5.4 Facets as small multiples

Faceting creates one panel for each level of a variable. This gives each group its own space.

```
ggplot(data = gap_1997, mapping = aes(x = gdpPercap, y = lifeExp)) +  
  geom_point(alpha = 0.6) +  
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE) +  
  scale_x_log10(labels = dollar_format(accuracy = 1)) +  
  facet_wrap(~ continent) +  
  labs(  
    title = "Economic Growth and Life Expectancy",  
    subtitle = "One panel per continent",  
    x = "GDP per capita",  
    y = "Life expectancy",  
    caption = "Source: Gapminder."  
  ) +  
  theme_minimal()
```



Source: Gapminder.

The default shared scales make comparisons across panels fair. Africa and Europe are plotted on the same x and y ranges, so differences in location and spread are visible.

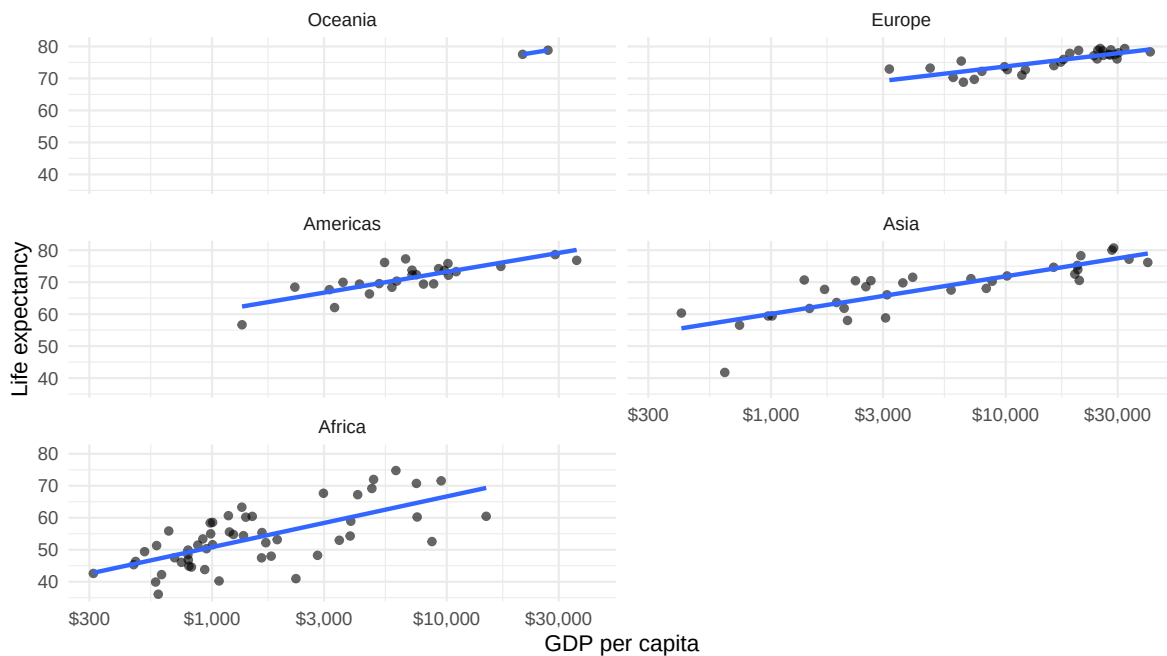
5.5 Reordering facets

Facet order is not always meaningful by default. We can reorder the continent variable by average life expectancy.

```
gap_1997_ordered <- gap_1997 |>
  mutate(
    continent = fct_reorder(
      .f = continent,
      .x = lifeExp,
      .fun = mean,
      .desc = TRUE
    )
  )

ggplot(data = gap_1997_ordered, mapping = aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE) +
  scale_x_log10(labels = dollar_format(accuracy = 1)) +
  facet_wrap(~ continent, ncol = 2) +
  labs(
    title = "Facets Ordered by Average Life Expectancy",
    x = "GDP per capita",
    y = "Life expectancy",
    caption = "Source: Gapminder."
  ) +
  theme_minimal()
```

Facets Ordered by Average Life Expectancy



Source: Gapminder.

`fct_reorder()` changes the order of the factor levels. That order is then used by `facet_wrap()`.

5.6 Free scales

Sometimes fixed scales make within-group patterns hard to see. Free scales let each panel use its own axis range.

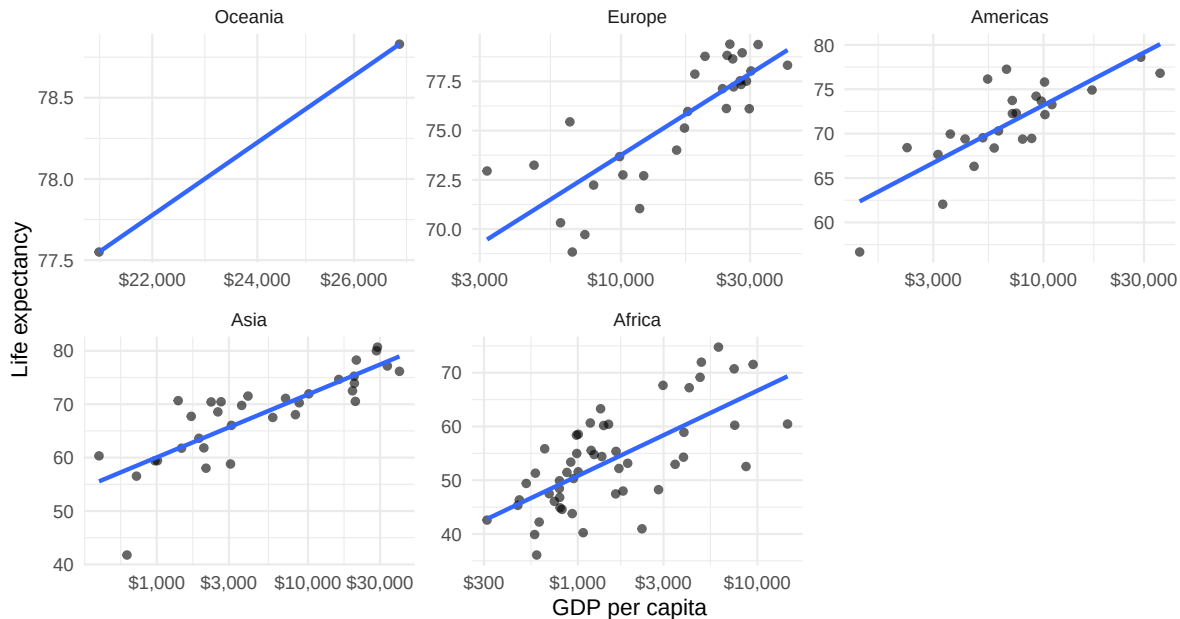
```
ggplot(data = gap_1997_ordered, mapping = aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE) +
  scale_x_log10(labels = dollar_format(accuracy = 1)) +
  facet_wrap(~ continent, scales = "free") +
  labs(
    title = "Free Scales Reveal Within-Continent Detail",
    subtitle = "Use with care: cross-panel comparisons become harder",
    x = "GDP per capita",
    y = "Life expectancy",
    caption = "Source: Gapminder."
```



```
) +  
theme_minimal()
```

Free Scales Reveal Within-Continent Detail

Use with care: cross-panel comparisons become harder



Source: Gapminder.

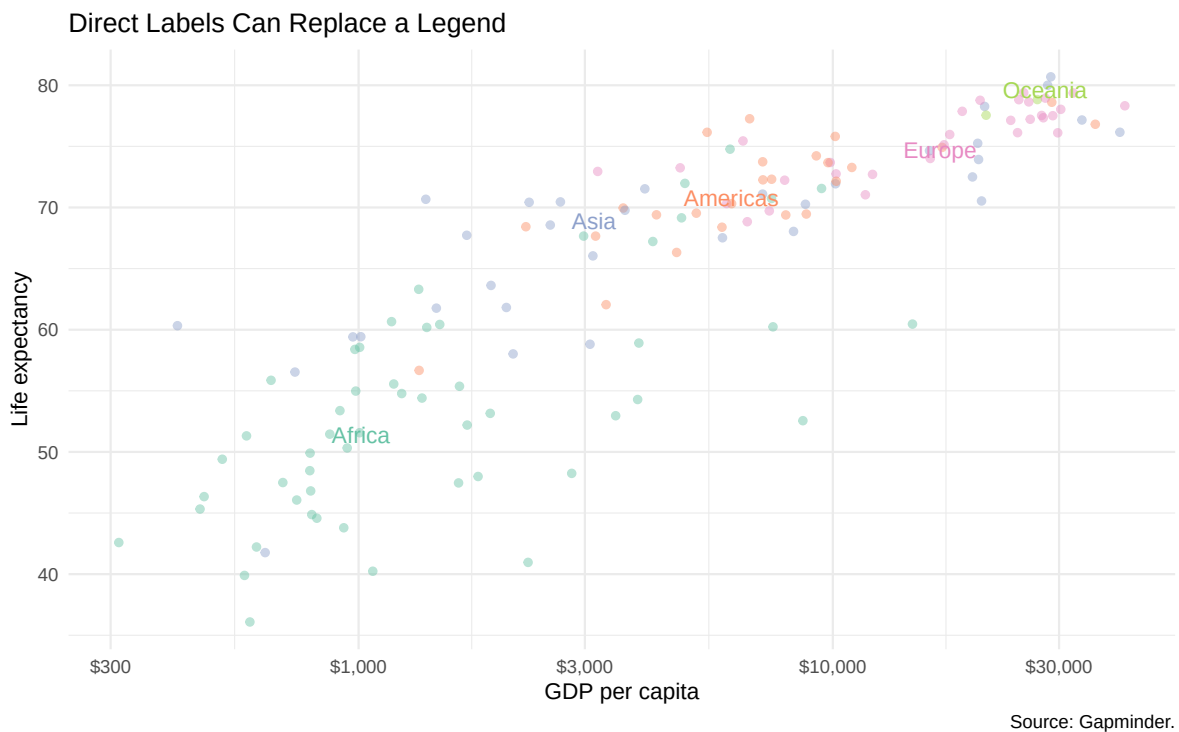
Free scales are not wrong, but they answer a different question. Fixed scales emphasize comparison across groups. Free scales emphasize detail within each group.

5.7 Direct labels

Legends require the reader to look back and forth between the data and the key. Direct labels can reduce that work.

```
continent_summary <- gap_1997 |>  
  group_by(continent) |>  
  summarize(  
    gdpPercap = median(gdpPercap),  
    lifeExp = median(lifeExp)  
  )  
  
ggplot(data = gap_1997, mapping = aes(x = gdpPercap, y = lifeExp, color = continent)) +
```

```
geom_point(alpha = 0.45, show.legend = FALSE) +
geom_text_repel(
  data = continent_summary,
  mapping = aes(label = continent),
  size = 4,
  show.legend = FALSE
) +
scale_x_log10(labels = dollar_format(accuracy = 1)) +
scale_color_brewer(palette = "Set2") +
labs(
  title = "Direct Labels Can Replace a Legend",
  x = "GDP per capita",
  y = "Life expectancy",
  caption = "Source: Gapminder."
) +
theme_minimal()
```



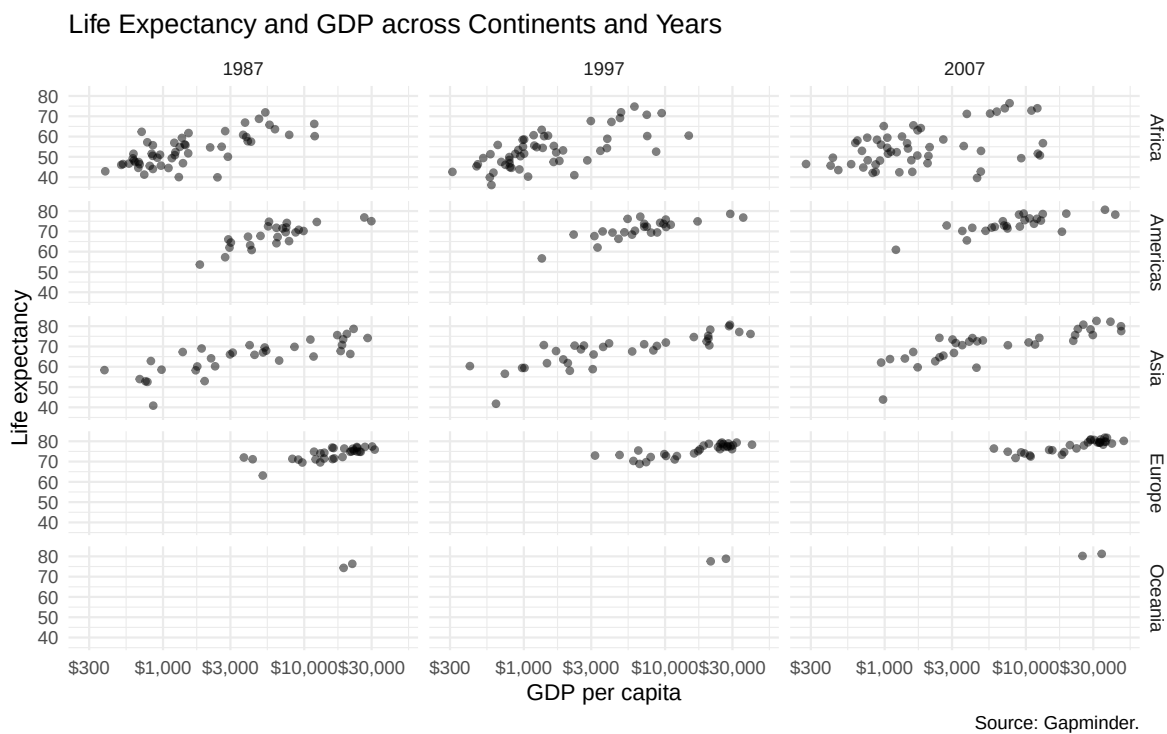
Direct labels work best when there are not too many groups and there is enough space for the labels.

5.8 facet_grid()

`facet_wrap()` is usually the easiest choice for one grouping variable. `facet_grid()` is useful when the panel layout has rows and columns with substantive meaning.

```
gap_recent <- gapminder |>
  filter(year %in% c(1987, 1997, 2007))

ggplot(data = gap_recent, mapping = aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha = 0.5, size = 1.3) +
  scale_x_log10(labels = dollar_format(accuracy = 1)) +
  facet_grid(continent ~ year) +
  labs(
    title = "Life Expectancy and GDP across Continents and Years",
    x = "GDP per capita",
    y = "Life expectancy",
    caption = "Source: Gapminder."
  ) +
  theme_minimal()
```



Read `facet_grid(continent ~ year)` as “continent defines the rows, year defines the columns.”

5.9 Patchwork

Facets repeat the same plot structure across groups. Sometimes you want to combine different plots into one figure. The `patchwork` package lets you assemble separate `ggplot` objects.

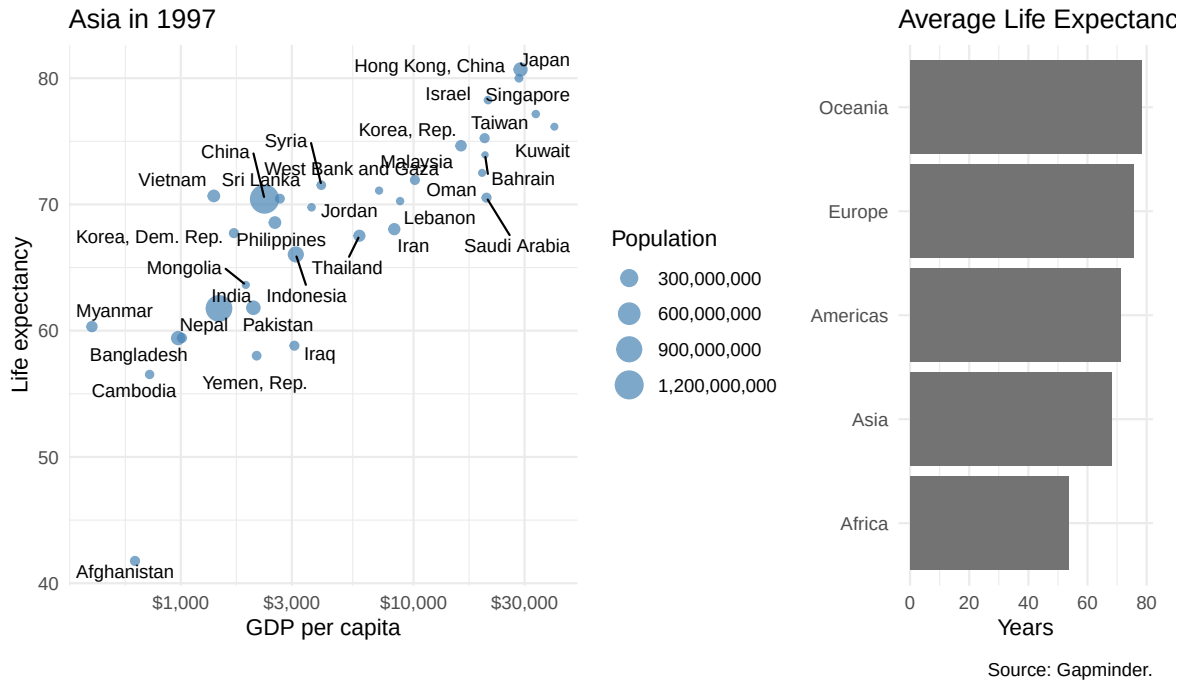
```
asia_scatter <- gap_1997 |>
  filter(continent == "Asia") |>
  ggplot(mapping = aes(x = gdpPercap, y = lifeExp)) +
  geom_point(aes(size = pop), color = "steelblue", alpha = 0.7) +
  geom_text_repel(aes(label = country), size = 3, max.overlaps = 12) +
  scale_x_log10(labels = dollar_format(accuracy = 1)) +
  scale_size_continuous(labels = comma) +
  labs(
    title = "Asia in 1997",
    x = "GDP per capita",
    y = "Life expectancy",
    size = "Population"
  ) +
  theme_minimal()

continent_bar <- gap_1997 |>
  group_by(continent) |>
  summarize(avg_life_exp = mean(lifeExp)) |>
  ggplot(mapping = aes(x = reorder(continent, avg_life_exp), y = avg_life_exp)) +
  geom_col(fill = "gray45") +
  coord_flip() +
  labs(
    title = "Average Life Expectancy",
    x = NULL,
    y = "Years"
  ) +
  theme_minimal()

asia_scatter + continent_bar +
  plot_layout(widths = c(2, 1)) +
  plot_annotation(
    title = "Two Views of Gapminder Data",
```

```
caption = "Source: Gapminder."
)
```

Two Views of Gapminder Data



Source: Gapminder.

5.10 Extra: State Policy Snapshot

State-level political and policy data work well for comparison examples. This section uses a small local file from the course `Data/` folder.

The inspection function `glimpse()` comes from `dplyr`, which is loaded as part of the `tidyverse`.

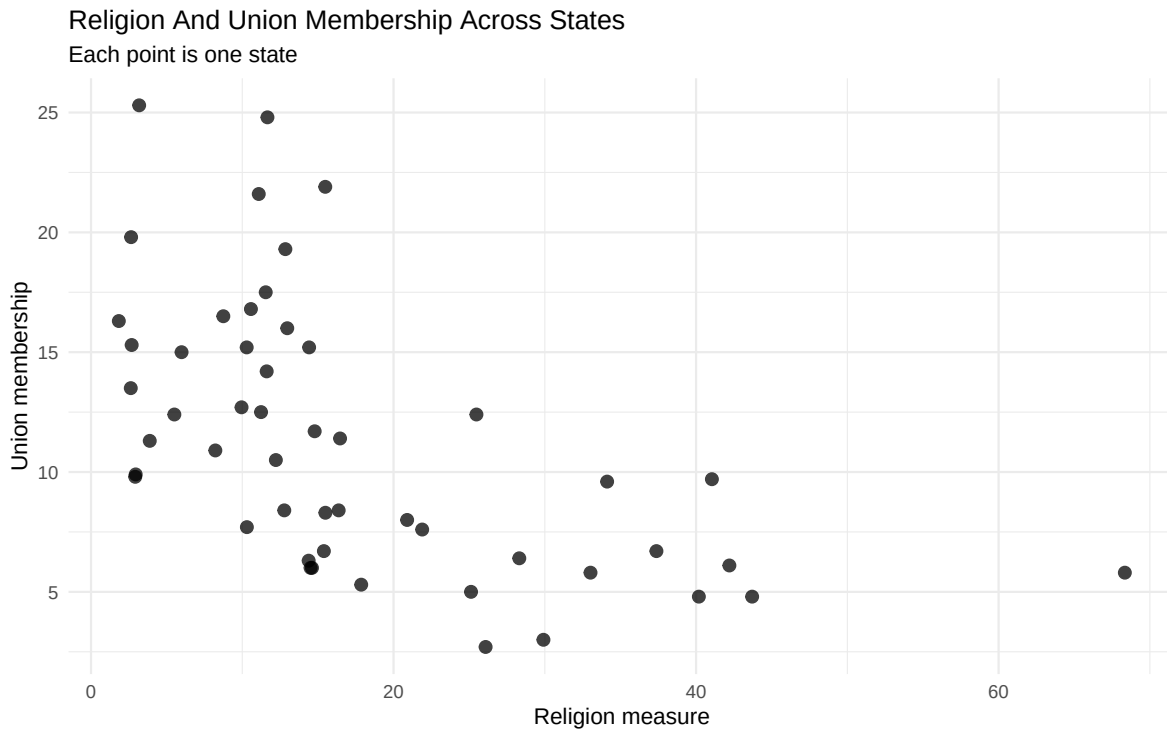
```
state_policy <- read_csv(
  "Data/state_policy/state_data_sorted.csv",
  show_col_types = FALSE
) |>
mutate(
  union_pct = 100 * union,
  religion_pct = 100 * religion,
  region = str_to_title(cregion)
)
```

```
glimpse(state_policy)
```

```
Rows: 51
Columns: 11
$ abb      <chr> "AK", "AL", "AR", "AZ", "CA", "CO", "CT", "DC", "DE", "FL~
$ union    <dbl> 0.219, 0.097, 0.048, 0.063, 0.165, 0.084, 0.153, 0.127, 0~
$ religion  <dbl> 0.1548637, 0.4104603, 0.4370625, 0.1438850, 0.0874711, 0.~
$ cst      <chr> "AK", "AL", "AR", "AZ", "CA", "CO", "CT", "DC", "DE", "FL~
$ cregion   <chr> "west", "south", "south", "west", "west", "west", "northe~
$ region_num <dbl> 4, 3, 3, 4, 4, 4, 2, 2, 3, 3, 3, 4, 1, 4, 1, 1, 1, 3, 3, ~
$ state_num <dbl> 50, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 51, 11, 12, 13, 14, 15~
$ pop      <dbl> 626932, 4447100, 2673400, 5130632, 33871648, 4301261, 340~
$ union_pct <dbl> 21.9, 9.7, 4.8, 6.3, 16.5, 8.4, 15.3, 12.7, 12.4, 6.0, 6.~
$ religion_pct <dbl> 15.48637, 41.04603, 43.70625, 14.38850, 8.74711, 12.77672~
$ region    <chr> "West", "South", "South", "West", "West", "West", "Northe~
```

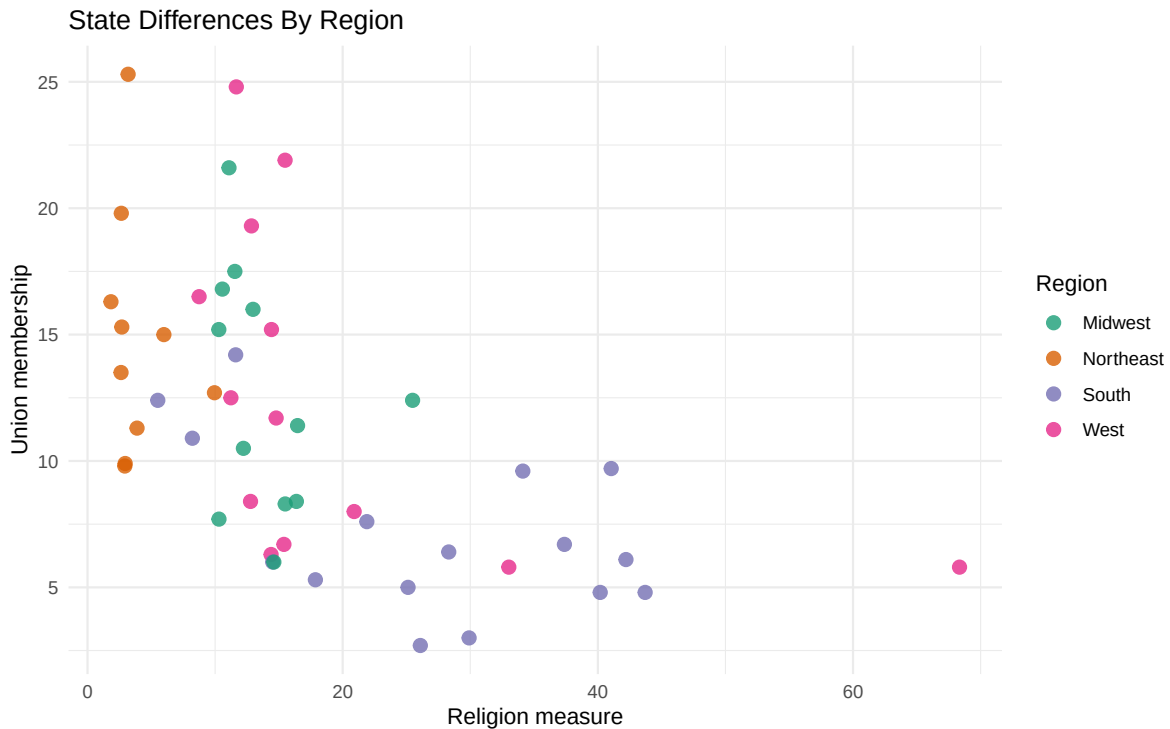
The two variables in the next plot are percentages. The point is not to make a causal claim. The point is to practice separating and comparing state-level observations.

```
ggplot(
  state_policy,
  aes(x = religion_pct, y = union_pct)
) +
  geom_point(size = 2.5, alpha = 0.75) +
  labs(
    title = "Religion And Union Membership Across States",
    subtitle = "Each point is one state",
    x = "Religion measure",
    y = "Union membership"
  ) +
  theme_minimal()
```



Color can separate regions without creating multiple panels. This is useful when the plot is still readable with all observations in one place.

```
ggplot(
  state_policy,
  aes(x = religion_pct, y = union_pct, color = region)
) +
  geom_point(size = 2.8, alpha = 0.8) +
  scale_color_brewer(palette = "Dark2") +
  labs(
    title = "State Differences By Region",
    x = "Religion measure",
    y = "Union membership",
    color = "Region"
  ) +
  theme_minimal()
```

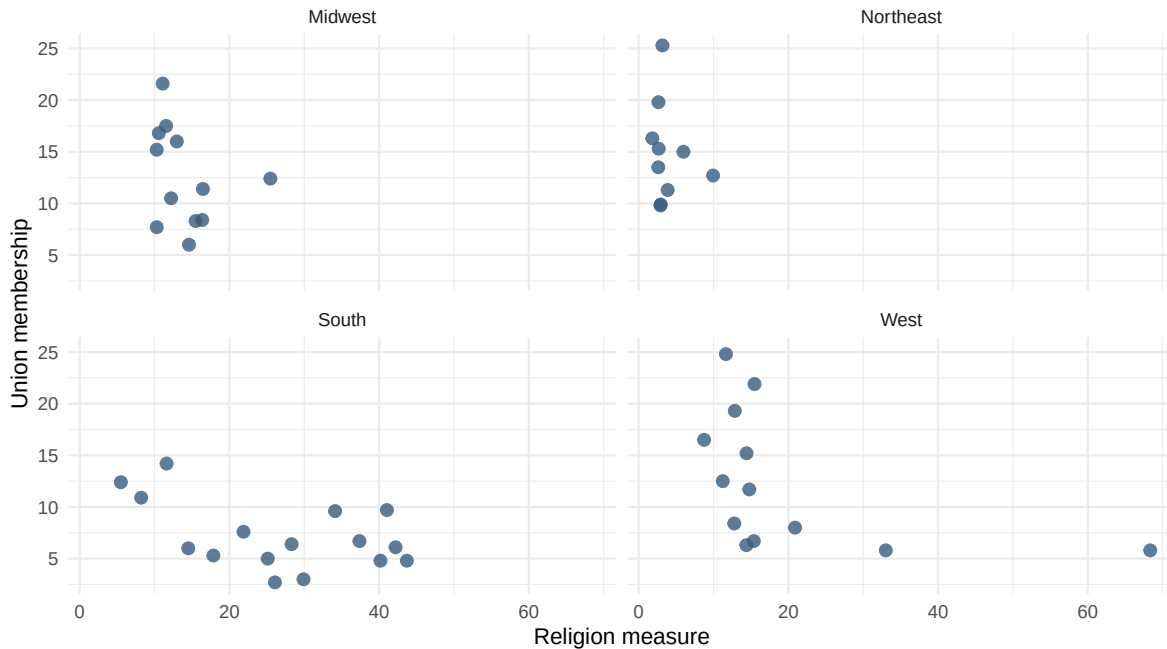


Facets split the same data into small multiples. This makes regional comparisons clearer when the color version starts to feel crowded.

```
ggplot(
  state_policy,
  aes(x = religion_pct, y = union_pct)
) +
  geom_point(color = "#355c7d", size = 2.5, alpha = 0.8) +
  facet_wrap(~ region) +
  labs(
    title = "State Differences By Region",
    subtitle = "Facets separate the same scatterplot into regional panels",
    x = "Religion measure",
    y = "Union membership"
  ) +
  theme_minimal()
```


State Differences By Region

Facets separate the same scatterplot into regional panels

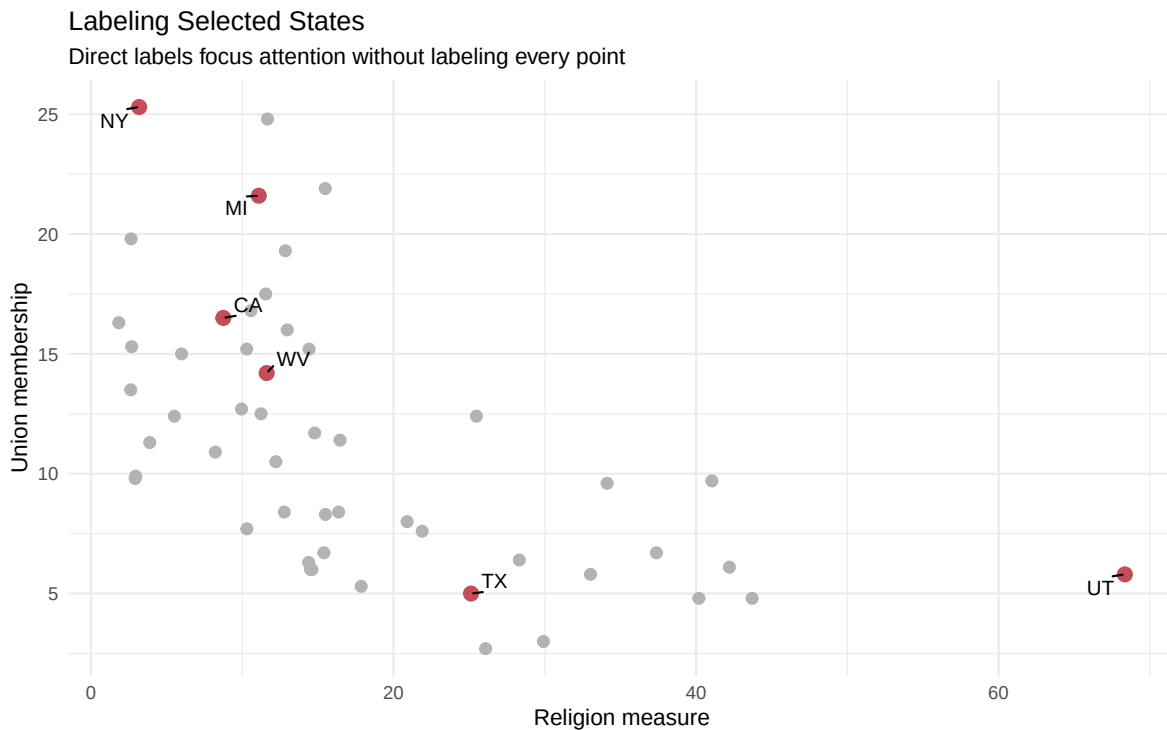


Direct labels are useful when a few cases matter. Labeling every state would crowd the figure, so the next example labels only selected states.

```
label_states <- c("CA", "MI", "NY", "TX", "UT", "WV")

ggplot(
  state_policy,
  aes(x = religion_pct, y = union_pct)
) +
  geom_point(color = "gray70", size = 2.3) +
  geom_point(
    data = filter(state_policy, abb %in% label_states),
    color = "#c44d58",
    size = 3
  ) +
  geom_text_repel(
    data = filter(state_policy, abb %in% label_states),
    aes(label = abb),
    size = 3.5,
    min.segment.length = 0
  ) +
```

```
labs(
  title = "Labeling Selected States",
  subtitle = "Direct labels focus attention without labeling every point",
  x = "Religion measure",
  y = "Union membership"
) +
theme_minimal()
```



5.11 Extra: PISA international scores

The PISA dataset contains results from the 2006 OECD Programme for International Student Assessment. Each of the roughly 152,000 rows is one student. Variables include country, gender, and average scores in math, reading, and science.

```
pisa <- read_csv("Data/pisa/pisa_data_clean.csv", show_col_types = FALSE) |>
  janitor::clean_names()

pisa |>
```

```
select(country_code, gender, average_math_score, average_reading_score, average_science_score)
slice_head(n = 6)
```

```
# A tibble: 6 x 5
  country_code gender average_math_score average_reading_score
  <chr>         <chr>           <dbl>             <dbl>
1 Albania      Female           490.             372.
2 Albania      Female           434.             402.
3 Albania      Female           439.             551.
4 Albania      Female           417.             350.
5 Albania      Female           453.             458.
6 Albania      Female           385.             398.
# i 1 more variable: average_science_score <dbl>
```

Summarize to country-gender means to reduce 152k rows to a manageable comparison dataset.

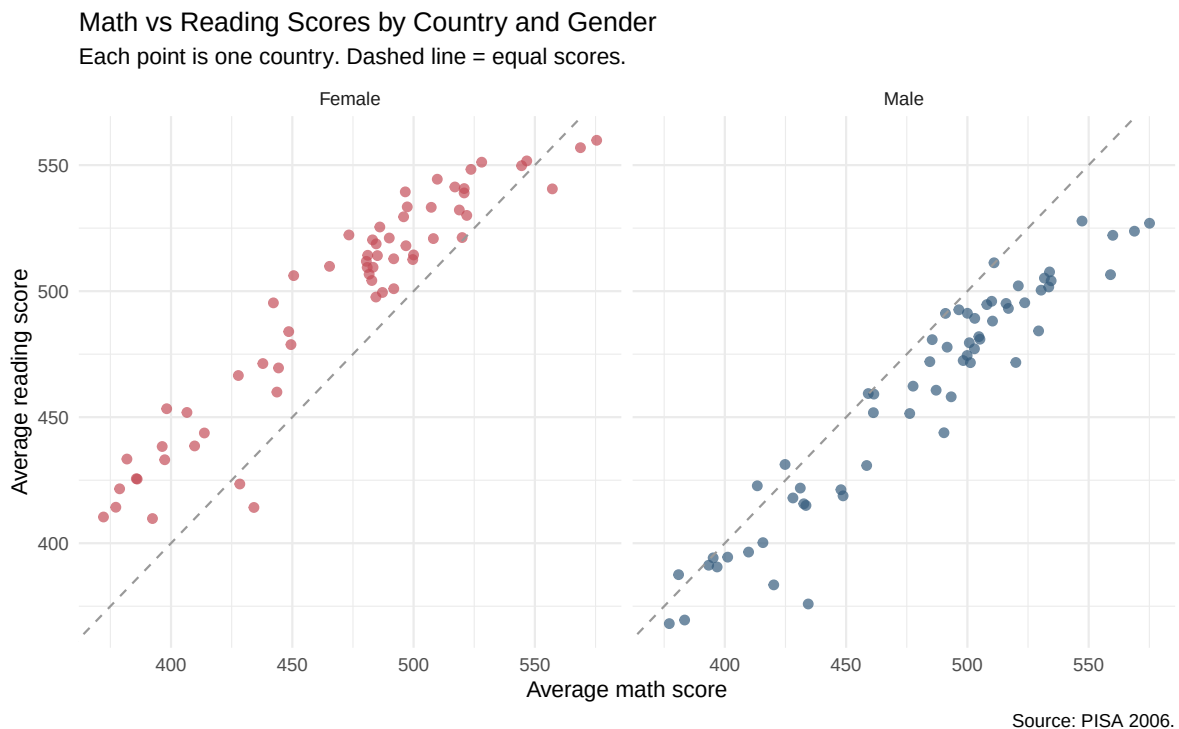
```
pisa_country <- pisa |>
  group_by(country_code, gender) |>
  summarize(
    math      = mean(average_math_score, na.rm = TRUE),
    reading   = mean(average_reading_score, na.rm = TRUE),
    science   = mean(average_science_score, na.rm = TRUE),
    .groups = "drop"
  )

pisa_country |> slice_head(n = 6)
```

```
# A tibble: 6 x 5
  country_code gender  math reading science
  <chr>         <chr> <dbl>   <dbl>   <dbl>
1 Albania      Female  392.   410.   403.
2 Albania      Male    395.   394.   400.
3 Argentina    Female  397.   433.   422.
4 Argentina    Male    410.   396.   416.
5 Australia    Female  490.   521.   510.
6 Australia    Male    503.   489.   516.
```

A scatterplot of math against reading scores, colored by gender, shows how the two subjects correlate across countries. Faceting by gender lets us compare the pattern for male and female students side by side.

```
ggplot(pisa_country, aes(x = math, y = reading, color = gender)) +
  geom_point(alpha = 0.7, size = 1.8) +
  geom_abline(linetype = "dashed", color = "gray60") +
  facet_wrap(~ gender) +
  scale_color_manual(values = c("Female" = "#c44d58", "Male" = "#355c7d")) +
  labs(
    title = "Math vs Reading Scores by Country and Gender",
    subtitle = "Each point is one country. Dashed line = equal scores.",
    x = "Average math score",
    y = "Average reading score",
    caption = "Source: PISA 2006."
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```



Countries cluster above the dashed line for female students (reading > math) and below for male students (math > reading). The facet makes this systematic gender gap visible at a glance.

We can also rank countries by math score and look at the top and bottom performers.

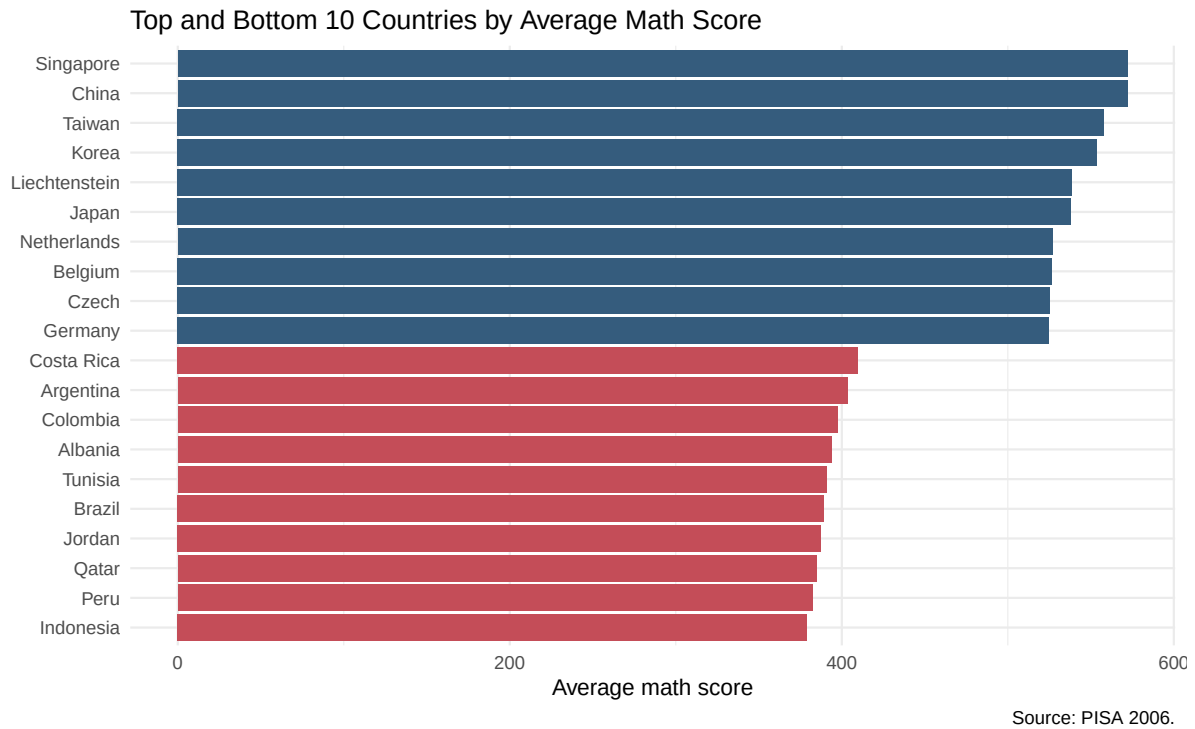
```

pisa_overall <- pisa |>
  group_by(country_code) |>
  summarize(
    math      = mean(average_math_score, na.rm = TRUE),
    reading   = mean(average_reading_score, na.rm = TRUE),
    .groups = "drop"
  )

top_bottom <- pisa_overall |>
  slice_max(math, n = 10) |>
  bind_rows(pisa_overall |> slice_min(math, n = 10)) |>
  mutate(group = if_else(math >= median(pisa_overall$math), "Top 10", "Bottom 10"))

ggplot(top_bottom, aes(x = math, y = reorder(country_code, math), fill = group)) +
  geom_col(show.legend = FALSE) +
  scale_fill_manual(values = c("Top 10" = "#355c7d", "Bottom 10" = "#c44d58")) +
  labs(
    title = "Top and Bottom 10 Countries by Average Math Score",
    x = "Average math score",
    y = NULL,
    caption = "Source: PISA 2006."
  ) +
  theme_minimal()

```



5.12 Exercise

Use `gapminder` to make a comparison plot for 2007.

1. Filter the data to `year == 2007`.
2. Make a scatterplot of GDP per capita and life expectancy.
3. Separate continents using color.
4. Make a second version using `facet_wrap(~ continent)`.
5. Decide which version makes the comparison clearer and write one sentence explaining why.

6 Distributions

6.1 Why distributions matter

A distribution shows how values of a variable are spread out. Before comparing means, fitting models, or making polished charts, it is often useful to ask simpler questions:

- What values are common?
- Are there extreme values?
- Is the variable symmetric or skewed?
- Do groups have similar or different spreads?
- Are there gaps, clusters, or unusual cases?

Distribution plots are especially useful during exploratory data analysis.

6.2 Histograms

A histogram divides a continuous variable into bins and counts how many observations fall in each bin.

The `midwest` dataset is included with `ggplot2`. It has one row per county in five Midwestern states.

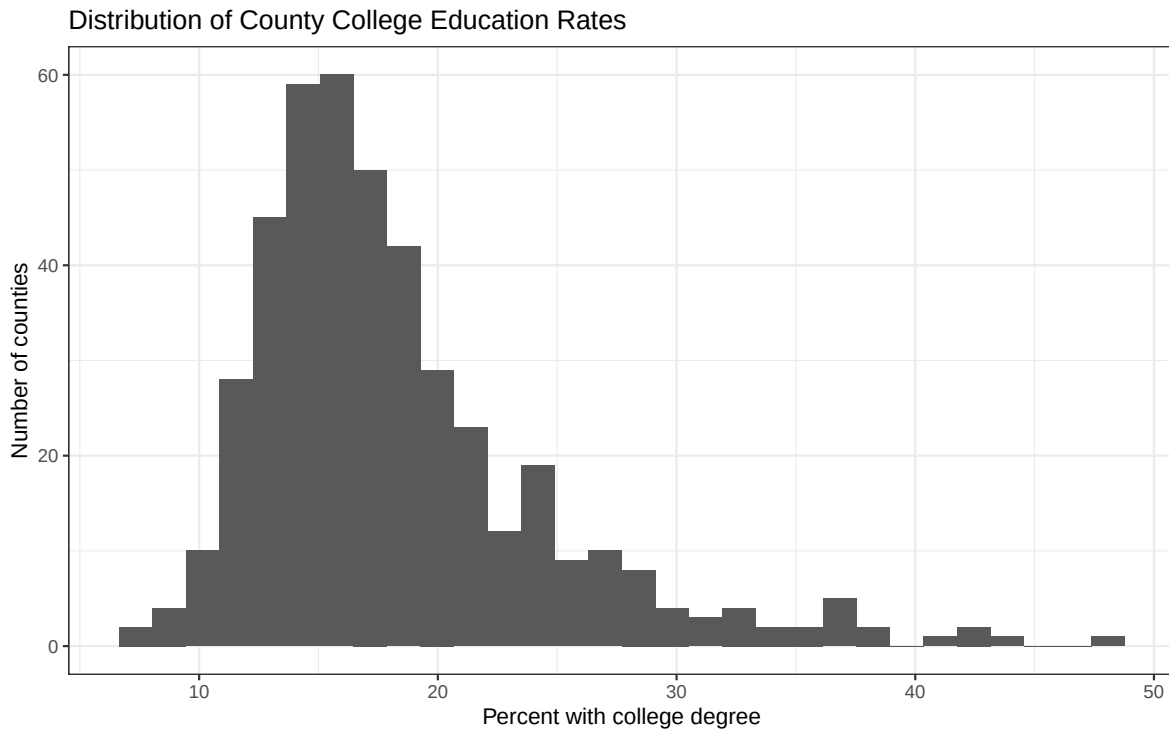
```
midwest |>
  select(state, county, percollege, poptotal, area) |>
  head(10)
```

```
# A tibble: 10 x 5
   state county  percollege poptotal  area
   <chr> <chr>      <dbl>    <int> <dbl>
1 IL    ADAMS      19.6     66090 0.052
2 IL    ALEXANDER   11.2     10626 0.014
3 IL    BOND       17.0     14991 0.022
4 IL    BOONE      17.3     30806 0.017
5 IL    BROWN     14.5      5836 0.018
```

6	IL	BUREAU	18.9	35688	0.05
7	IL	CALHOUN	11.9	5322	0.017
8	IL	CARROLL	16.2	16805	0.027
9	IL	CASS	14.1	13437	0.024
10	IL	CHAMPAIGN	41.3	173025	0.058

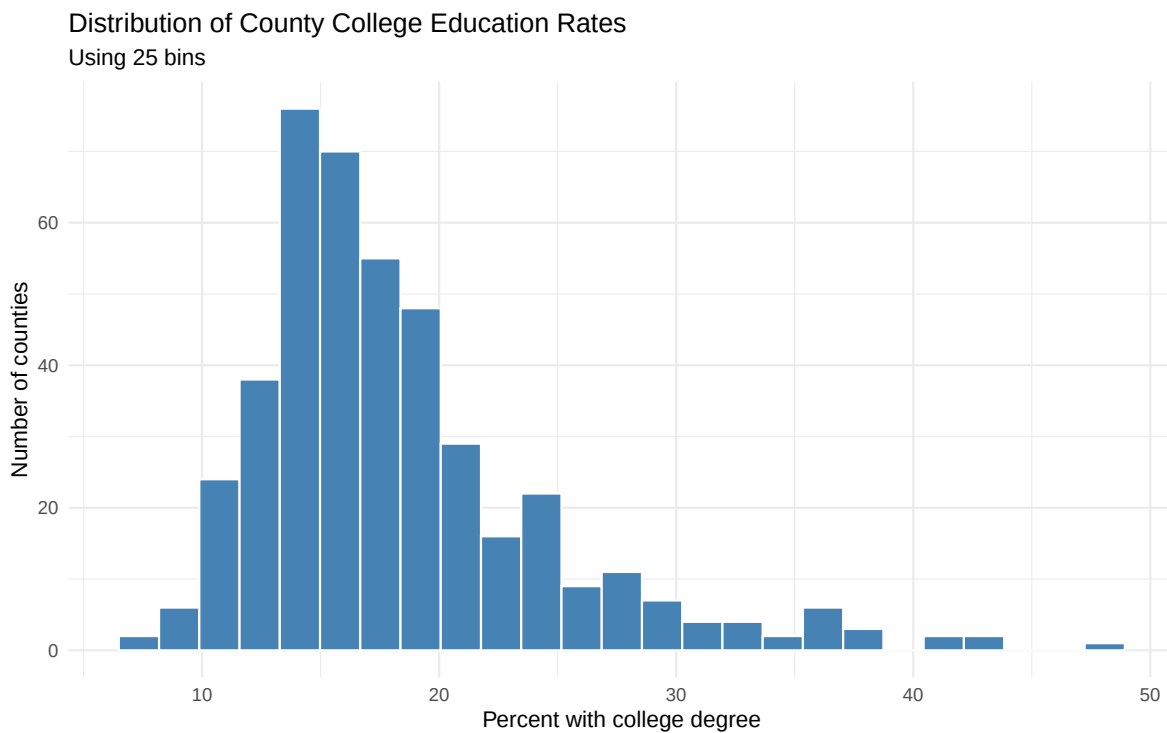
We will start with `percollege`, the percentage of adults in each county with a college degree.

```
ggplot(
  data = midwest |> filter(!is.na(percollege)),
  mapping = aes(x = percollege)
) +
  geom_histogram() +
  labs(
    title = "Distribution of County College Education Rates",
    x = "Percent with college degree",
    y = "Number of counties"
  ) +
  theme_bw()
```



When we do not set the number of bins, `ggplot2` chooses a default. That default is not always the best choice, so it is worth trying several values.


```
ggplot(
  data = midwest |> filter(!is.na(percollege)),
  mapping = aes(x = percollege)
) +
  geom_histogram(bins = 25, fill = "steelblue", color = "white") +
  labs(
    title = "Distribution of County College Education Rates",
    subtitle = "Using 25 bins",
    x = "Percent with college degree",
    y = "Number of counties"
  ) +
  theme_minimal()
```



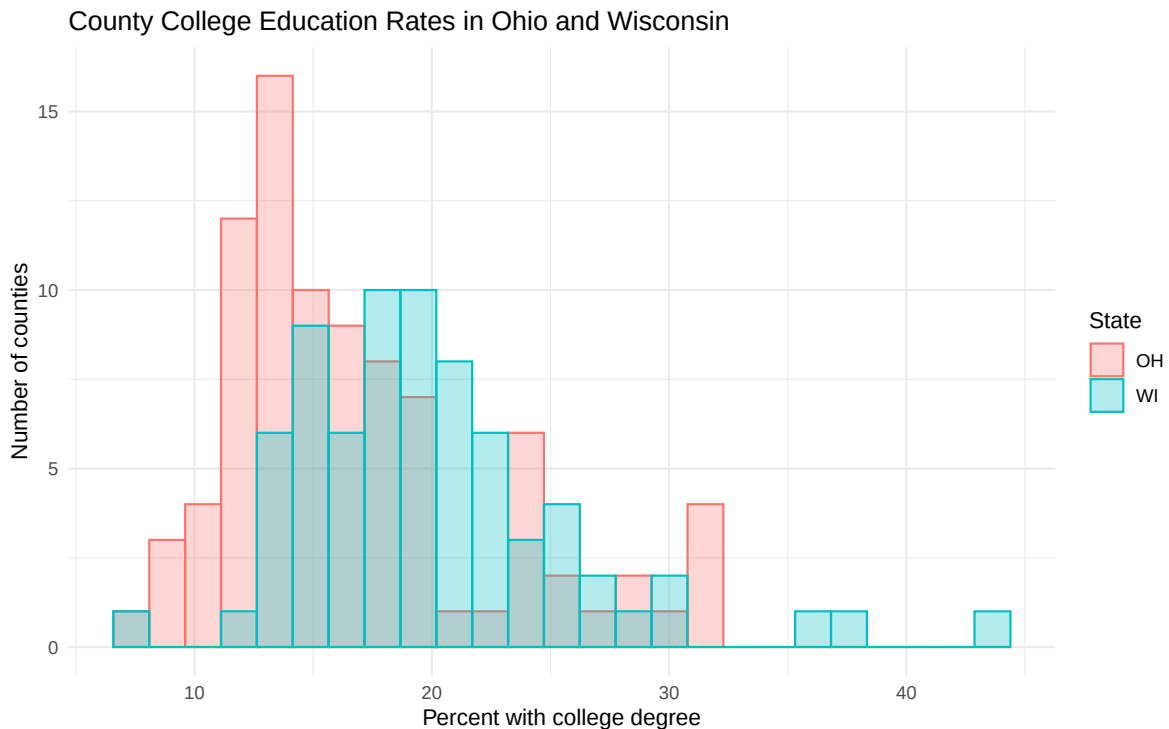
Changing the number of bins changes the visual summary. Very few bins can hide structure. Too many bins can make the plot look noisy. Histograms are useful for seeing skew, clustering, and unusual tails.

6.3 Comparing histograms

We can compare distributions across groups by mapping a group to fill and using transparency.

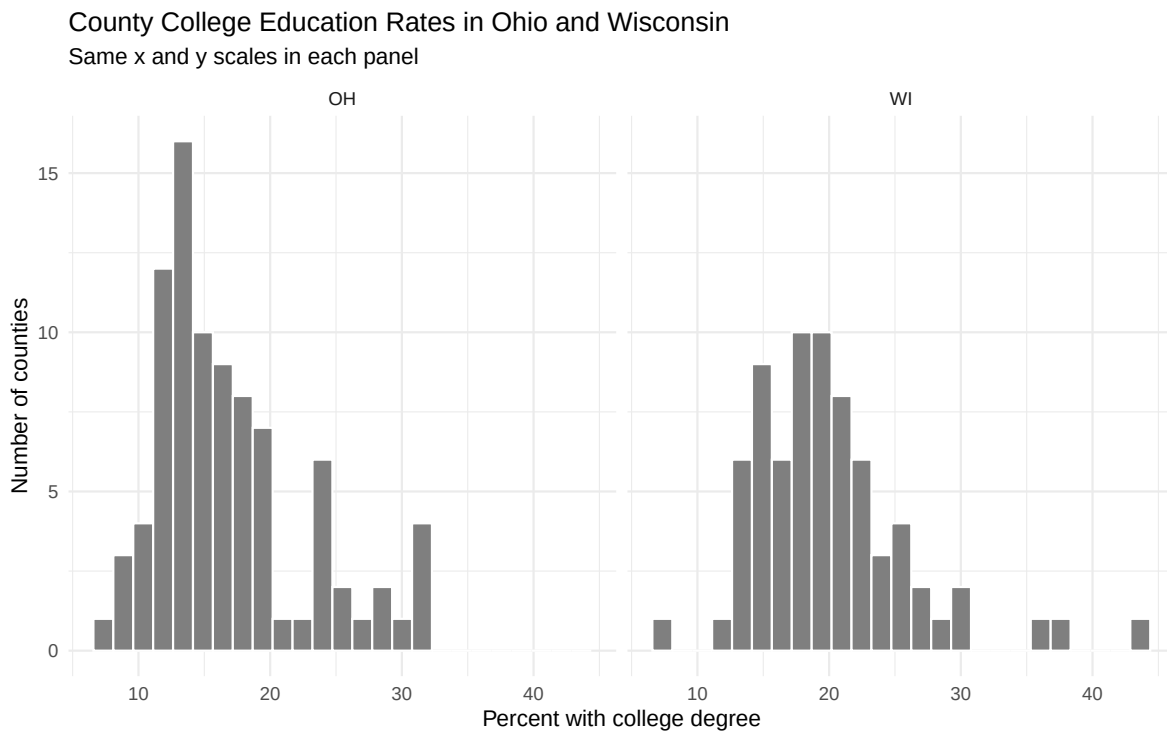
```
oh_wi <- midwest |>
  filter(state %in% c("OH", "WI"), !is.na(percollege))

ggplot(data = oh_wi, mapping = aes(x = percollege, fill = state, color = state)) +
  geom_histogram(alpha = 0.3, bins = 25, position = "identity") +
  labs(
    title = "County College Education Rates in Ohio and Wisconsin",
    x = "Percent with college degree",
    y = "Number of counties",
    fill = "State",
    color = "State"
  ) +
  theme_minimal()
```



Overlaid histograms are compact, but they can be difficult to read when the groups overlap. Small multiples are often clearer.

```
ggplot(data = oh_wi, mapping = aes(x = percollege)) +
  geom_histogram(bins = 25, fill = "gray50", color = "white") +
  facet_wrap(~ state) +
  labs(
    title = "County College Education Rates in Ohio and Wisconsin",
    subtitle = "Same x and y scales in each panel",
    x = "Percent with college degree",
    y = "Number of counties"
  ) +
  theme_minimal()
```

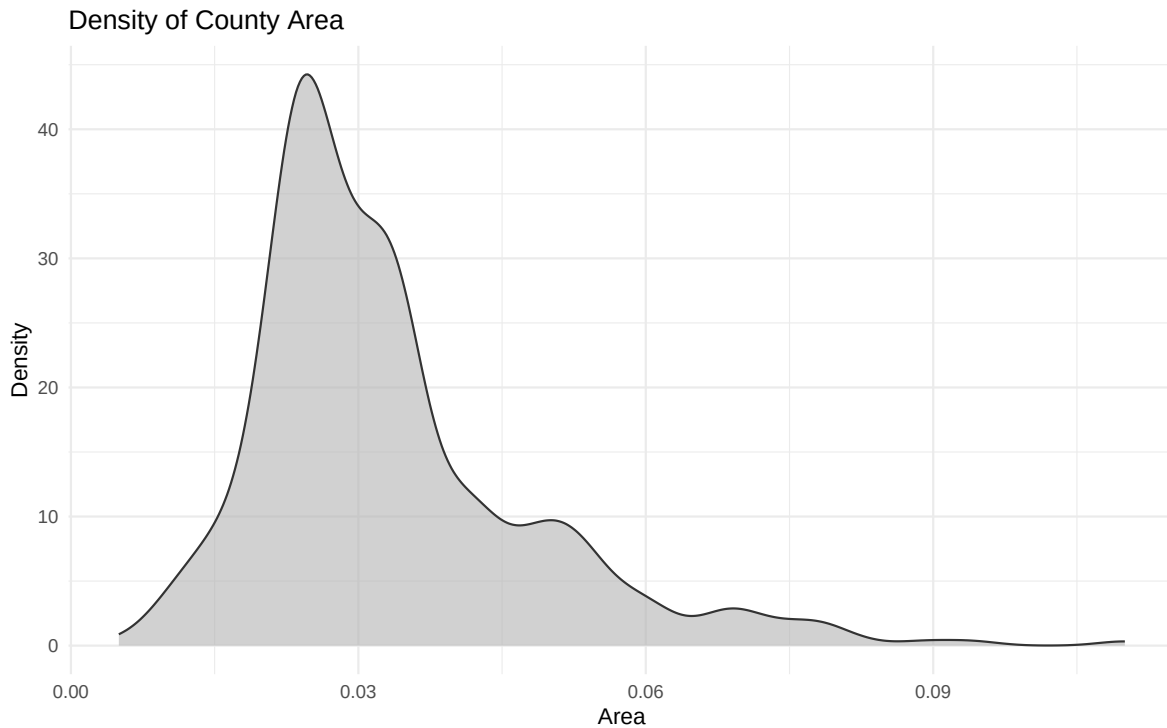


Because the panels share scales by default, the two histograms can be compared directly.

6.4 Density plots

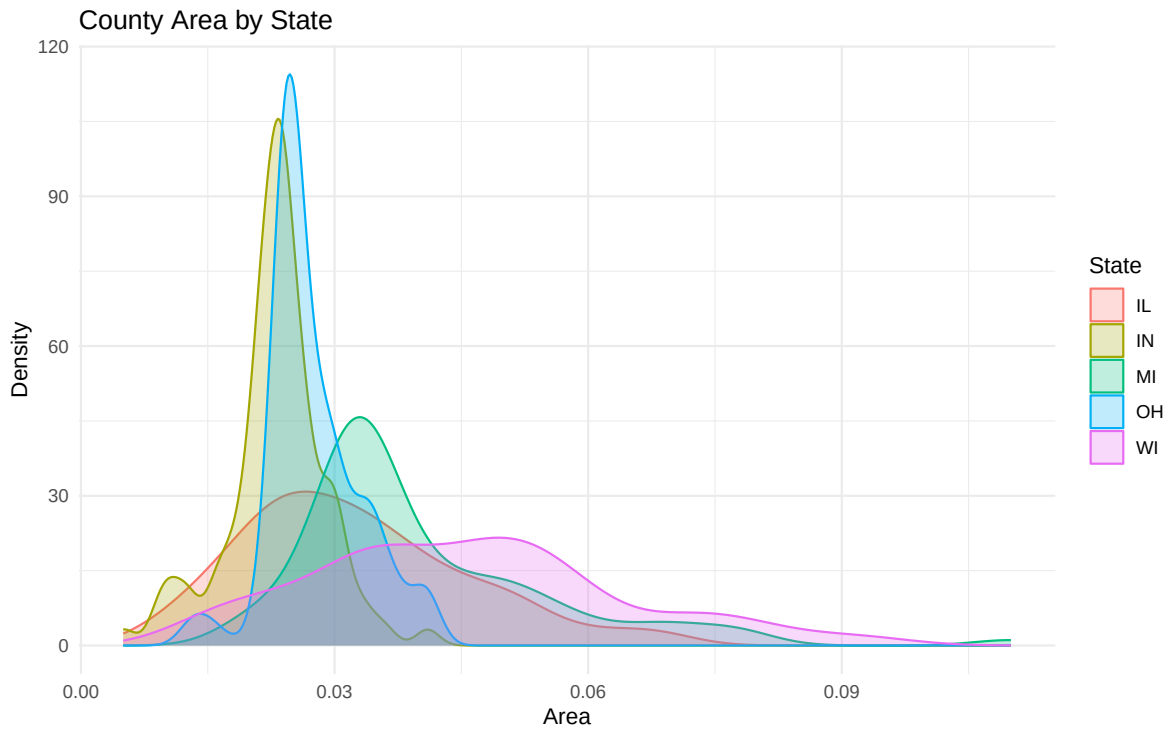
A density plot is a smoothed version of a distribution. Instead of showing counts in bins, it estimates the shape of the distribution.

```
ggplot(data = midwest, mapping = aes(x = area)) +
  geom_density(fill = "gray70", color = "gray20", alpha = 0.6) +
  labs(
    title = "Density of County Area",
    x = "Area",
    y = "Density"
  ) +
  theme_minimal()
```



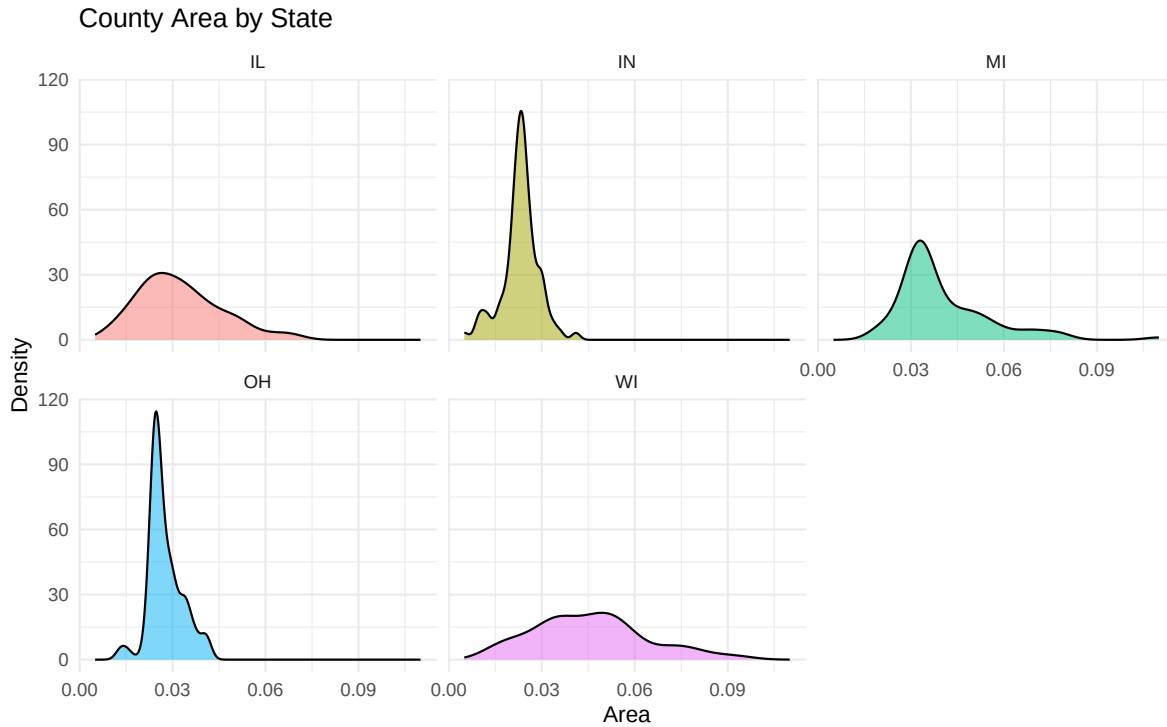
Density plots are useful for comparing shapes across groups.

```
ggplot(data = midwest, mapping = aes(x = area, fill = state, color = state)) +
  geom_density(alpha = 0.25) +
  labs(
    title = "County Area by State",
    x = "Area",
    y = "Density",
    fill = "State",
    color = "State"
  ) +
  theme_minimal()
```



When there are several groups, faceting can again reduce clutter.

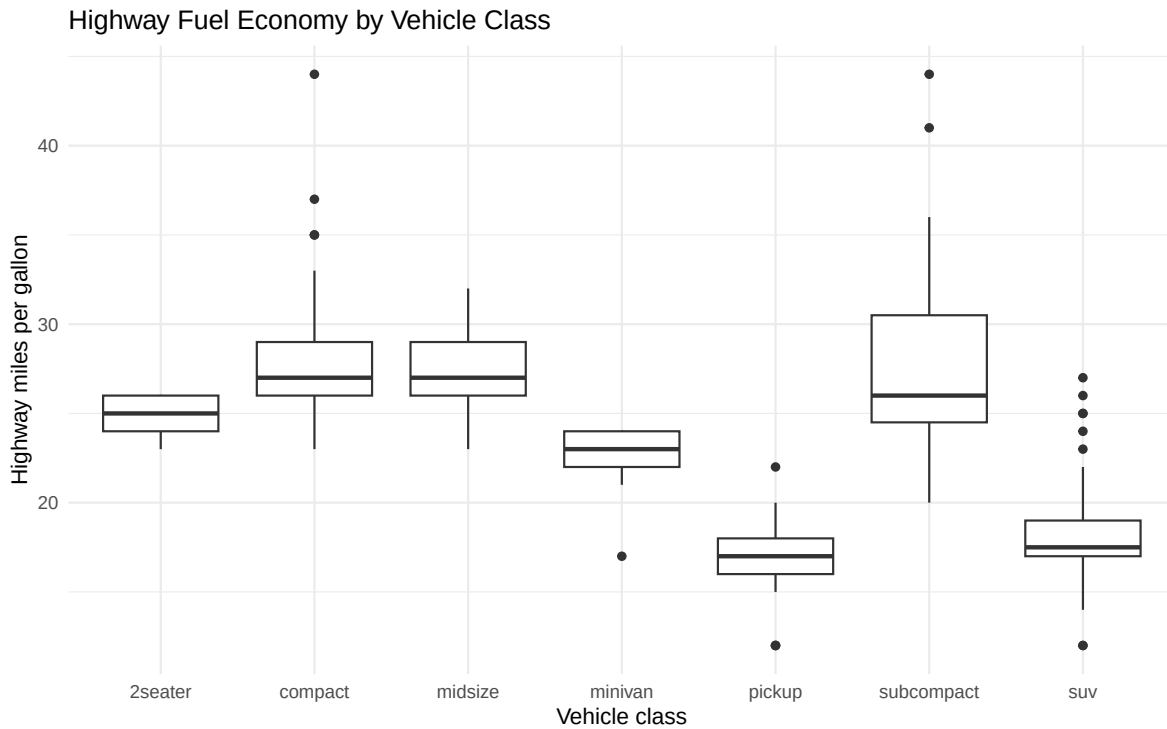
```
ggplot(data = midwest, mapping = aes(x = area, fill = state)) +  
  geom_density(alpha = 0.5, show.legend = FALSE) +  
  facet_wrap(~ state) +  
  labs(  
    title = "County Area by State",  
    x = "Area",  
    y = "Density"  
  ) +  
  theme_minimal()
```



6.5 Boxplots

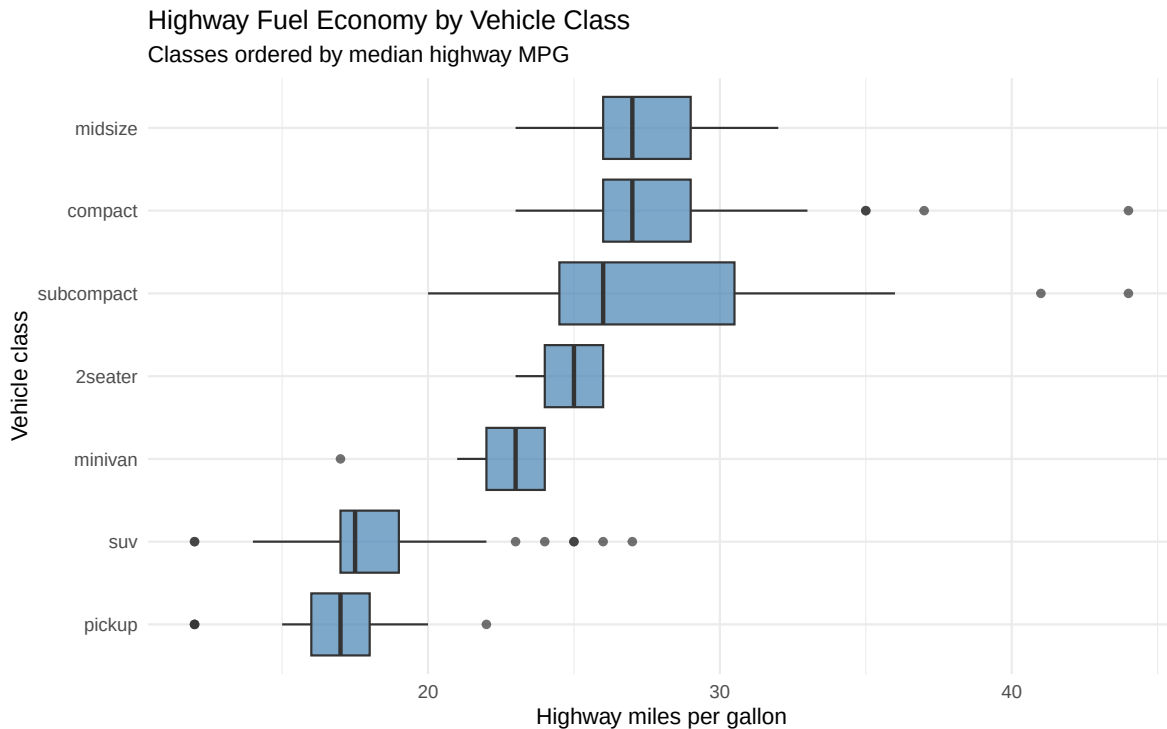
Boxplots summarize a distribution with a median, interquartile range, whiskers, and outliers. They are compact and good for comparing many groups.

```
ggplot(data = mpg, mapping = aes(x = class, y = hwy)) +
  geom_boxplot() +
  labs(
    title = "Highway Fuel Economy by Vehicle Class",
    x = "Vehicle class",
    y = "Highway miles per gallon"
  ) +
  theme_minimal()
```



The x-axis is crowded. We can reorder vehicle classes by median highway MPG and flip the coordinates.

```
ggplot(data = mpg, mapping = aes(x = reorder(class, hwy, FUN = median), y = hwy)) +
  geom_boxplot(fill = "steelblue", alpha = 0.7) +
  coord_flip() +
  labs(
    title = "Highway Fuel Economy by Vehicle Class",
    subtitle = "Classes ordered by median highway MPG",
    x = "Vehicle class",
    y = "Highway miles per gallon"
  ) +
  theme_minimal()
```

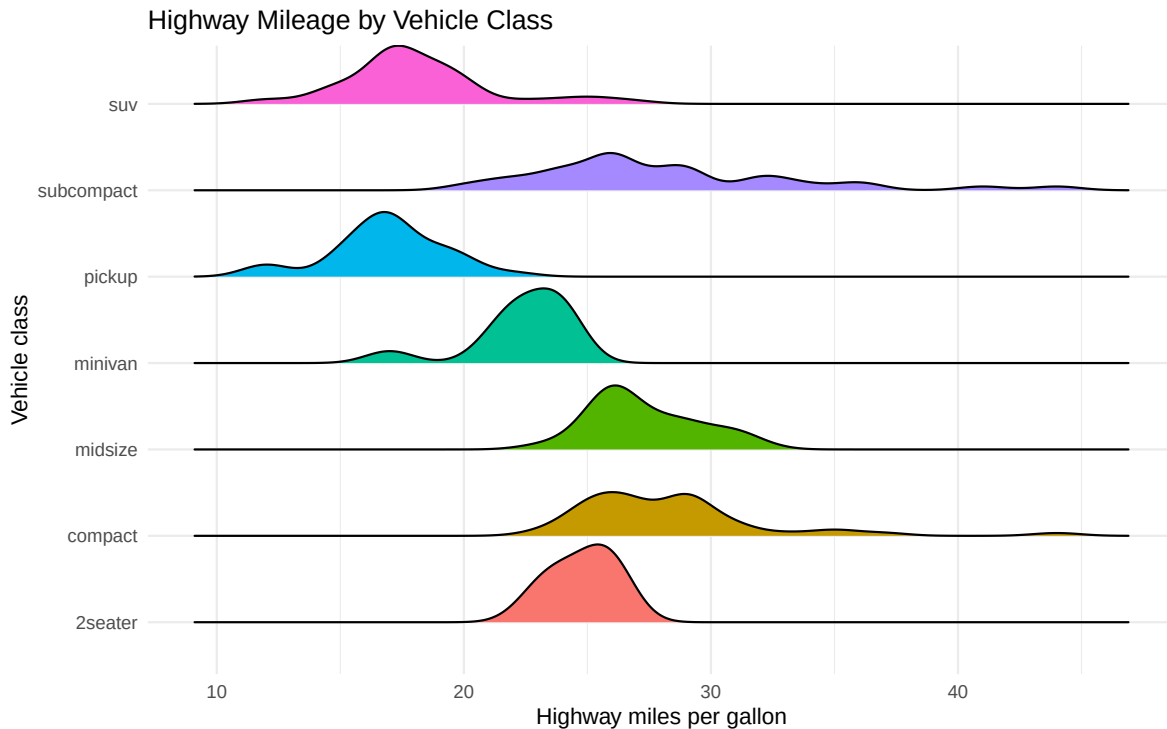


Boxplots hide individual observations, but they make the comparison of medians and spreads easy.

6.6 Ridge plots

Ridge plots stack density plots vertically. They are useful when you want to compare many distributions and still see the shape of each one. They use `geom_density_ridges()` from the `ggridges` package.

```
ggplot(mpg, aes(x = hwy, y = class, fill = class)) +
  ggridges::geom_density_ridges(scale = 0.9, show.legend = FALSE) +
  labs(
    title = "Highway Mileage by Vehicle Class",
    x = "Highway miles per gallon",
    y = "Vehicle class"
  ) +
  theme_minimal()
```

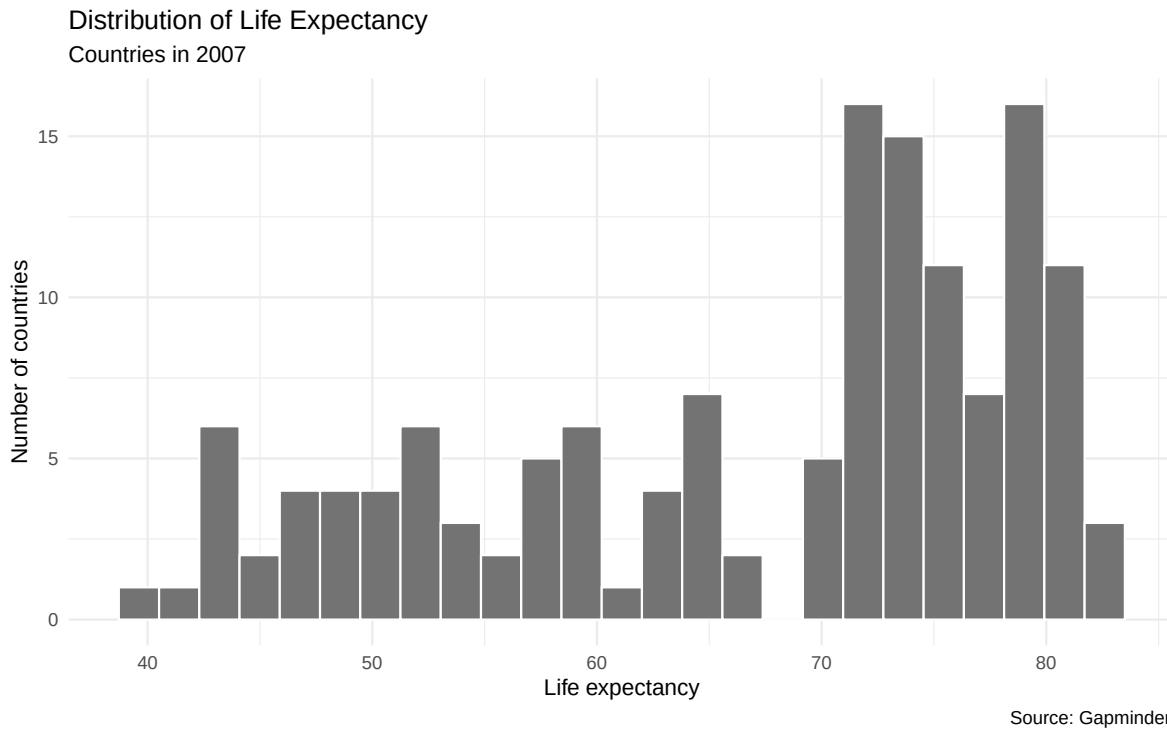
Ridge plots work best when there are enough categories to make a single boxplot crowded, but few enough that each density still has room to be readable.

6.7 Distributions in Gapminder

Distribution plots can also help before making cross-national comparisons.

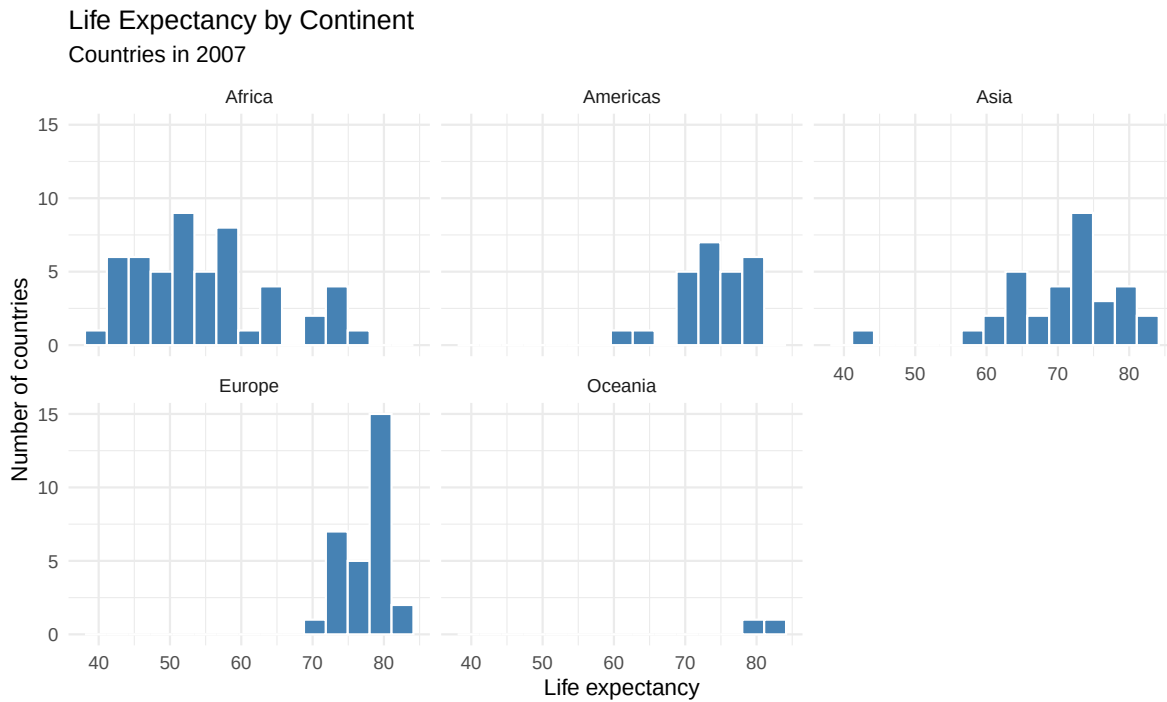
```
gap_2007 <- gapminder |>
  filter(year == 2007)

ggplot(data = gap_2007, mapping = aes(x = lifeExp)) +
  geom_histogram(bins = 25, fill = "gray45", color = "white") +
  labs(
    title = "Distribution of Life Expectancy",
    subtitle = "Countries in 2007",
    x = "Life expectancy",
    y = "Number of countries",
    caption = "Source: Gapminder."
  ) +
  theme_minimal()
```



Now separate by continent. Faceted histograms keep the same geom but split the data into panels.

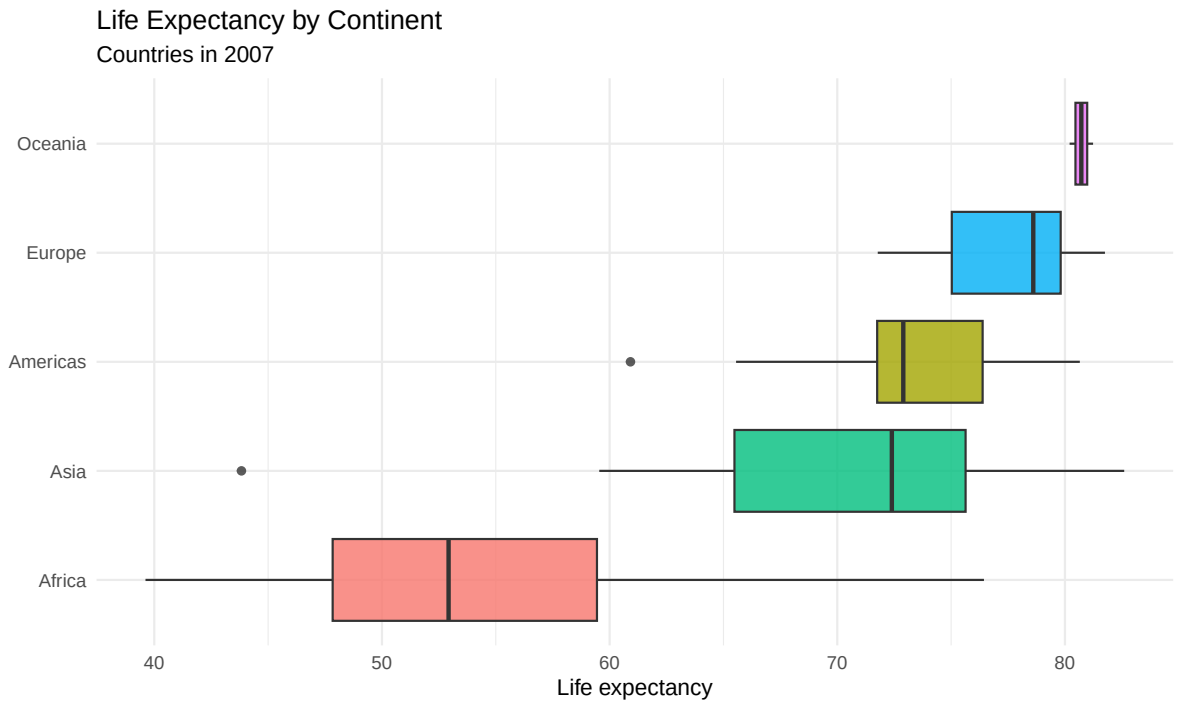
```
ggplot(data = gap_2007, mapping = aes(x = lifeExp)) +  
  geom_histogram(bins = 15, fill = "steelblue", color = "white") +  
  facet_wrap(~ continent) +  
  labs(  
    title = "Life Expectancy by Continent",  
    subtitle = "Countries in 2007",  
    x = "Life expectancy",  
    y = "Number of countries",  
    caption = "Source: Gapminder."  
  ) +  
  theme_minimal()
```



Source: Gapminder.

A boxplot gives a more compact comparison.

```
ggplot(data = gap_2007, mapping = aes(x = reorder(continent, lifeExp, FUN = median), y = lifeExp)) +
  geom_boxplot(aes(fill = continent), show.legend = FALSE, alpha = 0.8) +
  coord_flip() +
  labs(
    title = "Life Expectancy by Continent",
    subtitle = "Countries in 2007",
    x = NULL,
    y = "Life expectancy",
    caption = "Source: Gapminder."
  ) +
  theme_minimal()
```



6.8 Choosing among histograms, densities, and boxplots

Use a histogram when you want to show counts and binning is meaningful.

Use a density plot when you want to compare smooth distribution shapes.

Use a boxplot when you want compact comparisons across many groups.

No single distribution plot is always best. It is normal to make several versions while exploring the data.

6.9 Exercise 1: histogram

Use the built-in `iris` dataset.

1. Create a histogram of `Sepal.Length`.
2. Try `bins = 10`, `bins = 30`, and `bins = 50`.
3. Add a title and readable axis labels.
4. Write one sentence explaining which bin choice is easiest to interpret.

```
data(iris)
```

```
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

```
# Write your plot here.
```

6.10 Exercise 2: density plot

Use the built-in diamonds dataset.

1. Create a density plot of `carat`.
2. Map `cut` to fill.
3. Use `alpha` so the overlapping densities remain visible.
4. Try a faceted version with `facet_wrap(~ cut)`.

```
diamonds |>
  select(carat, cut, price) |>
  head(10)
```

```
# A tibble: 10 x 3
   carat cut      price
  <dbl> <ord>    <int>
1  0.23 Ideal      326
2  0.21 Premium    326
3  0.23 Good       327
4  0.29 Premium    334
5  0.31 Good       335
6  0.24 Very Good  336
7  0.24 Very Good  336
8  0.26 Very Good  337
9  0.22 Fair       337
10 0.23 Very Good  338
```

```
# Write your plot here.
```

6.11 Exercise 3: boxplot

Use the built-in `mpg` dataset.

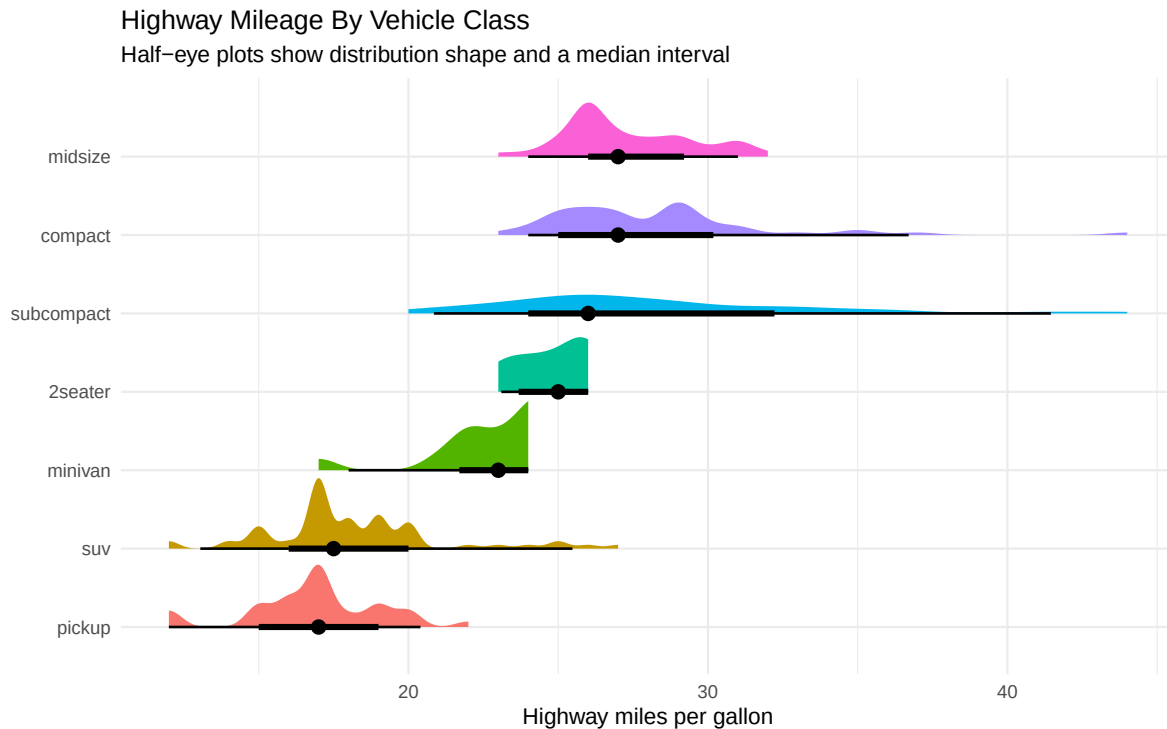
1. Compare `hwy` across vehicle `class`.
2. Reorder the classes by median `hwy`.
3. Flip the coordinates if the labels are hard to read.

```
# Write your plot here.
```

6.12 Extra: distribution intervals with `ggdist`

The `ggdist` package adds distribution geoms that combine several ideas at once. A half-eye plot shows the shape of a distribution while also marking a summary interval. It is useful when a boxplot feels too compressed but a full density plot takes too much space.

```
if (requireNamespace("ggdist", quietly = TRUE)) {  
  mpg |>  
    mutate(class = fct_reorder(class, hwy, .fun = median)) |>  
    ggplot(aes(x = hwy, y = class, fill = class)) +  
    ggdist::stat_halfeye(  
      adjust = 0.7,  
      width = 0.65,  
      point_interval = ggdist::median_qi,  
      show.legend = FALSE  
    ) +  
    labs(  
      title = "Highway Mileage By Vehicle Class",  
      subtitle = "Half-eye plots show distribution shape and a median interval",  
      x = "Highway miles per gallon",  
      y = NULL  
    ) +  
    theme_minimal()  
}
```

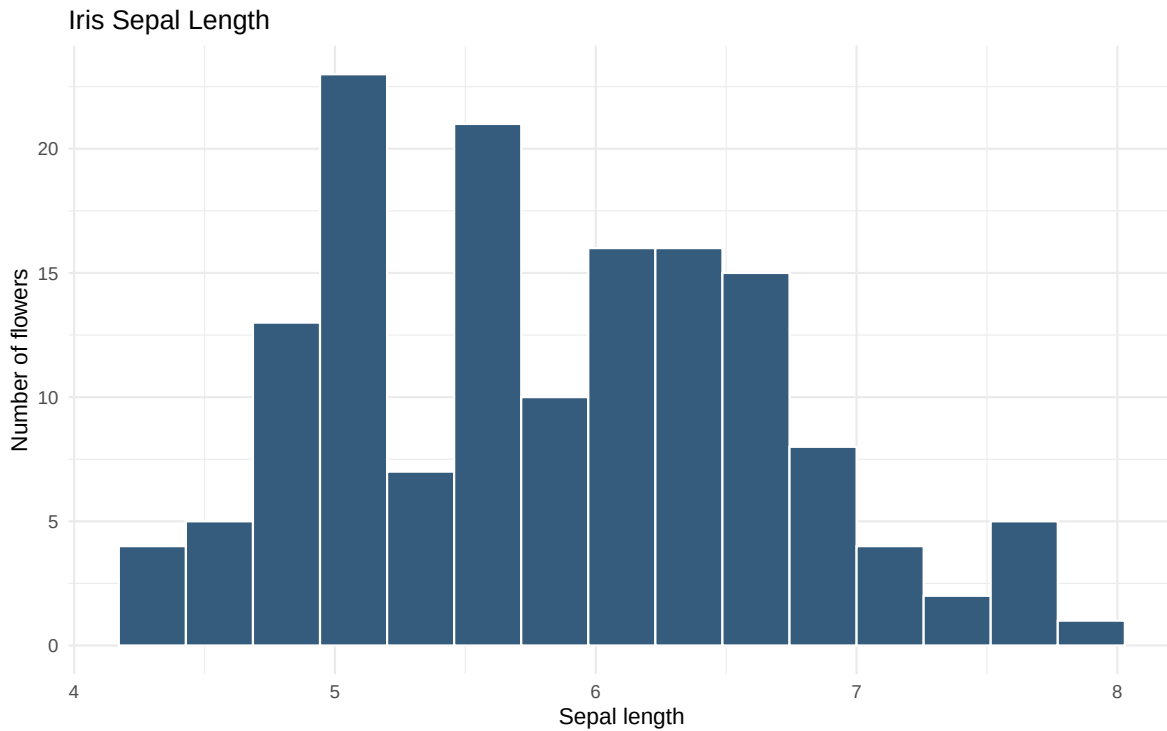


This is not a replacement for histograms, densities, or boxplots. It is another option when the goal is to show both distribution shape and uncertainty or spread in a compact display.

6.13 Extra: worked exercise variants

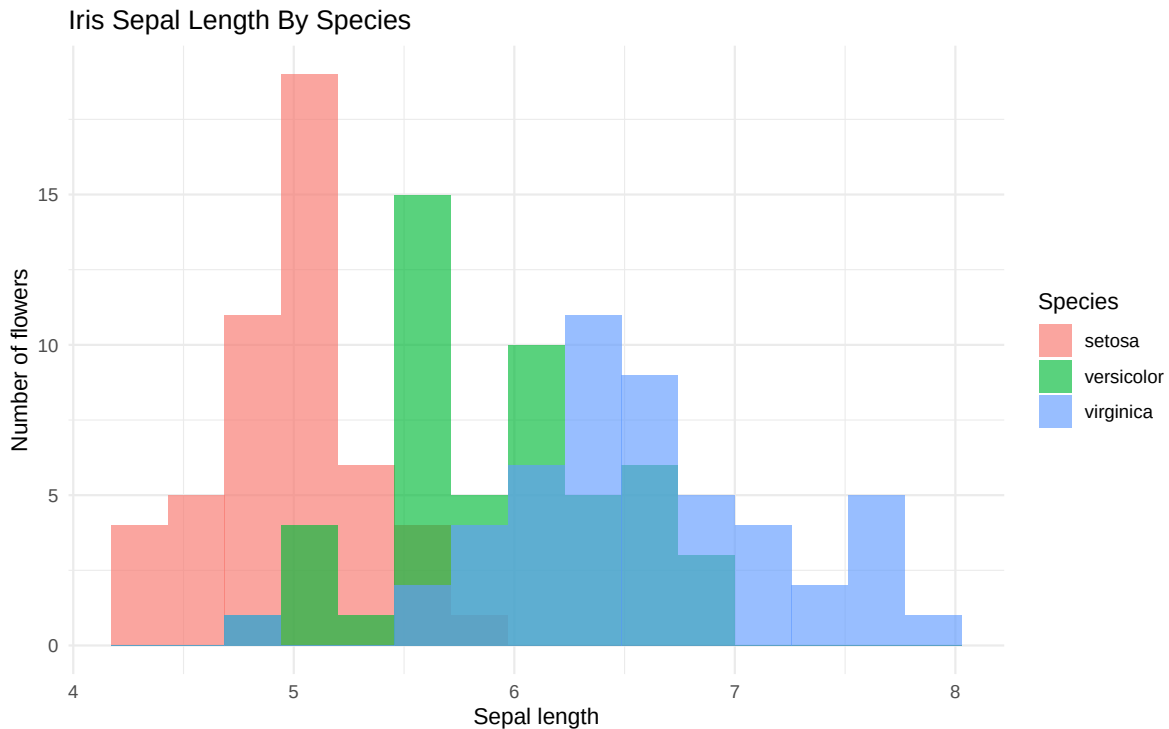
The next three chunks are short exercise-style examples. Each uses a built-in dataset and changes only one or two plotting decisions at a time.

```
iris |>
  ggplot(aes(x = Sepal.Length)) +
  geom_histogram(bins = 15, fill = "#355c7d", color = "white") +
  labs(
    title = "Iris Sepal Length",
    x = "Sepal length",
    y = "Number of flowers"
  ) +
  theme_minimal()
```



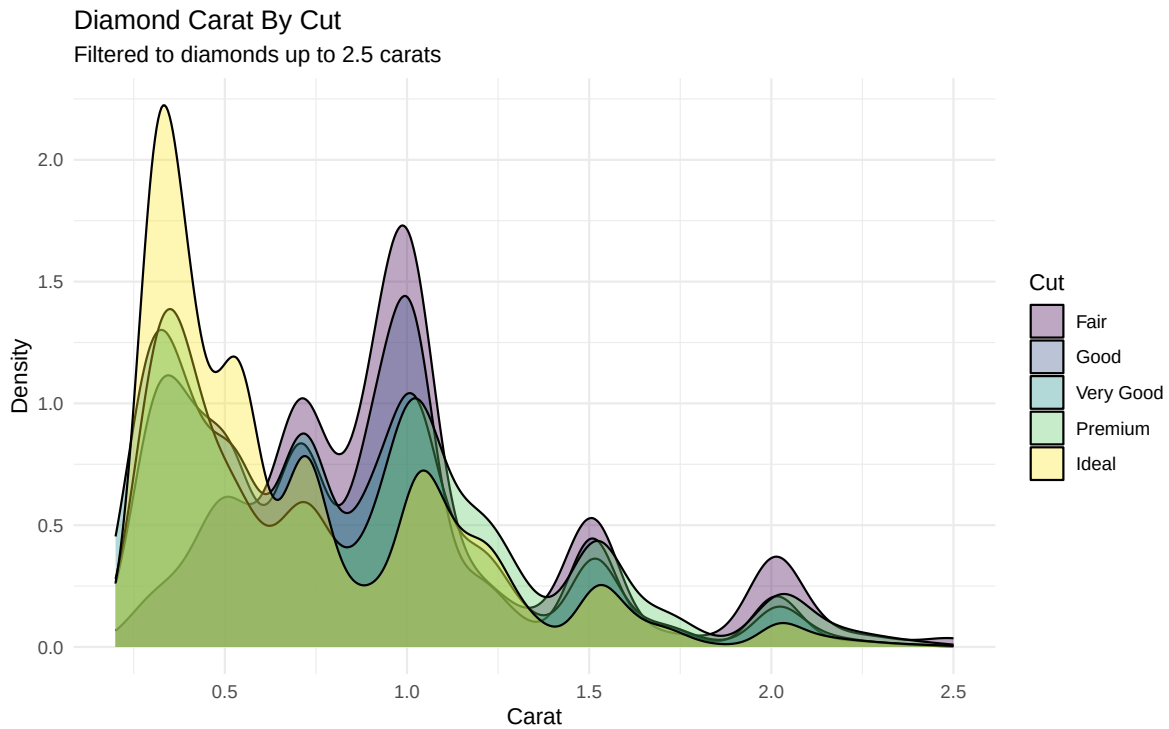
The same distribution can be separated by group. Here, `fill = Species` separates the histogram by species, while `alpha` makes overlapping bars partly transparent.

```
iris |>
  ggplot(aes(x = Sepal.Length, fill = Species)) +
  geom_histogram(bins = 15, alpha = 0.65, position = "identity") +
  labs(
    title = "Iris Sepal Length By Species",
    x = "Sepal length",
    y = "Number of flowers",
    fill = "Species"
  ) +
  theme_minimal()
```

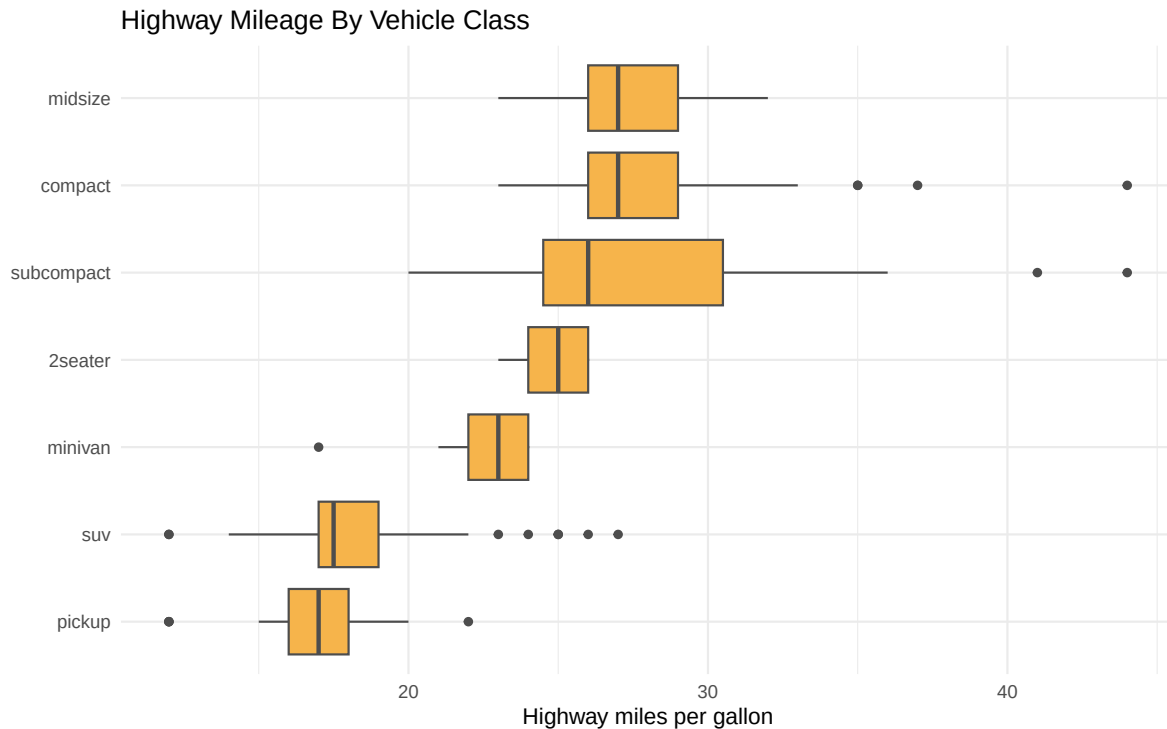
The `diamonds` dataset is larger, so density plots are often easier to compare than overlapping histograms.

```
diamonds |>
  filter(carat <= 2.5) |>
  ggplot(aes(x = carat, fill = cut)) +
  geom_density(alpha = 0.35) +
  labs(
    title = "Diamond Carat By Cut",
    subtitle = "Filtered to diamonds up to 2.5 carats",
    x = "Carat",
    y = "Density",
    fill = "Cut"
  ) +
  theme_minimal()
```



Boxplots are useful when the goal is a compact comparison of medians and spread across categories.

```
mpg |>
  ggplot(aes(x = fct_reorder(class, hwy, .fun = median), y = hwy)) +
  geom_boxplot(fill = "#f6b44b", color = "gray30") +
  coord_flip() +
  labs(
    title = "Highway Mileage By Vehicle Class",
    x = NULL,
    y = "Highway miles per gallon"
  ) +
  theme_minimal()
```



6.14 Extra: PISA score distributions

Individual-level data like PISA is well suited for distribution plots because 152,000 students provide enough data to make the shape of each distribution clearly visible.

```
pisa <- read_csv("Data/pisa/pisa_data_clean.csv", show_col_types = FALSE) |>
  janitor::clean_names()
```

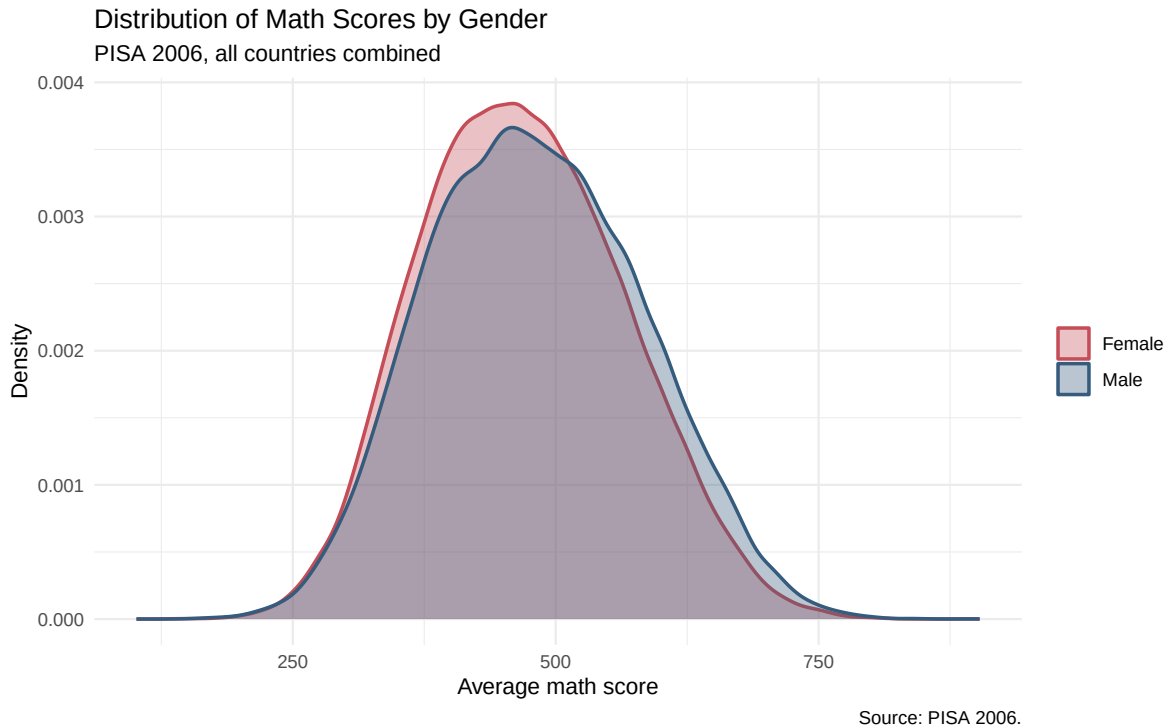
A density plot comparing math score distributions by gender shows the shape of the gap rather than just the means.

```
ggplot(pisa, aes(x = average_math_score, fill = gender, color = gender)) +
  geom_density(alpha = 0.35, linewidth = 0.8) +
  scale_fill_manual(values = c("Female" = "#c44d58", "Male" = "#355c7d")) +
  scale_color_manual(values = c("Female" = "#c44d58", "Male" = "#355c7d")) +
  labs(
    title = "Distribution of Math Scores by Gender",
    subtitle = "PISA 2006, all countries combined",
    x = "Average math score",
```

```

y = "Density",
fill = NULL,
color = NULL,
caption = "Source: PISA 2006."
) +
theme_minimal()

```



The distributions overlap almost entirely. The male distribution sits slightly to the right, but the practical difference is small compared to the within-gender variation.

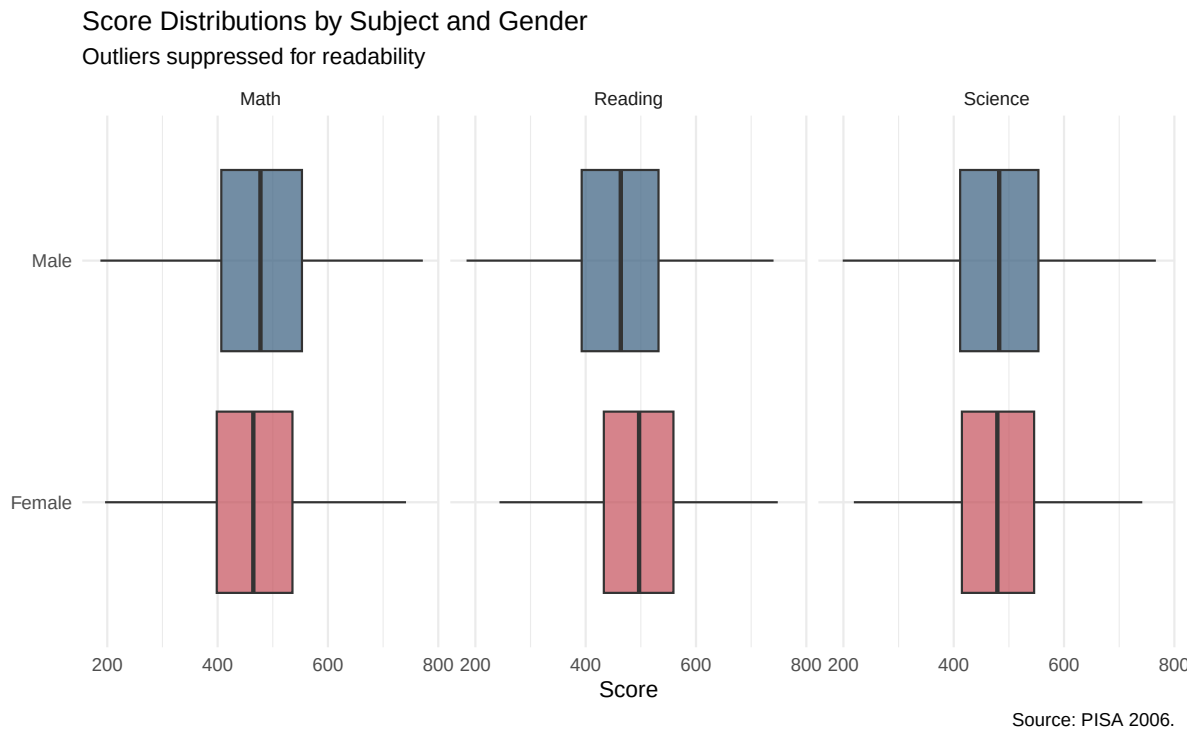
To compare all three subjects at once, pivot to long format and use facets.

```

pisa |>
  select(gender,
    Math    = average_math_score,
    Reading = average_reading_score,
    Science = average_science_score) |>
  pivot_longer(-gender, names_to = "subject", values_to = "score") |>
  ggplot(aes(x = score, y = gender, fill = gender)) +
  geom_boxplot(outliers = FALSE, alpha = 0.7) +
  facet_wrap(~ subject) +

```

```
scale_fill_manual(values = c("Female" = "#c44d58", "Male" = "#355c7d")) +
labs(
  title = "Score Distributions by Subject and Gender",
  subtitle = "Outliers suppressed for readability",
  x = "Score",
  y = NULL,
  caption = "Source: PISA 2006."
) +
theme_minimal() +
theme(legend.position = "none")
```

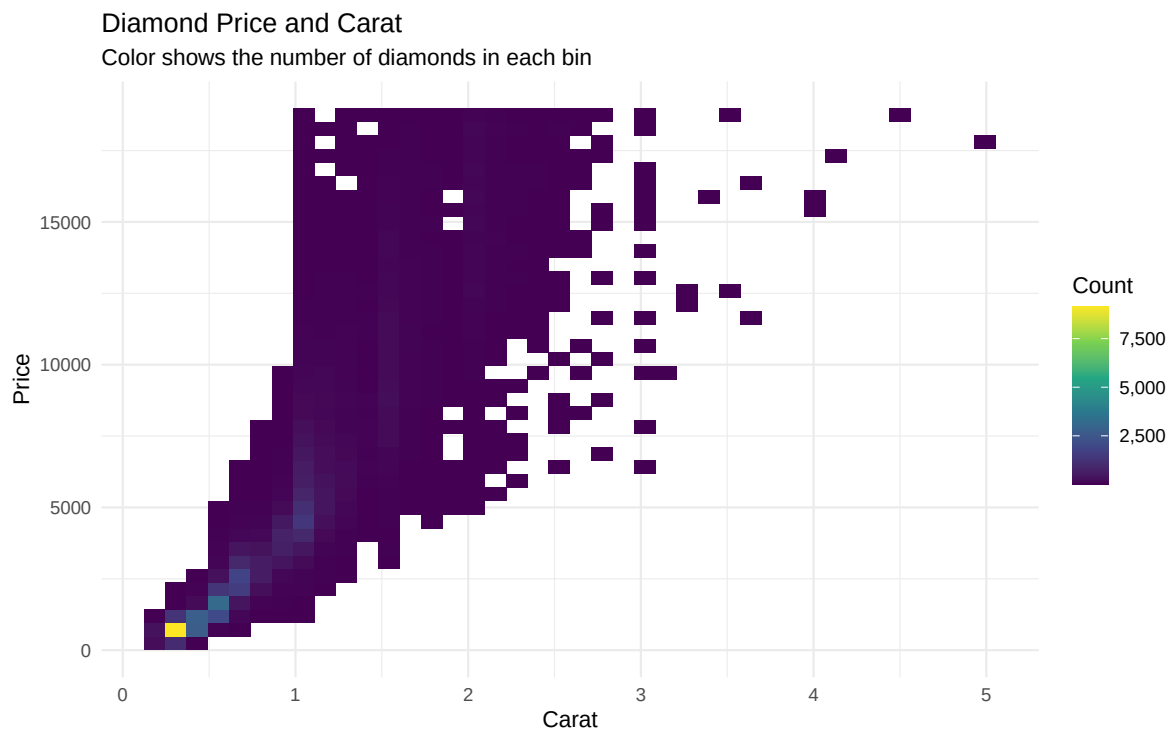


The faceted boxplot reveals the subject-specific pattern: females score higher in reading, males score slightly higher in math, and science scores are nearly equal. Faceting makes this three-way comparison readable in one display.

6.15 Extra: heatmaps for dense two-variable distributions

When there are too many points, a scatterplot can turn into a cloud of overplotting. A two-dimensional bin plot counts observations inside rectangular bins.

```
ggplot(data = diamonds, mapping = aes(x = carat, y = price)) +
  geom_bin2d(bins = 40) +
  scale_fill_viridis_c(labels = comma) +
  labs(
    title = "Diamond Price and Carat",
    subtitle = "Color shows the number of diamonds in each bin",
    x = "Carat",
    y = "Price",
    fill = "Count"
  ) +
  theme_minimal()
```



This is a distribution plot for two variables at once. It is useful when individual points would overlap too much.

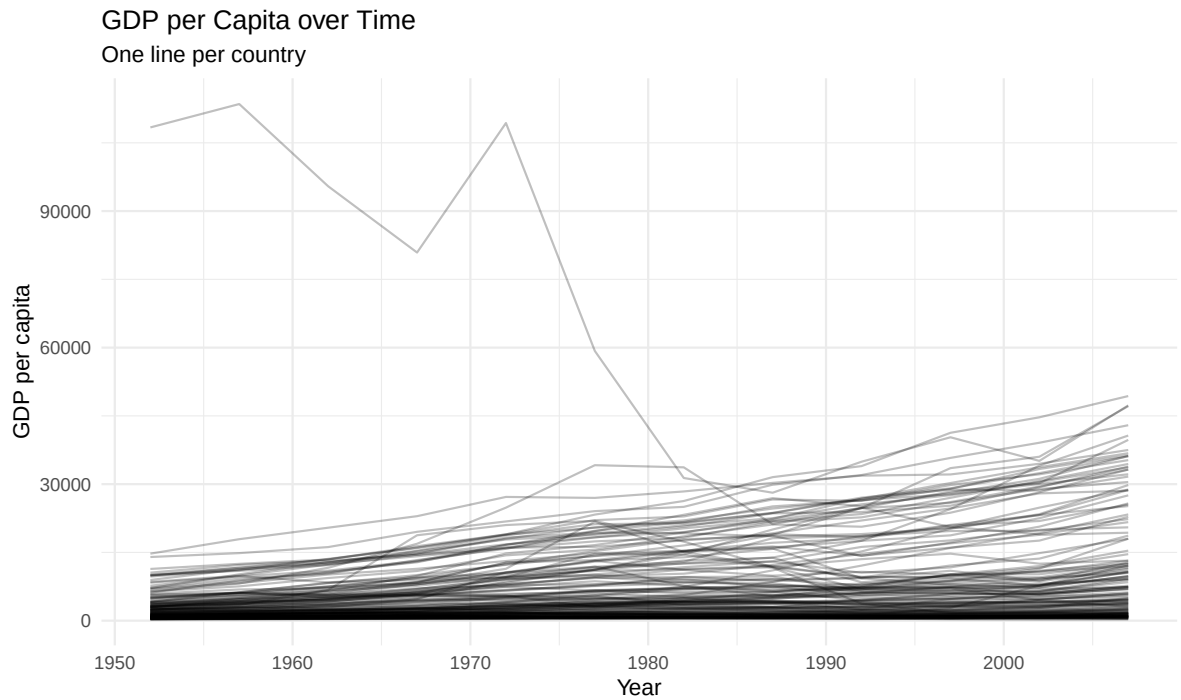
7 Lines, Bars, and Annotation

7.1 Line charts

Line charts are designed for ordered data, especially time. They work best when the x-axis has a meaningful sequence and the line connects observations that should be read as a path.

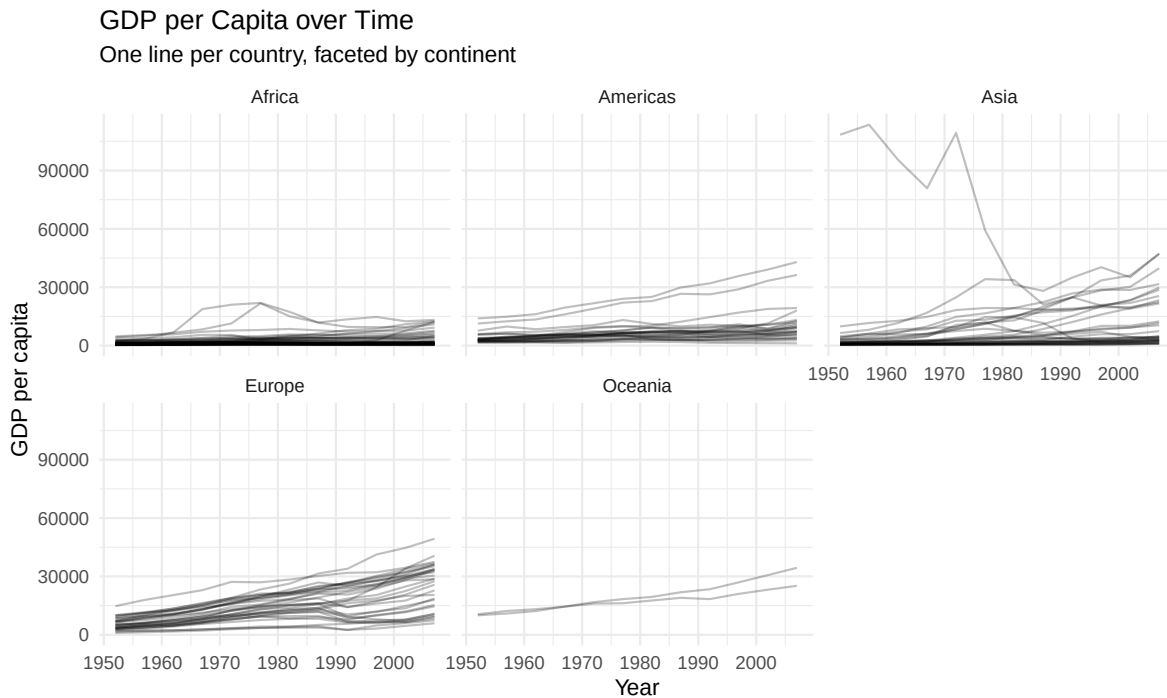
We will begin with GDP per capita over time in Gapminder.

```
ggplot(data = gapminder, mapping = aes(x = year, y = gdpPercap, group = country)) +  
  geom_line(alpha = 0.25) +  
  labs(  
    title = "GDP per Capita over Time",  
    subtitle = "One line per country",  
    x = "Year",  
    y = "GDP per capita",  
    caption = "Source: Gapminder."  
  ) +  
  theme_minimal()
```



This plot has too many lines in one panel. We can separate continents with facets.

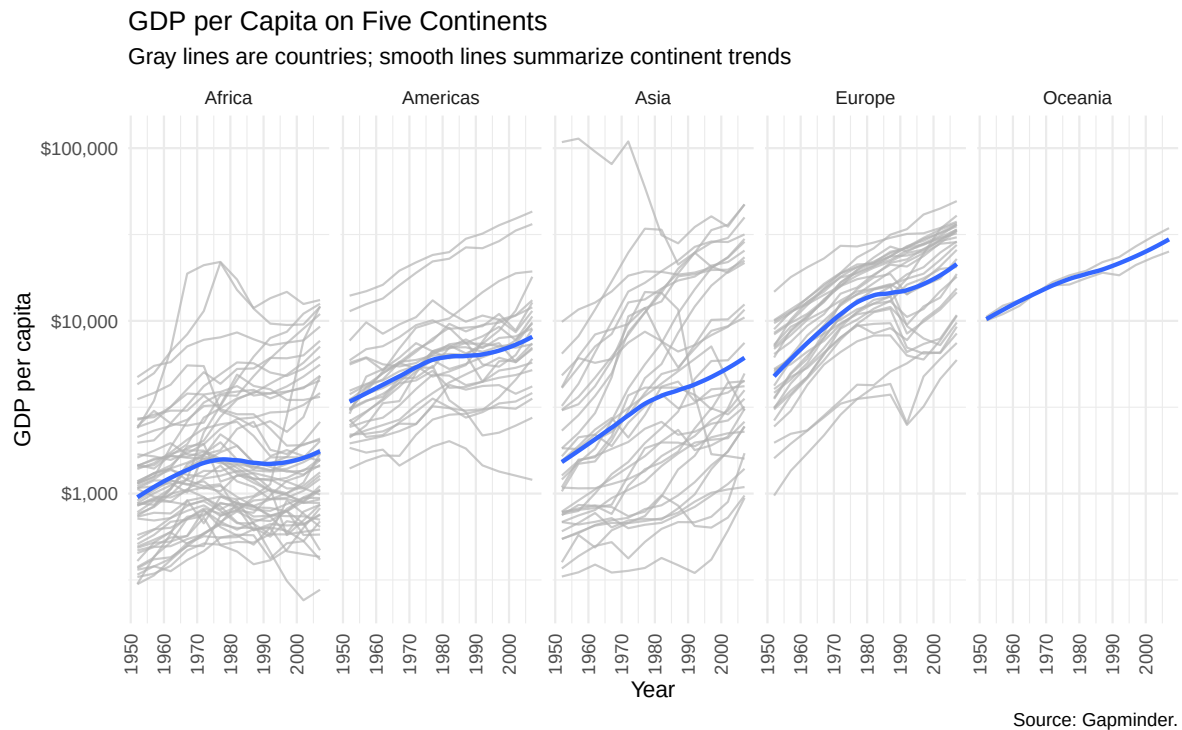
```
ggplot(data = gapminder, mapping = aes(x = year, y = gdpPercap, group = country)) +  
  geom_line(alpha = 0.25) +  
  facet_wrap(~ continent) +  
  labs(  
    title = "GDP per Capita over Time",  
    subtitle = "One line per country, faceted by continent",  
    x = "Year",  
    y = "GDP per capita",  
    caption = "Source: Gapminder."  
  ) +  
  theme_minimal()
```

Kuwait is an extreme case in this dataset. A log scale makes the vertical comparison easier.

```
ggplot(data = gapminder, mapping = aes(x = year, y = gdpPercap, group = country)) +
  geom_line(color = "gray70", alpha = 0.7) +
  geom_smooth(
    mapping = aes(group = continent),
    method = "loess",
    formula = y ~ x,
    se = FALSE,
    linewidth = 1
  ) +
  scale_y_log10(labels = dollar_format(accuracy = 1)) +
  facet_wrap(~ continent, ncol = 5) +
  labs(
    title = "GDP per Capita on Five Continents",
    subtitle = "Gray lines are countries; smooth lines summarize continent trends",
    x = "Year",
    y = "GDP per capita",
    caption = "Source: Gapminder."
  ) +
  theme_minimal() +
```

```
theme(axis.text.x = element_text(angle = 90, vjust = 0.5))
```



7.2 Working with dates

Some datasets store time as actual dates. Others store years as numbers. The `lubridate` package helps convert text or numeric values into date objects.

```
make_date(2026)
```

```
[1] "2026-01-01"
```

```
ymd(c("2009-01-02", "2009 01 03", "Created on 2009 1 6"))
```

```
[1] "2009-01-02" "2009-01-03" "2009-01-06"
```

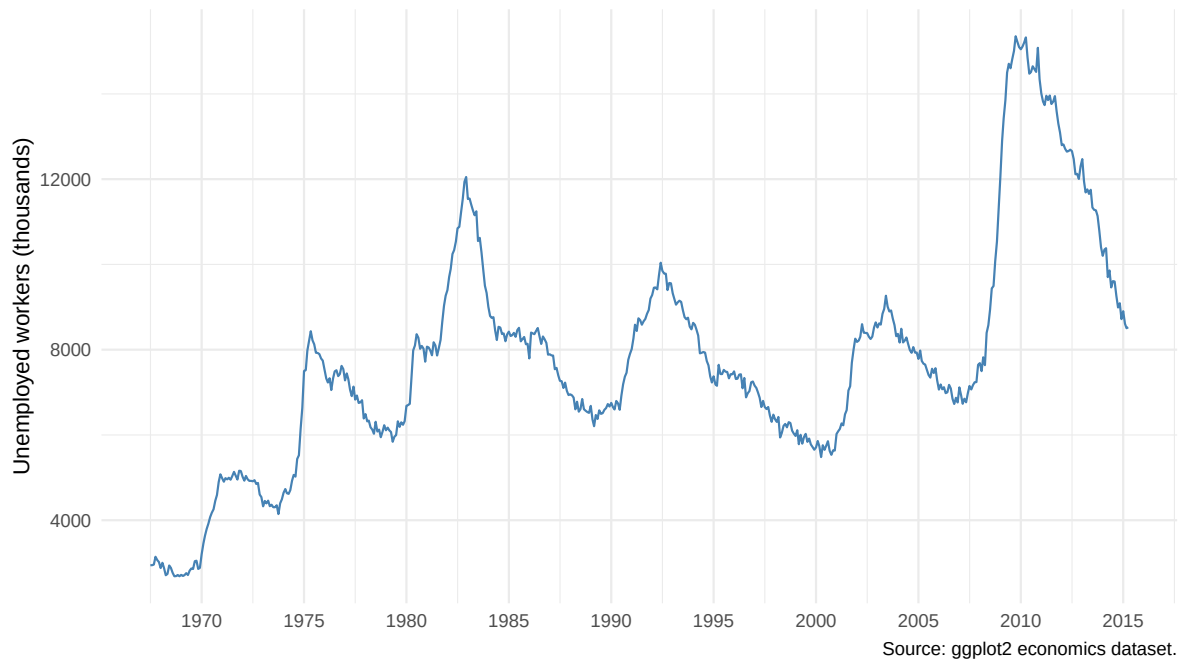
The built-in `economics` dataset has monthly dates.

```
economics |>
  select(date, unemploy, psavert) |>
  slice_head(n = 10)
```

```
# A tibble: 10 x 3
   date      unemploy psavert
  <date>      <dbl>   <dbl>
1 1967-07-01    2944    12.6
2 1967-08-01    2945    12.6
3 1967-09-01    2958    11.9
4 1967-10-01    3143    12.9
5 1967-11-01    3066    12.8
6 1967-12-01    3018    11.8
7 1968-01-01    2878    11.7
8 1968-02-01    3001    12.3
9 1968-03-01    2877    11.7
10 1968-04-01    2709    12.3
```

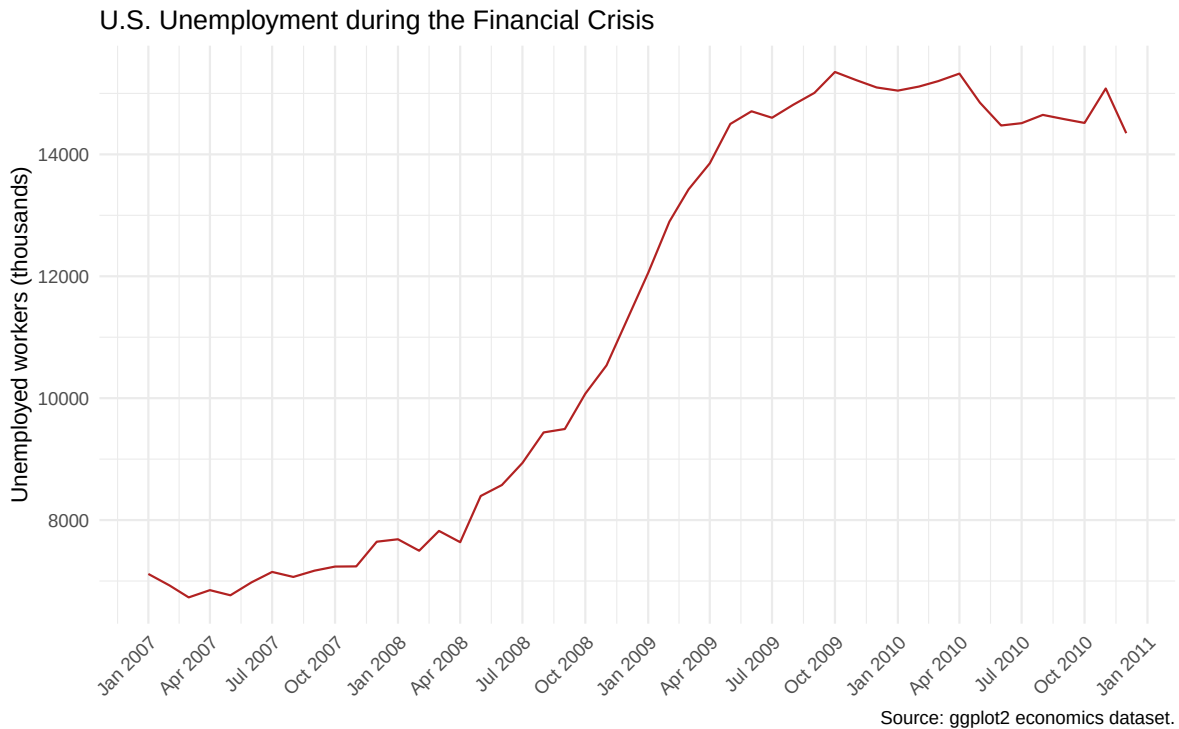
```
ggplot(data = economics, mapping = aes(x = date, y = unemploy)) +
  geom_line(color = "steelblue") +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y") +
  labs(
    title = "U.S. Unemployment",
    subtitle = "Monthly observations, 1967-2015",
    x = NULL,
    y = "Unemployed workers (thousands)",
    caption = "Source: ggplot2 economics dataset."
  ) +
  theme_minimal()
```

U.S. Unemployment
Monthly observations, 1967–2015



For a shorter period, use denser date labels.

```
economics |>
  filter(date >= "2007-01-01", date <= "2010-12-31") |>
  ggplot(mapping = aes(x = date, y = unemploy)) +
  geom_line(color = "firebrick") +
  scale_x_date(date_breaks = "3 months", date_labels = "%b %Y") +
  labs(
    title = "U.S. Unemployment during the Financial Crisis",
    x = NULL,
    y = "Unemployed workers (thousands)",
    caption = "Source: ggplot2 economics dataset."
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



7.3 Highlighting trends

Line charts often need highlighting. A common strategy is to show all lines in gray and draw the important line in a stronger color.

The state policy dataset is useful for this because it has repeated state-year observations. The examples below use unemployment rates by state.

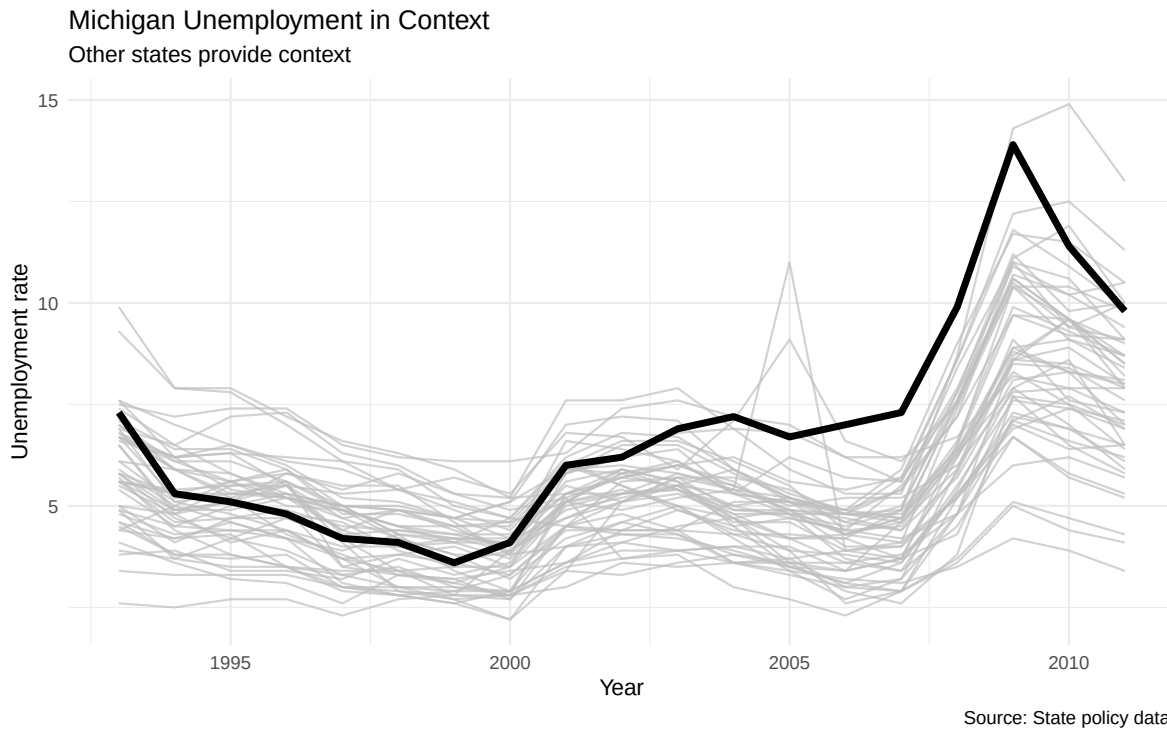
```
state_policy_panel <- haven::read_dta("Data/state_policy/state_data_1993_2016.dta") |>
  mutate(st = as.character(st)) |>
  filter(!is.na(unemploy))

state_policy_panel |>
  select(st, year, unemploy) |>
  head(10)
```

```
# A tibble: 10 x 3
  st      year unemploy
<chr> <dbl>   <dbl>
1 AK     1993     7.5
```

2	AK	1994	7.20
3	AK	1995	7.40
4	AK	1996	7.40
5	AK	1997	6.5
6	AK	1998	6.20
7	AK	1999	6.10
8	AK	2000	6.10
9	AK	2001	6.30
10	AK	2002	7.40

```
ggplot(data = state_policy_panel, mapping = aes(x = year, y = unemploy, group = st)) +
  geom_line(color = "gray75", alpha = 0.7) +
  geom_line(
    data = state_policy_panel |> filter(st == "MI"),
    color = "black",
    linewidth = 1.6
  ) +
  labs(
    title = "Michigan Unemployment in Context",
    subtitle = "Other states provide context",
    x = "Year",
    y = "Unemployment rate",
    caption = "Source: State policy data."
  ) +
  theme_minimal()
```

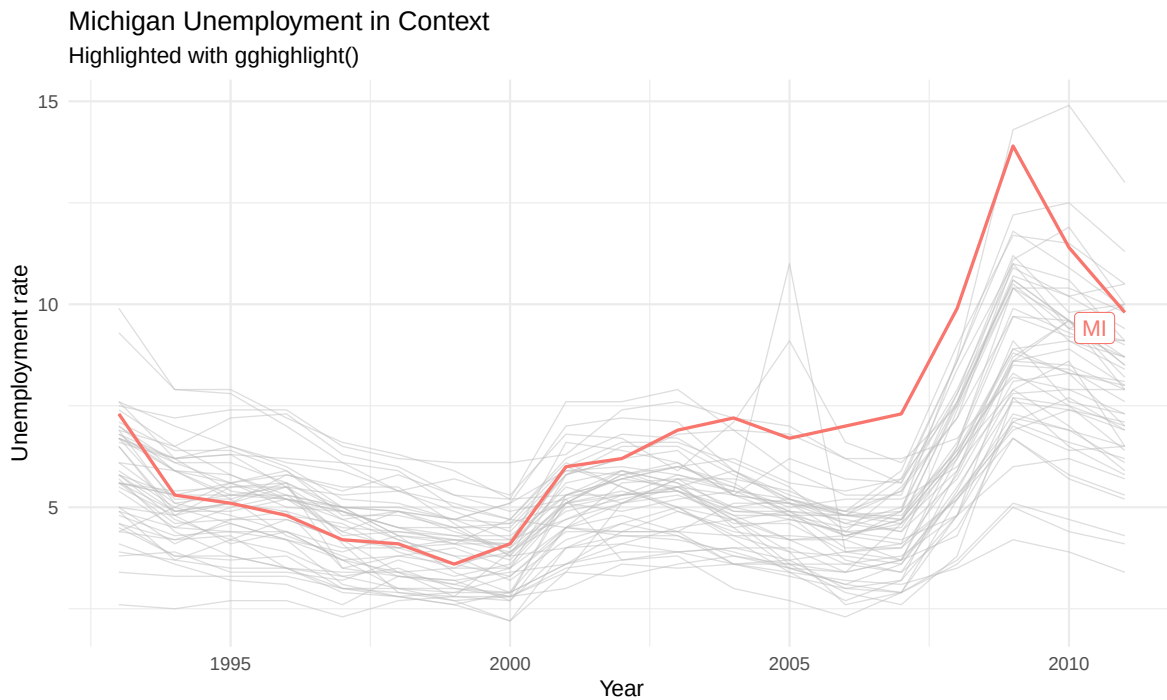


The plot above identifies Michigan visually but not by name. Chapter 8 returns to manual direct labels in a fuller workflow. Here, the shorter option is `gghighlight()`.

The `gghighlight` package automates highlighting and direct labeling when the highlighting rule can be stated as a logical condition.

```
ggplot(data = state_policy_panel, mapping = aes(x = year, y = unemploy, color = st)) +
  geom_line(linewidth = 0.75) +
  gghighlight(
    st == "MI",
    use_group_by = FALSE,
    label_key = st,
    unhighlighted_params = list(linewidth = 0.3, alpha = 0.5)
  ) +
  labs(
    title = "Michigan Unemployment in Context",
    subtitle = "Highlighted with gghighlight()",
    x = "Year",
    y = "Unemployment rate",
    caption = "Source: State policy data."
  ) +
```

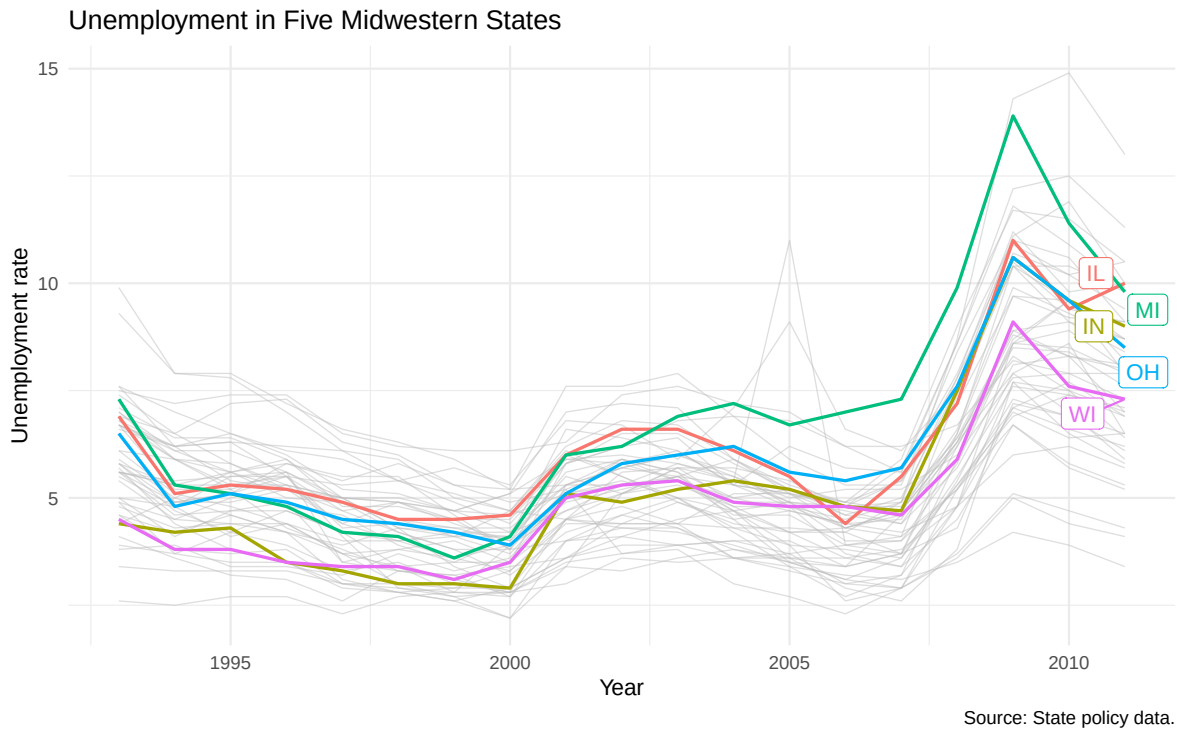
```
theme_minimal() +
theme(legend.position = "none")
```



Source: State policy data.

We can also highlight a set of cases. The condition can name several states with `%in%`.

```
ggplot(data = state_policy_panel, mapping = aes(x = year, y = unemploy, color = st)) +
  geom_line(linewidth = 0.75) +
  gghighlight(
    st %in% c("IL", "IN", "MI", "OH", "WI"),
    use_group_by = FALSE,
    label_key = st,
    unhighlighted_params = list(linewidth = 0.3, alpha = 0.5)
  ) +
  labs(
    title = "Unemployment in Five Midwestern States",
    x = "Year",
    y = "Unemployment rate",
    caption = "Source: State policy data."
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```

7.4 Bar charts

Bar charts are useful for comparing quantities across categories. When the data is already summarized, use `geom_col()`.

The `midwest` dataset has county-level observations. To compare average college education by state, we first summarize.

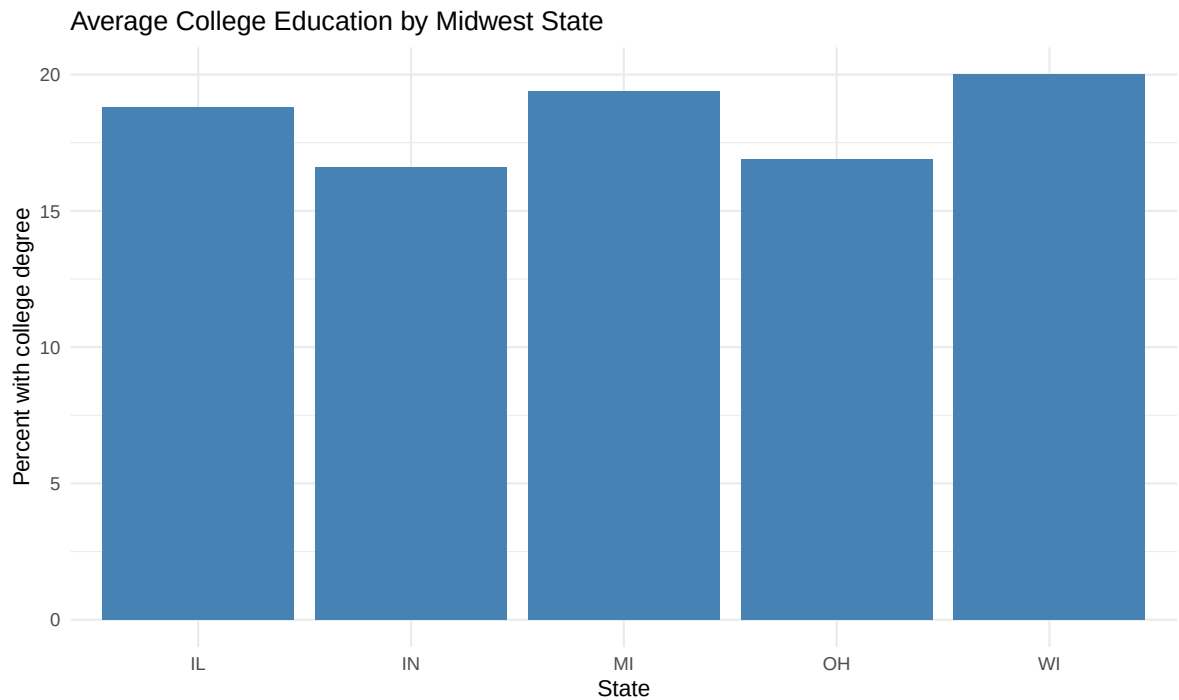
```
college_by_state <- midwest |>
  filter(!is.na(percollege)) |>
  group_by(state) |>
  summarize(
    avg_college = round(mean(percollege), 1)
  )
```

```
college_by_state
```

```
# A tibble: 5 x 2
  state avg_college
```

	<chr>	<dbl>
1	IL	18.8
2	IN	16.6
3	MI	19.4
4	OH	16.9
5	WI	20

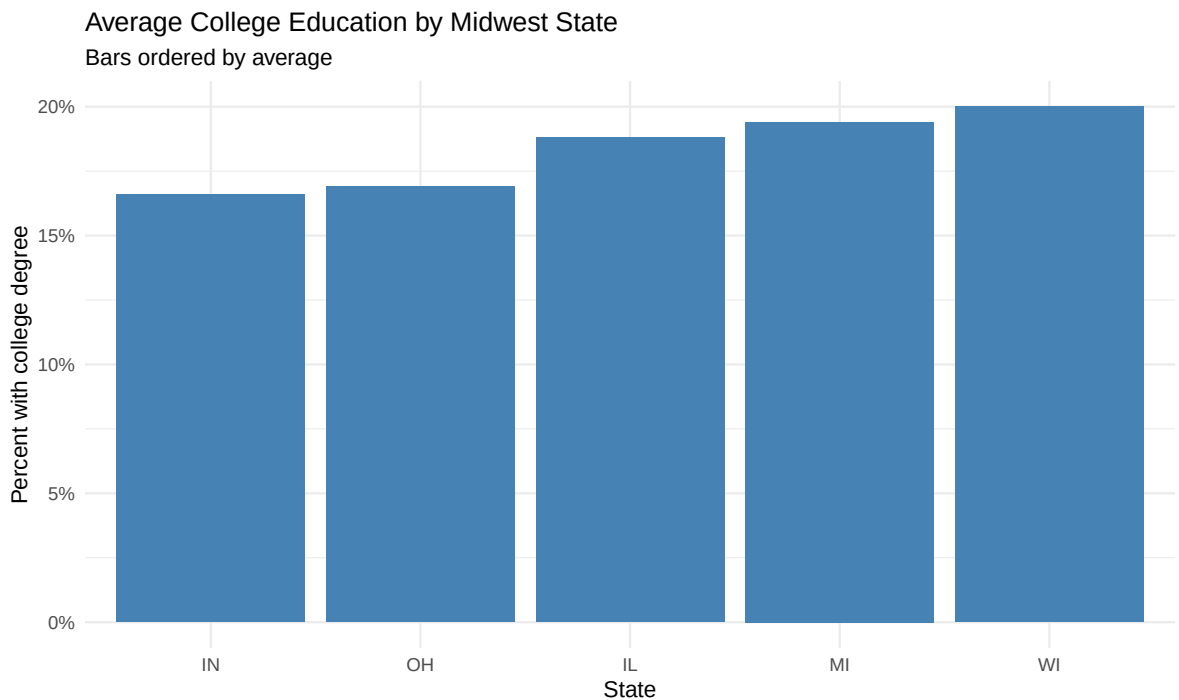
```
ggplot(data = college_by_state, mapping = aes(x = state, y = avg_college)) +
  geom_col(fill = "steelblue") +
  labs(
    title = "Average College Education by Midwest State",
    x = "State",
    y = "Percent with college degree",
    caption = "Source: ggplot2 midwest dataset."
  ) +
  theme_minimal()
```



Source: ggplot2 midwest dataset.

Ordering the bars by value makes comparison easier.

```
ggplot(data = college_by_state, mapping = aes(x = reorder(state, avg_college), y = avg_college)) +
  geom_col(fill = "steelblue") +
  scale_y_continuous(labels = percent_format(scale = 1)) +
  labs(
    title = "Average College Education by Midwest State",
    subtitle = "Bars ordered by average",
    x = "State",
    y = "Percent with college degree",
    caption = "Source: ggplot2 midwest dataset."
  ) +
  theme_minimal()
```

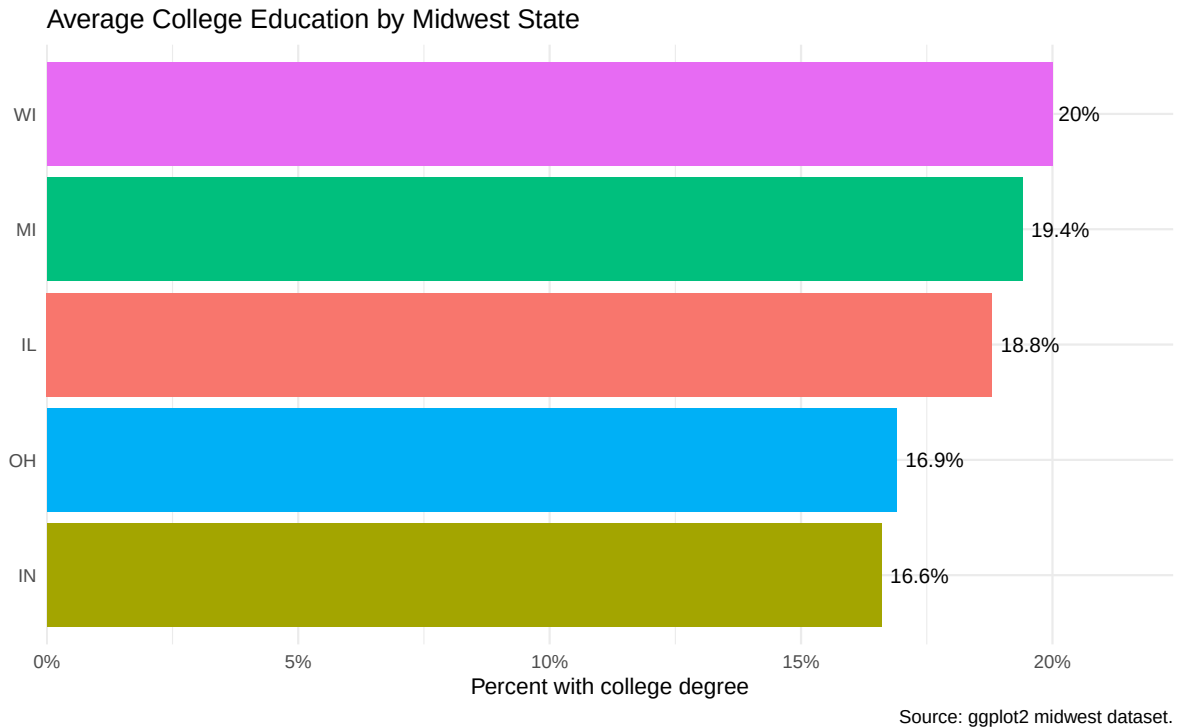


Source: ggplot2 midwest dataset.

If category labels are long, `coord_flip()` often helps.

```
ggplot(data = college_by_state, mapping = aes(x = reorder(state, avg_college), y = avg_college)) +
  geom_col(aes(fill = state), show.legend = FALSE) +
  geom_text(aes(label = str_c(avg_college, "%")), hjust = -0.15, size = 3.5) +
  coord_flip() +
  scale_y_continuous(labels = percent_format(scale = 1), expand = expansion(mult = c(0, 0.12))) +
  labs(
    title = "Average College Education by Midwest State",
```

```
x = NULL,
y = "Percent with college degree",
caption = "Source: ggplot2 midwest dataset."
) +
theme_minimal()
```



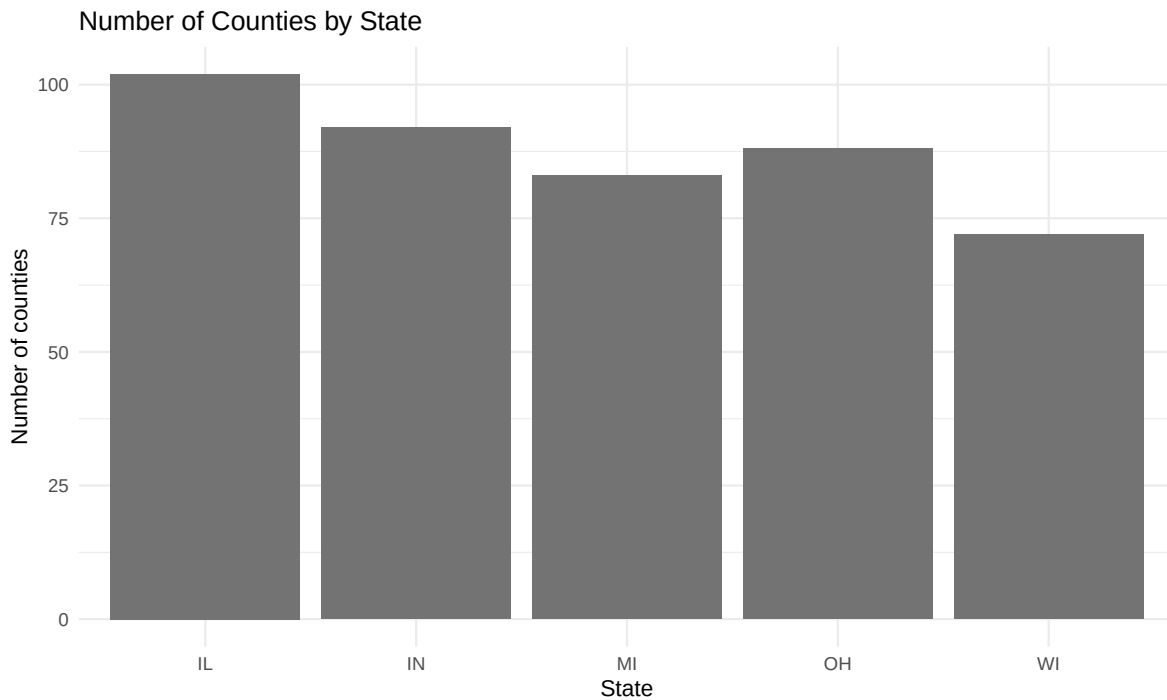
7.5 geom_col() versus geom_bar()

Use `geom_col()` when you already have the heights of the bars.

Use `geom_bar()` when you want ggplot2 to count rows.

```
ggplot(data = midwest, mapping = aes(x = state)) +
  geom_bar(fill = "gray45") +
  labs(
    title = "Number of Counties by State",
    x = "State",
    y = "Number of counties",
    caption = "Source: ggplot2 midwest dataset."
  )
```

```
) +  
theme_minimal()
```



Source: ggplot2 midwest dataset.

Here, no y variable is mapped. `geom_bar()` counts the number of rows in each state.

7.6 Dodged bars

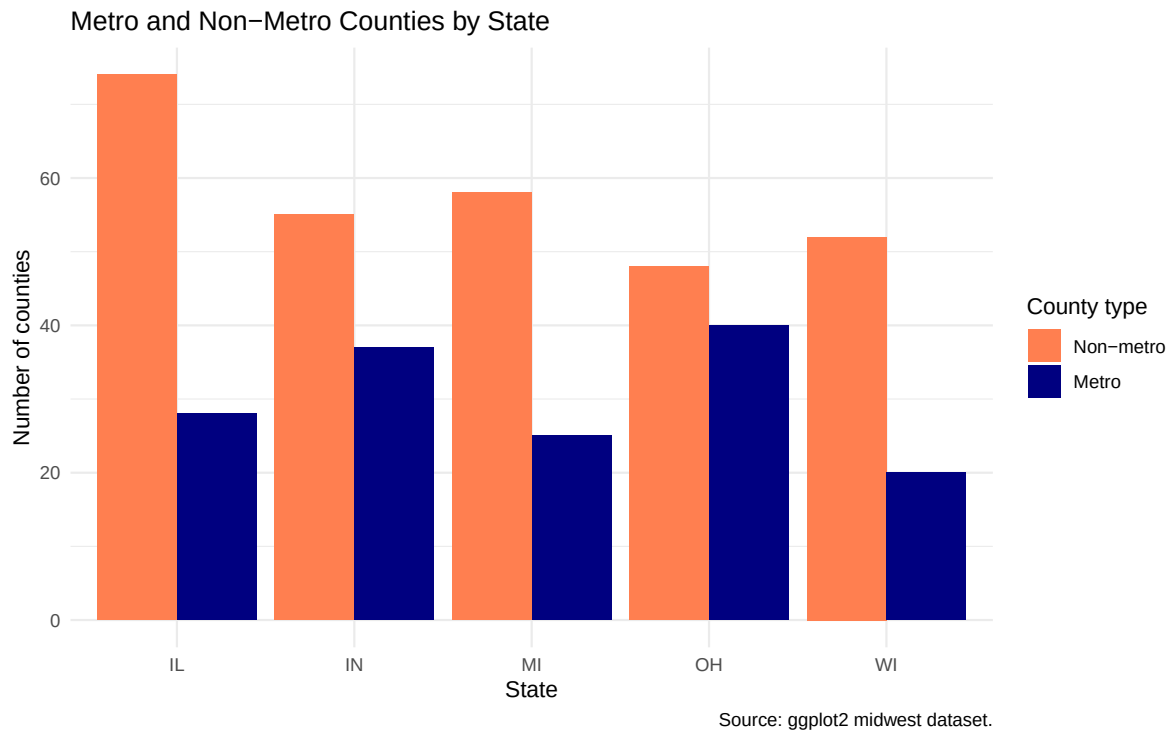
Dodged bars place bars side by side within each category. Here we compare metro and non-metro counties within states.

```
ggplot(data = midwest, mapping = aes(x = state, fill = factor(inmetro))) +  
  geom_bar(position = "dodge") +  
  scale_fill_manual(  
    values = c("0" = "coral", "1" = "navy"),  
    labels = c("0" = "Non-metro", "1" = "Metro")  
  ) +  
  labs(  
    title = "Metro and Non-Metro Counties by State",  
    x = "State",
```

```

y = "Number of counties",
fill = "County type",
caption = "Source: ggplot2 midwest dataset."
) +
theme_minimal()

```



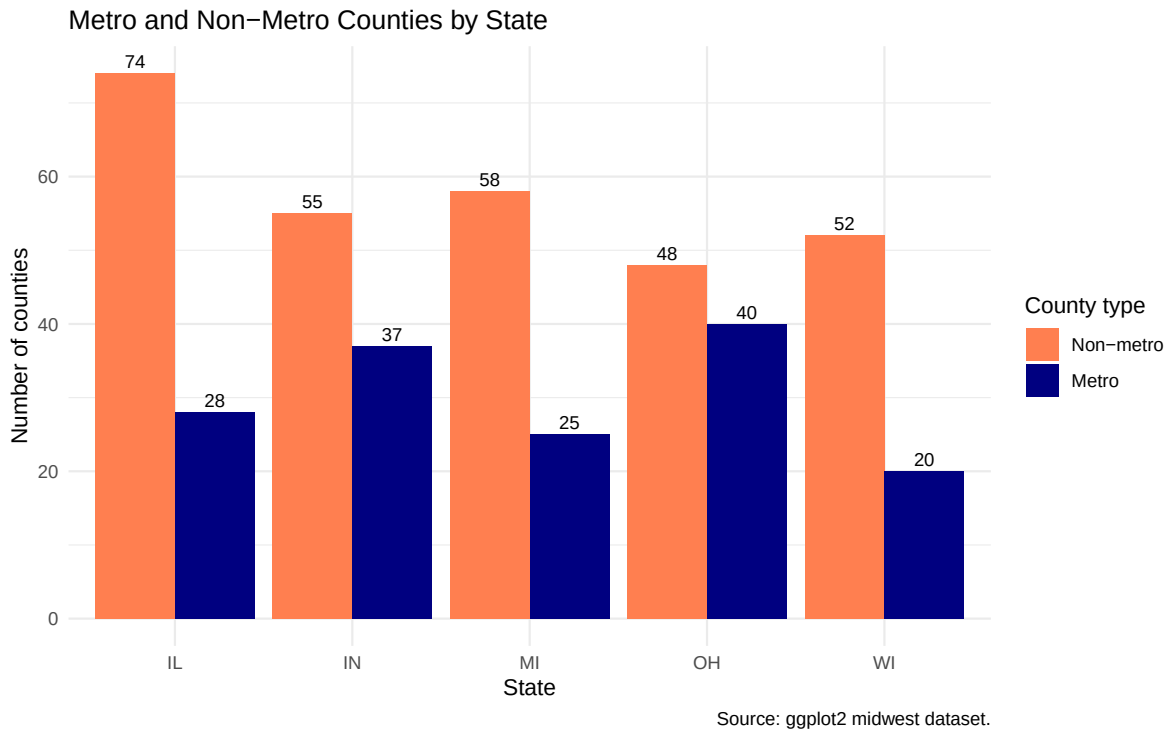
Adding labels to dodged bars requires the text to use the same dodging.

```

ggplot(data = midwest, mapping = aes(x = state, fill = factor(inmetro))) +
  geom_bar(position = position_dodge(width = 0.9)) +
  geom_text(
    mapping = aes(label = after_stat(count), group = factor(inmetro)),
    stat = "count",
    position = position_dodge(width = 0.9),
    vjust = -0.4,
    size = 3
  ) +
  scale_fill_manual(
    values = c("0" = "coral", "1" = "navy"),
    labels = c("0" = "Non-metro", "1" = "Metro")
  ) +

```

```
labs(
  title = "Metro and Non-Metro Counties by State",
  x = "State",
  y = "Number of counties",
  fill = "County type",
  caption = "Source: ggplot2 midwest dataset."
) +
theme_minimal()
```



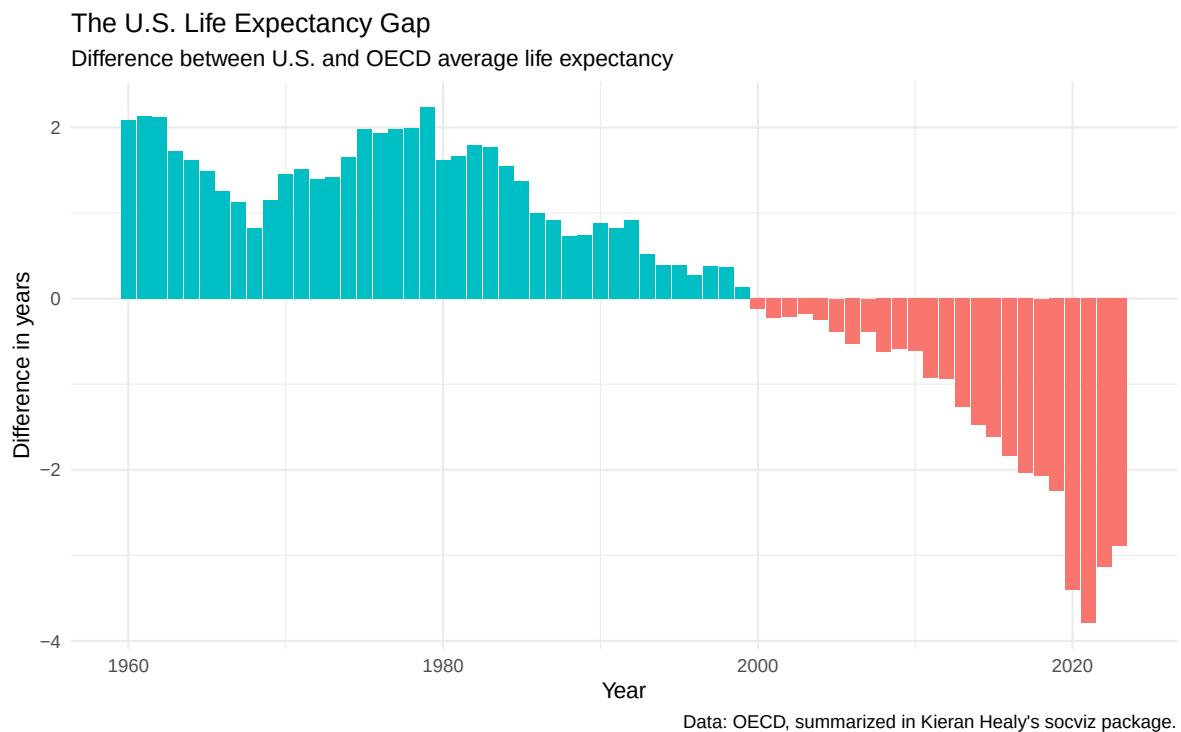
7.7 Bar chart example: OECD life expectancy gap

A local copy of the `oecd_sum` summary lives at `Data/oecd/oecd_sum.csv`. Each row is one year, with columns for U.S. life expectancy, the OECD non-U.S. average, the difference, and a `hi_lo` indicator for whether the U.S. is above or below the average.

```
oecd_sum <- read_csv("Data/oecd/oecd_sum.csv", show_col_types = FALSE)

ggplot(data = oecd_sum, mapping = aes(x = year, y = diff, fill = hi_lo)) +
  geom_col() +
```

```
guides(fill = "none") +
labs(
  title = "The U.S. Life Expectancy Gap",
  subtitle = "Difference between U.S. and OECD average life expectancy",
  x = "Year",
  y = "Difference in years",
  caption = "Data: OECD, summarized in Kieran Healy's socviz package."
) +
theme_minimal()
```



This is a bar chart because the main comparison is the size and direction of a yearly difference.

7.8 Exercise

Create an ordered bar chart from a summarized dataset.

1. Use `midwest`.
2. Group by `state`.
3. Calculate the median of `percollege`.
4. Make an ordered `geom_col()` chart.

5. Add value labels.
6. Remove any redundant legend.

```
# Write your plot here.
```

7.9 Extra: slopegraphs

Slopegraphs compare the same units at a small number of time points. They are especially useful when the start and end values matter more than the intermediate path. The form was popularized by Edward Tufte in *The Visual Display of Quantitative Information*, but the design predates him by a century.

A generic Tufte-style slopegraph:

Ben Fry's slopegraph of baseball salary per win:

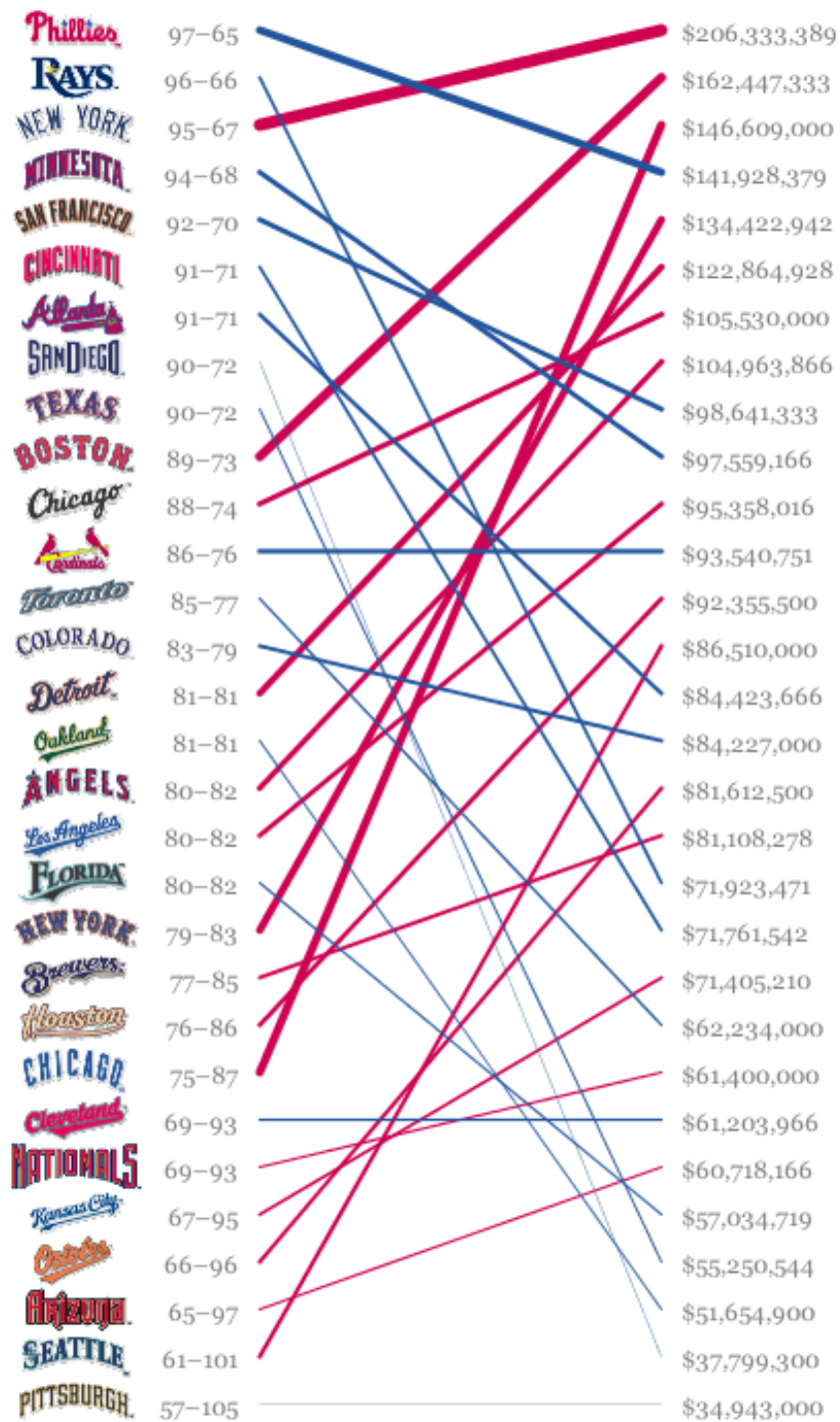


Figure 7.1: Baseball slopegraph by Ben Fry

And one from *Scribner's Statistical Atlas of the United States* in 1883, well before the modern term existed:

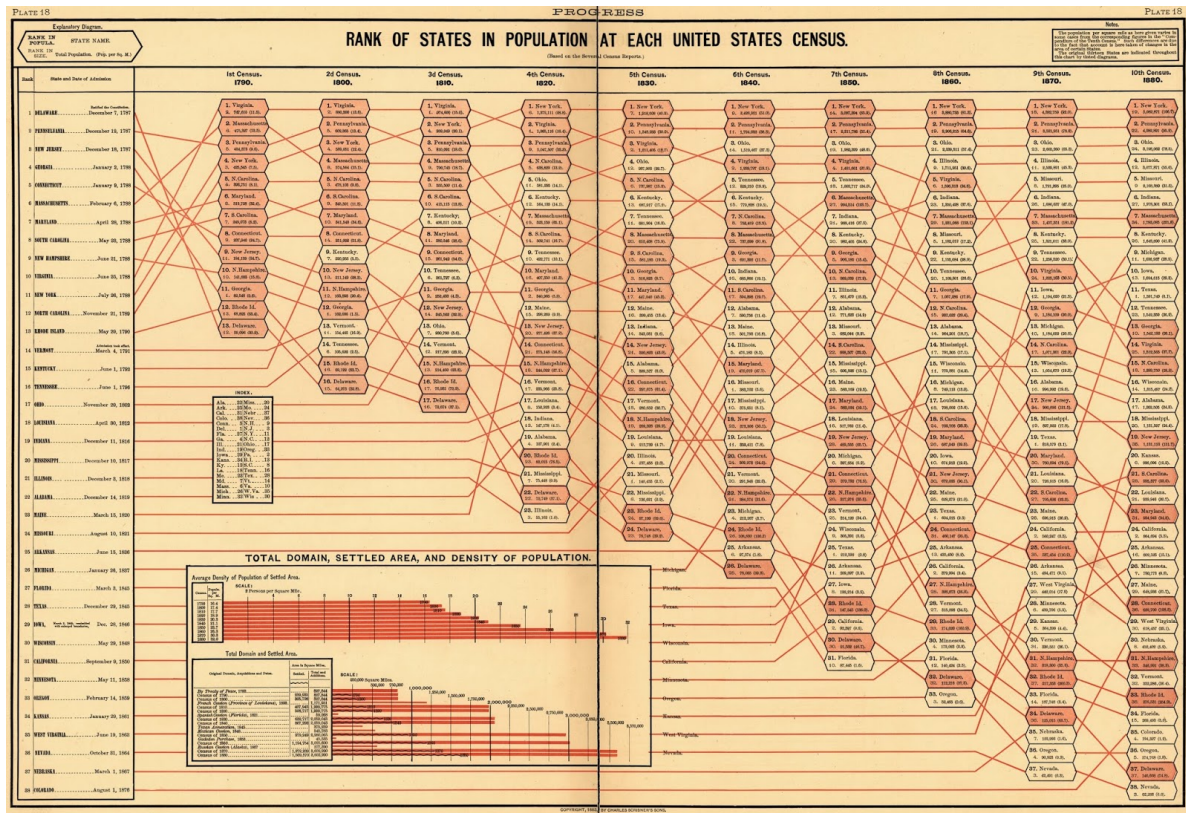


Figure 7.2: Scribner's Statistical Atlas, 1883

Unlike a traditional line chart, a slopegraph emphasizes the start and end values of each unit, so change is read as the slope of a line rather than as a path through many points.

The states table is a local snapshot of Correlates of State Policy data, originally fetched with `cspp::get_cspp_data()`. `polconserv` is policy conservatism (the negation of `pollib_median`), so larger values mean a more conservative policy environment.

```
states <- read_csv("Data/state_policy/cspp_states.csv", show_col_types = FALSE)

library(ggslopegraph)

ggslopegraph(
  dataframe = states |>
    mutate(year = as.character(year)) |>
```

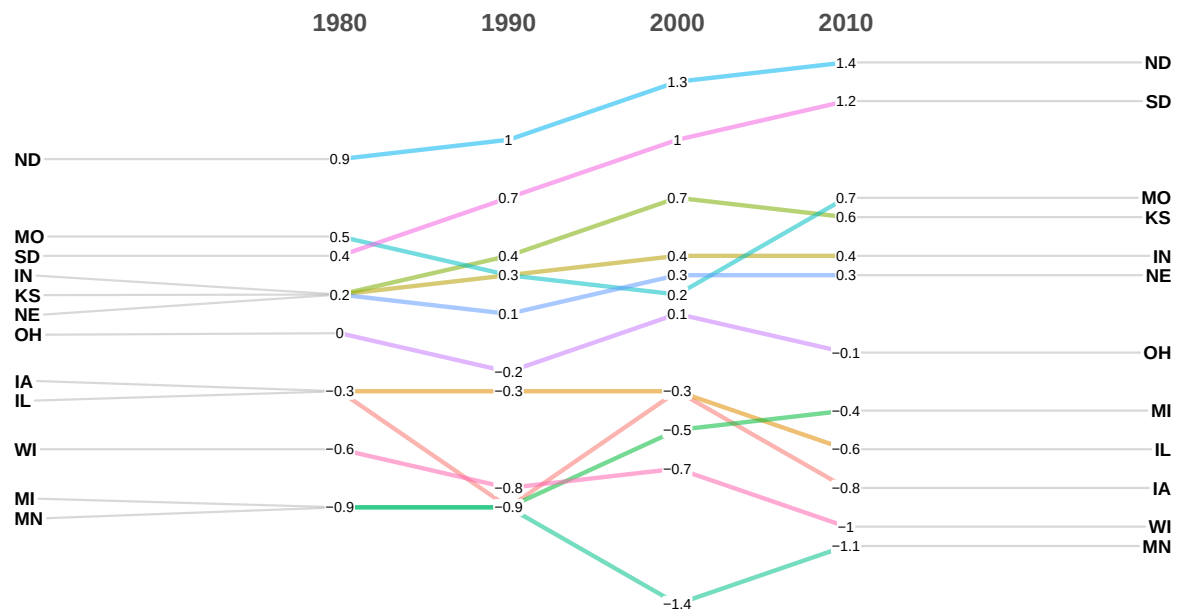
```

  filter(year %in% c("1980", "1990", "2000", "2010"), region.name == "midwest"),
  Times = year,
  Measurement = polconserv,
  Grouping = st,
  Title = "Tufte-style Slopegraph of Policy Conservatism",
  SubTitle = "Midwest, 1980-2010",
  Caption = "Source: Correlates of State Policy via the cspp package."
)

```

Tufte-style Slopegraph of Policy Conservatism

Midwest, 1980-2010



Source: Correlates of State Policy via the cspp package.

`ggslopegraph()` requires the `Times` column to be a character or factor, which is why `year` is converted before filtering. If `ggslopegraph` is not available, the same plot can be built with plain `geom_line()` and `geom_text()`.